Dokumentasi Resmi & Sintaks C#

Bahasa: C# | Total Chunk/Dokumen: 174 | Disiapkan untuk RAG System

[csharp_001] C# - Installation & Setup

Install .NET on macOS: Overview

Sumber: ht

Install .NET on macOS
[!div class="op_single_selector"]
- Install on Windows
- Install on macOS
- Install on Linux

This article teaches you about which versions of .NET are supported on macOS, how to install .NET, and what the difference is between the SDK and runtime.

The latest version of .NET is 10.
[!div class="button"]
Download .NET

[csharp_002] C# - Installation & Setup

Install .NET on macOS: Supported versions

Sumber: ht

Supported versions

The following table lists the supported .NET releases, and which macOS they're supported on. These versions remain supported until either the version of .NET reaches end-of-support or the version of macOS is no longer supported.

macOS Version	.NET
macOS 26 "Tahoe"	10.0, 9.0, 8.0
macOS 15 "Sequoia"	10.0, 9.0, 8.0
macOS 14 "Sonoma"	10.0, 9.0, 8.0

!INCLUDE [versions-not-supported]

[csharp_003] C# - Installation & Setup

Install .NET on macOS: Runtime or SDK

Sumber: ht

Runtime or SDK

The **runtime** is used to run apps created with .NET. When an app author publishes an app, they can include the runtime with their app. If they don't include the runtime, it's up to the user to install the correct runtime.

There are two runtimes you can install on macOS, and both are included with the SDK.

- _ASP.NET Core Runtime_\
  Runs ASP.NET Core apps. Includes the .NET runtime. **Not available as an installer.**

- _.NET Runtime_\
  This runs normal .NET apps, but not specialized apps, such as apps built on ASP.NET Core.

The **SDK** is used to build and publish .NET apps and libraries. The latest SDK supports building apps for previous versions of .NET. In normal circumstances, you would only need the latest SDK installed.

Installing the SDK includes both the standard .NET Runtime and the ASP.NET Core Runtime. For example, if you have .NET SDK 9.0 installed, then .NET Runtime 9.0 and ASP.NET Core 9.0 Runtime are both installed. However, any other runtime version wouldn't be installed with the SDK and would require you to install it separately.

[csharp_004] C# - Installation & Setup

Install .NET on macOS: Choose how to install .NET

Sumber: ht

Choose how to install .NET

There are different ways to install .NET, and some products might manage their own version of .NET. If you install .NET through software that manages its own version of .NET, it might not be enabled system-wide. Make sure you understand the implications of installing .NET through other software.

If you're unsure which method you should choose after reviewing the lists in the following sections, you probably want to use the .NET Installer package.

Developers

- Visual Studio Code - C# Dev Kit

  Install the **C# Dev Kit** extension for Visual Studio Code to develop .NET apps. The extension can use an already installed SDK or install one for you.

Users and Developers

- .NET Installer

  Use the standalone installer to install .NET. This method is the typical way to install .NET on your developer or user machine.

- Install .NET with a script

  A bash script that can automate the install of the SDK or Runtime. You can choose which version of .NET to install.

- Install .NET manually

  Use this installation method when you need to install .NET to a specific folder, and run it apart from other copies of .NET.

[csharp_005] C# - Installation & Setup

Install .NET on macOS: Install .NET

Sumber: ht

Install .NET

Installer packages are available for macOS, an easy way of installing .NET.

01. Open a browser and navigate to <https://dotnet.microsoft.com/download/dotnet>.
01. Select the link to the .NET version you want to install, such as **.NET 10.0**.

:::image type="content" source="media/macos/download-page.png" alt-text="The .NET download website. Versions 6.0 through 9.0 are listed. A red box highlights those download links.":::

This link brings you to the page with links to download that version of .NET

If you're going to install the SDK, choose the latest .NET version. The SDK supports building apps for previous versions of .NET.
If you're unsure which version to download, choose the version marked **latest**.

01. This page presents the download links for the SDK and the Runtime. Here you download the .NET SDK or .NET Runtime.

:::image type="content" source="media/macos/sdk-runtime-download-page.png" alt-text="The .NET download website showing the SDK and Runtime download links. The SDK and Runtime headers are highlighted with a red box. Each box has an arrow pointing down to the macOS section.":::

There are two sections highlighted in the previous image. If you're downloading the SDK, refer to section 1. For the .NET Runtime, refer to section 2.

- Section 1 (SDK)

This section is the SDK download area. Under the **Installers** column for the **macOS** row, two architectures are listed: **Arm64** and **x64**.

- If you're running an Apple processor, such as an **M1** or an **M3 Pro**, select **Arm64**.
- If you're running an Intel processor, select **x64**.

- Section 2 (Runtime)

This section contains the runtime downloads. Notice that links for the **Installers** column in the **macOS** row are empty! This section is empty because the **ASP.NET Core Runtime**, is only provided in the SDK, or through binary installation.

Scroll further down to find the standard **.NET Runtime** for download.

:::image type="content" source="media/macos/runtime-links.png" alt-text="A screenshot showing just the .NET Runtime download table from the .NET download website. The macOS row is highlighted with a red box.":::

- If you're running an Apple processor, such as an **M1** or an **M3 Pro**, select **Arm64**.
- If you're running an Intel processor, select **x64**.

01. Once the download completes, open it.
01. Follow the steps in the Installer.

:::image type="content" source="media/macos/installer.png" alt-text="A screenshot showing just the .NET installer running on macOS.":::

[csharp_006] C# - Installation & Setup

Install .NET on macOS: Install .NET manually

Sumber: ht

Install .NET manually

As an alternative to the macOS installers, you can download and manually install the SDK and runtime. Manual installation is usually performed as part automation in a continuous integration scenario. Developers and users usually want to use the installer.
Use the install-dotnet.sh script to perform these steps automatically.

01. Open a browser and navigate to <https://dotnet.microsoft.com/download/dotnet>.
01. Select the link to the .NET version you want to install, such as **.NET 8.0**.

This link brings you to the page with links to download that version of .NET

If you're going to install the SDK, choose the latest .NET version. The SDK supports building apps for previous versions of .NET.
If you're unsure which version to download, choose the version marked **latest**.

:::image type="content" source="media/macos/download-page.png" alt-text="The .NET download website. Versions 6.0 through 9.0 are listed. A red box highlights those download links.":::

01. Select the link to the SDK or Runtime you want to install. Look for the **Binaries** column in the **macOS** row.

:::image type="content" source="media/macos/manual-download-page.png" alt-text="The .NET download website showing the SDK download links. The SDK header is highlighted with a red box. The box has an arrow pointing down to the macOS section.":::

- If you're running an Apple processor, such as an **M1** or an **M3 Pro**, select **Arm64**.
- If you're running an Intel processor, select **x64**.

01. Open a terminal and navigate to where the .NET binary was downloaded.
01. Extract the tarball to where you want to .NET on your system. The following example uses the **HOME** directory `~/Applications/.dotnet`.

Test that .NET is working by changing the directory to where .NET was installed, and run the `dotnet --info` command:

Contoh Kode / Perintah CLI:

```
mkdir -p ~/Applications/.dotnet
    tar -xf "dotnet-sdk-9.0.100-rc.2.24474.11-osx-arm64.tar" -C ~/Applications/.dotnet/

// -----------------------------------------

chdir ~/Applications/.dotnet/
./dotnet --info
```

[csharp_007] C# - Installation & Setup

Install .NET on macOS: Install .NET with a script

Sumber: ht

Install .NET with a script

The dotnet-install scripts are used for automation and unelevated installs of the runtime. You can download the script from <https://dot.net/v1/dotnet-install.sh>.

The script defaults to installing the latest long term support (LTS) version, which is .NET 8. You can choose a specific release by specifying the `channel` switch. Include the `runtime` switch to install a runtime. Otherwise, the script installs the SDK.
These commands are provided a script snippet at the end of this procedure.

01. Open a terminal.
01. Navigate to a folder where you want to download the script, such as _~/Downloads_.
01. If you don't have the `wget` command, install it with **Brew**

01. Run the following command to download the script:

01. Give the script execute permissions

01. Run the script to install .NET.

    The script defaults to installing the latest SDK to the `~/.dotnet` directory.

Here are all the commands as a single bash script:

Test .NET by navigating to the `~/.dotnet` folder and running the `dotnet --info` command:
Some programs might use environment variables to find .NET on your system, and using the `dotnet` command might not work when opening a new terminal. For help resolving this issue, see Make .NET available system-wide section.

Contoh Kode / Perintah CLI:

```
brew install wget

// ----------------------------------------

wget https://dot.net/v1/dotnet-install.sh

// ----------------------------------------

chmod +x dotnet-install.sh

// ----------------------------------------
```

```
./dotnet-install.sh

// ----------------------------------------

chdir ~/Downloads
brew install wget
wget https://dot.net/v1/dotnet-install.sh
chmod +x dotnet-install.sh
./dotnet-install.sh

// ----------------------------------------

chdir ~/.dotnet
./dotnet --info
```

[csharp_008] C# - Installation & Setup

Install .NET on macOS: Install .NET for Visual Studio Code

Sumber: ht

Install .NET for Visual Studio Code

Visual Studio Code is a powerful and lightweight source code editor that runs on your desktop. Visual Studio Code can use the SDK already installed on your system. Additionally, the C# Dev Kit extension will install .NET for you if it's not already installed.

For instructions on installing .NET through Visual Studio Code, see Getting Started with C# in VS Code.

[csharp_009] C# - Installation & Setup

Install .NET on macOS: Notarization

Sumber: ht

Notarization

Software created for macOS that's distributed with a Developer ID must be notarized, including apps made with .NET.

If you run a non-notarized app, an error window similar to the following image is displayed:

!macOS Catalina notarization alert

For more information about how enforced-notarization affects .NET (and your .NET apps), see Working with macOS Catalina Notarization.

[csharp_010] C# - Installation & Setup

Install .NET on macOS: Validation

Sumber: ht

Validation

!INCLUDE [verify-download-intro]

Use the `shasum -a 512` command to print the checksum of the file you've downloaded. For example, the following command reports the checksum of the _dotnet-sdk-9.0.306-osx-x64.tar.gz_ file:

Compare the checksum with the value provided by the download site.

Use a checksum file to validate

The .NET release notes contain a link to a checksum file you can use to validate your downloaded file. The following steps describe how to download the checksum file and validate a .NET install binary:

01. The release notes page for .NET 9 on GitHub at <https://github.com/dotnet/core/tree/main/release-notes/9.0#releases> contains a section named **Releases**. The table in that section links to the downloads and checksum files for each .NET 9 release. The following image shows the .NET 8 release table as a reference:

:::image type="content" source="media/install-sdk/release-notes-root.png" alt-text="The github release notes version table for .NET":::

01. Select the link for the version of .NET that you downloaded.

The previous section used .NET SDK 9.0.306, which is in the .NET 9.0.10 release.

01. In the release page, you can see the .NET Runtime and .NET SDK version, and a link to the checksum file. The following image shows the .NET 8 release table as a reference:

:::image type="content" source="media/install-sdk/release-notes-version.png" alt-text="The download table with checksums for .NET":::

01. Right-click on the **Checksum** link and copy it to your clipboard.

01. Open a terminal.

01. Use `curl -O {link}` to download the checksum file.

Replace the link in the following command with the link you copied.

01. With both the checksum file and the .NET release file downloaded to the same directory, use the `shasum -a 512 -c {file}` command to validate the downloaded file.

When validation passes, you see the file printed with the **OK** status:

If you see the file marked as **FAILED**, the file you downloaded isn't valid and shouldn't be used.

Contoh Kode / Perintah CLI:

```
$ shasum -a 512 dotnet-sdk-9.0.306-osx-x64.tar.gz
a9700f98e5aa4f70b2a08ddba2b1c6085106b0d17828bd719fdcef460b06c890b32d752fbff8e4659cd1ca4174b4b211b301fe682
439ea9a24b6521ca5a64c69  dotnet-sdk-9.0.306-osx-x64.tar.gz

// ----------------------------------------

curl -O https://builds.dotnet.microsoft.com/dotnet/checksums/9.0.10-sha.txt

// ----------------------------------------
```

```
$ shasum -a 512 -c 9.0.10-sha.txt
    dotnet-sdk-9.0.306-osx-x64.tar.gz: OK

// ----------------------------------------

$ shasum -a 512 -c 9.0.10-sha.txt
    dotnet-sdk-9.0.306-osx-x64.tar.gz: FAILED
    shasum: WARNING: 1 computed checksum did NOT match
```

[csharp_011] C# - Installation & Setup

Install .NET on macOS: Arm-based Macs

Sumber: ht

Arm-based Macs

The following sections describe things you should consider when installing .NET on an Arm-based Mac.

<!-- This section is mirrored in the windows.md file. Changes here should be applied there -->

Path differences

On an Arm-based Mac, all Arm64 versions of .NET are installed to the normal _/usr/local/share/dotnet/_ folder. However, when you install the **x64** version of .NET SDK, it's installed to the _/usr/local/share/dotnet/x64/dotnet/_ folder.

Path variables

Environment variables that add .NET to system path, such as the `PATH` variable, might need to be changed if you have both the x64 and Arm64 versions of the .NET SDK installed. Additionally, some tools rely on the `DOTNET_ROOT` environment variable, which would also need to be updated to point to the appropriate .NET SDK installation folder.

[csharp_012] C# - Installation & Setup

Install .NET on macOS: Troubleshooting

Sumber: ht

Troubleshooting

The following sections are available to help troubleshoot issues:

- Arm-based Macs
- Make .NET available system-wide

Make .NET available system-wide

Sometimes apps on your system, including the terminal, need to find where .NET is installed. The .NET macOS Installer package should automatically configure your system. However, if you used the manual installation method or the .NET install script, you must add the directory where .NET was installed to the `PATH` variable.

Some apps might look for the `DOTNET_ROOT` variable when trying to determine where .NET is installed.

There are many different shells available for macOS and each has a different profile. For example:

- **Bash Shell**: _~/.profile_, _/etc/profile_
- **Korn Shell**: _~/.kshrc_ or _.profile_
- **Z Shell**: _~/.zshrc_ or _.zprofile_

Set the following two environment variables in your shell profile:

- `DOTNET_ROOT`

  This variable is set to the folder .NET was installed to, such as `$HOME/.dotnet`:

- `PATH`

  This variable should include both the `DOTNET_ROOT` folder and the `DOTNET_ROOT/tools` folder:

Contoh Kode / Perintah CLI:

```
export DOTNET_ROOT=$HOME/.dotnet

// ----------------------------------------

export PATH=$PATH:$DOTNET_ROOT:$DOTNET_ROOT/tools
```

[csharp_013] C# - Installation & Setup

Install .NET on macOS: Related content

Sumber: ht

Related content

- How to check if .NET is already installed.
- Working with macOS Catalina notarization.
- Tutorial: Create a .NET console application.
- Tutorial: Containerize a .NET app.

[csharp_014] C# - Installation & Setup

"Install .NET on Windows": Overview

Sumber: ht

Install .NET on Windows
[!div class="op_single_selector"]
- Install on Windows
- Install on macOS
- Install on Linux

This article teaches you about which versions of .NET are supported on Windows, how to install .NET, and what the difference is between the SDK and runtime.

Unlike .NET Framework, .NET isn't tied to your version of Windows. You can only have a single version of .NET Framework installed on Windows. But .NET is standalone and can be installed anywhere on your computer. Some apps might include their own copy of .NET.

By default, .NET is installed to the _Program Files\\dotnet_ directory on your computer, unless the installation method chooses a different directory.
If you're installing .NET system-wide, install with administrative privileges.

.NET is made up of the runtime and the SDK. The runtime runs .NET apps, and the SDK is used to create apps.

[csharp_015] C# - Installation & Setup

"Install .NET on Windows": Choose the correct runtime

Sumber: ht

Choose the correct runtime

There are three different runtimes for Windows, which enable different types of apps to run. The SDK includes all three runtimes, and an installer for a runtime might include an additional runtime. The following table describes which runtime is included with a particular .NET installer:

Installer	Includes .NET Runtime	Includes .NET Desktop Runtime	Includes ASP.NET Core Runtime
.NET Runtime	Yes	No	No
.NET Desktop Runtime	Yes	Yes	No
ASP.NET Core Runtime	No	No	Yes
.NET SDK	Yes	Yes	Yes

To ensure that you can run all .NET apps on Windows, install both the ASP.NET Core Runtime and the .NET Desktop Runtime. The ASP.NET Core Runtime runs web-based apps, and the .NET Desktop Runtime runs desktop apps, such as a Windows Presentation Foundation (WPF) or Windows Forms app.

[csharp_016] C# - Installation & Setup

"Install .NET on Windows": Choose how to install .NET

Sumber: ht

Choose how to install .NET

There are different ways to install .NET, and some products might manage their own version of .NET. If you install .NET through software that manages its own version of .NET, it might not be enabled system-wide. Make sure you understand the implications of installing .NET through other software.

If you're unsure which method you should choose after reviewing the lists in the following sections, you probably want to use the .NET Installer.

Developers

- Visual Studio

  Use **Visual Studio** to install .NET when you want to develop .NET apps using Visual Studio. Visual Studio manages its own copy of .NET. This method installs the SDK, Runtime, and Visual Studio templates.

- Visual Studio Code - C# Dev Kit

  Install the **C# Dev Kit** extension for Visual Studio Code to develop .NET apps. The extension can use an SDK that's already installed or install one for you.

Users and developers

- .NET Installer

Install .NET with a Windows Installer package, which is an executable that you run. This method can install the SDK and Runtime. Installs are performed system-wide.

- Windows Package Manager (WinGet)

Use **WinGet** to install .NET when you want to manage .NET through the command line. This method can install the SDK and Runtime. Installs are performed system-wide.

- PowerShell

A PowerShell script that can automate the install of the SDK or Runtime. You can choose which version of .NET to install.

[csharp_017] C# - Installation & Setup

"Install .NET on Windows": Supported versions

Sumber: ht

Supported versions

The following table is a list of currently supported .NET releases and the versions of Windows they're supported on. These versions remain supported until either the version of .NET reaches end-of-support or the version of Windows reaches end-of-life.
As a reminder, this table applies to modern .NET (as opposed to .NET Framework). To install .NET Framework, see the .NET Framework installation guide.

Windows 10 support is limited to Long-Term Servicing Channel (LTSC) and Enterprise editions only. Check the Windows lifecycle fact sheet for specific details.

Operating System	.NET 10 (Architectures)	.NET 9 (Architectures)	.NET 8 (Architectures)
Windows 11 (26H1, 25H2, 24H2, 23H2 Ent/Edu)	[OK]? x64, x86, Arm64	[OK]? x64, x86, Arm64	[OK]? x64, x86, Arm64
Windows 10 (21H2, 1809, 1607 LTSC/Enterprise)	[OK]? x64, x86, Arm64	[OK]? x64, x86, Arm64	[OK]? x64, x86, Arm64
Windows Server (2025, 23H2, 2022, 2019, 2016, 2012 R2, 2012)	[OK]? x64	[OK]? x64	[OK]? x64
Windows Server Core (2025, 2022, 2019, 2016, 2012 R2, 2012)	[OK]? x64	[OK]? x64, x86	[OK]? x64
Nano Server (2025, 2022, 2019)	[OK]? x64	[OK]? x64	[OK]? x64
Windows 8.1	[X]	[X]	[X]
Windows 7 SP1 [ESU][esu]	[X]	[X]	[X]

Windows 7 / 8.1 / Server 2012

There is no longer a version of .NET that's supported on **Windows 7 and Windows 8.1**. The last supported releases was .NET 6 and support ended on November 12, 2024.

Windows Server 2012 is still supported by any version of .NET that's still in support.

All three of these versions of Windows require further dependencies to be installed:

| Operating System | Prerequisites |

|--------------------------|--|
Windows 7 SP1 [ESU][esu]	- Microsoft Visual C++ 2015-2019 Redistributable [64-bit][vcc64] / [32-bit][vcc32] - KB3063858 [64-bit][kb64] / [32-bit][kb32] - Microsoft Root Certificate Authority 2011 (.NET Core 2.1 offline installer only)
Windows 8.1	Microsoft Visual C++ 2015-2019 Redistributable [64-bit][vcc64] / [32-bit][vcc32]
Windows Server 2012	Microsoft Visual C++ 2015-2019 Redistributable [64-bit][vcc64] / [32-bit][vcc32]
Windows Server 2012 R2	Microsoft Visual C++ 2015-2019 Redistributable [64-bit][vcc64] / [32-bit][vcc32]

The previous requirements are also required if you receive an error related to either of the following dlls:

- *api-ms-win-crt-runtime-l1-1-0.dll*
- *api-ms-win-cor-timezone-l1-1-0.dll*
- *hostfxr.dll*

[csharp_018] C# - Installation & Setup

"Install .NET on Windows": Arm-based Windows PCs

Sumber: ht

Arm-based Windows PCs

.NET is supported on Arm-based Windows PCs. The following sections describe things you should consider when installing .NET.

Path differences

On an Arm-based Windows PC, all Arm64 versions of .NET are installed to the normal _C:\\Program Files\\dotnet_ folder. However, the **x64** version of the .NET SDK is installed to the _C:\\Program Files\\dotnet\\x64_ folder.

Path variables

Environment variables that add .NET to system path, such as the `PATH` variable, might need to be changed if you have both the x64 and Arm64 versions of the .NET SDK installed. Additionally, some tools rely on the `DOTNET_ROOT` environment variable, which would also need to be updated to point to the appropriate .NET SDK installation folder.

[csharp_019] C# - Installation & Setup

"Install .NET on Windows": Install with Visual Studio

Sumber: ht

Install with Visual Studio

Visual Studio installs its own copy of .NET separate from other copies of .NET. Different versions of Visual Studio support different versions of .NET. The latest version of Visual Studio always supports the latest version of .NET.
[!div class="button"]
Download Visual Studio Installer

Visual Studio Installer installs and configures Visual Studio. Some Visual Studio workloads include .NET, such as **ASP.NET and web development** and **.NET Multi-platform App UI development**. Specific versions of .NET can be installed through the **Individual Components** tab.

The Visual Studio documentation provides instructions on how to:

- Install Visual Studio.

- Configure Visual Studio workloads.

:::image type="content" source="media/windows/vs-workloads.png" alt-text="A screenshot that shows Visual Studio Installer with the .NET Desktop workload highlighted with a red box.":::

.NET versions and Visual Studio

If you're using Visual Studio to develop .NET apps, the following table describes the minimum required version of Visual Studio based on the target .NET SDK version.

.NET SDK version	Visual Studio version
10	Visual Studio 2026 version 18.0 or higher.
9	Visual Studio 2022 version 17.12 or higher.
8	Visual Studio 2022 version 17.8 or higher.
7	Visual Studio 2022 version 17.4 or higher.
6	Visual Studio 2022 version 17.0 or higher.
5	Visual Studio 2019 version 16.8 or higher.
3.1	Visual Studio 2019 version 16.4 or higher.
3.0	Visual Studio 2019 version 16.3 or higher.
2.2	Visual Studio 2017 version 15.9 or higher.
2.1	Visual Studio 2017 version 15.7 or higher.

If you already have Visual Studio installed, you can check your version with the following steps.

01. Open Visual Studio.
01. Select **Help** > **About Microsoft Visual Studio**.
01. Read the version number from the **About** dialog.

For more information about, see .NET SDK, MSBuild, and Visual Studio versioning.

[csharp_020] C# - Installation & Setup

"Install .NET on Windows": Install with Visual Studio Code

Sumber: ht

Install with Visual Studio Code

Visual Studio Code is a powerful and lightweight source code editor that runs on your desktop. Visual Studio Code can use the SDK already installed on your system.
If you're installing .NET system-wide, install with administrative privileges.

This WinGet configuration file installs the latest .NET SDK, Visual Studio Code and the C# DevKit. If you already have any of them installed, WinGet will skip that step.

01. Download the file and double-click to run it.
01. Read the license agreement, type <kbd>y</kbd>, and select <kbd>Enter</kbd> when prompted to accept.
01. If you get a flashing User Account Control (UAC) prompt in your Taskbar, allow the installation to continue.

Additionally, the C# Dev Kit extension will install .NET for you if it's not already installed.

For instructions on installing .NET through Visual Studio Code, see Getting Started with C# in VS Code.

[csharp_021] C# - Installation & Setup

"Install .NET on Windows": .NET installer

Sumber: ht

.NET installer

The download page for .NET provides Windows Installer executables.
If you're installing .NET system-wide, install with administrative privileges.

1. Open a web browser and navigate to <https://dotnet.microsoft.com/download/dotnet>.
1. Select the version of .NET you want to download, such as 10.0.
1. Find the SDK or Runtime box that contains the links for downloading .NET.
1. Under the **Installers** column, find the **Windows** row and select the link for your CPU architecture. If you're unsure, select **x64** as it's the most common.

The browser should automatically download the installer.
The following image shows the SDK, but you can also download the Runtime.

:::image type="content" source="media/windows/dotnet-download-page-small.png" alt-text="An image of the .NET download page, with the SDK download link highlighted." lightbox="media/windows/dotnet-download-page.png":::

1. Open Windows Explorer and navigate to where the file was downloaded, most likely your **Downloads** folder.
1. Double-click on the file to install .NET.

The Windows Installer dialog is opened.

:::image type="content" source="media/windows/msi-installer.png" alt-text="A screenshot of the .NET installer app window.":::

1. Select **Install** and follow the instructions to install .NET.

To learn how to use the .NET CLI, see .NET CLI overview.

Command-line options

Use the `/?` parameter to display a list of options.

If you want to install .NET silently, such as in a production environment or to support continuous integration, use the following options:

- `/install`\
Installs .NET.

- `/quiet`\
Prevents any UI and prompts from displaying.

- `/norestart`\
Suppresses any attempts to restart.

If you've already installed .NET, use the .NET Installer to manage the installation. Instead of `/install`, use one of the

following options:

- `/uninstall`\
Remove this version of .NET.

- `/repair`\
Check if the installations key files or components are damaged and restore them.
The installer returns an exit code of **0** for success and an exit code of **3010** to indicate that a restart is required. Any other value is most likely an error code.

Microsoft Update

!INCLUDE [microsoft-update]

Choose when previous versions are removed

The installer executables always install new content before removing the previous installation. Applications that are running might be interrupted or crash when older runtimes are removed. To minimize the impact of updating .NET, you can specify when a previous .NET installation should be removed using a registry key.

.NET version	Registry key	Name	Type	Value
All	HKLM\SOFTWARE\Microsoft\.NET	RemovePreviousVersion	REG_SZ	`always`, `never`, or `nextSession`
.NET 10	HKLM\SOFTWARE\Microsoft\.NET\10.0	RemovePreviousVersion	REG_SZ	`always`, `never`, or `nextSession`
.NET 9	HKLM\SOFTWARE\Microsoft\.NET\9.0	RemovePreviousVersion	REG_SZ	`always`, `never`, or `nextSession`
.NET 8	HKLM\SOFTWARE\Microsoft\.NET\8.0	RemovePreviousVersion	REG_SZ	`always`, `never`, or `nextSession`

- `never` retains previous installations and requires manual intervention to remove previous .NET installations.
- `always` removes previous installations after the new version is installed. This is the default behavior in .NET.
- `nextSession` defers the removal until the next logon session from members in the Administrators group.
- Values are case-insensitive and invalid values default to `always`.

When the removal is deferred, the installer writes a command to the RunOnce registry key to uninstall the previous version. The command only executes if a user in the Administrators group logs on to the machine.
This feature is only available starting with .NET 8 (8.0.11). It only applies to the standalone installer executables and impacts distributions like WinGet that use them.

[csharp_022] C# - Installation & Setup

"Install .NET on Windows": Install with Windows Package Manager (WinGet)

Sumber: ht

Install with Windows Package Manager (WinGet)

You can install and manage .NET through the Windows Package Manager service, using the `winget.exe` tool. For more information about how to install and use **WinGet**, see Use the winget tool to install and manage applications.
If you're installing .NET system-wide, install with administrative privileges.

The .NET WinGet packages are:

- `Microsoft.DotNet.Runtime.10`—.NET Runtime 10.0
- `Microsoft.DotNet.AspNetCore.10`—ASP.NET Core Runtime 10.0
- `Microsoft.DotNet.DesktopRuntime.10`—.NET Desktop Runtime 10.0
- `Microsoft.DotNet.SDK.10`—.NET SDK 10.0
- `Microsoft.DotNet.Runtime.9`—.NET Runtime 9.0
- `Microsoft.DotNet.AspNetCore.9`—ASP.NET Core Runtime 9.0
- `Microsoft.DotNet.DesktopRuntime.9`—.NET Desktop Runtime 9.0
- `Microsoft.DotNet.SDK.9`—.NET SDK 9.0
- `Microsoft.DotNet.Runtime.8`—.NET Runtime 8.0
- `Microsoft.DotNet.AspNetCore.8`—ASP.NET Core Runtime 8.0
- `Microsoft.DotNet.DesktopRuntime.8`—.NET Desktop Runtime 8.0
- `Microsoft.DotNet.SDK.8`—.NET SDK 8.0

Install the SDK

If you install the SDK, you don't need to install the corresponding runtime.

01. Install WinGet.
01. Open a terminal, such as PowerShell or Command Prompt.
01. Run the `winget install` command and pass the name of the SDK package:

To learn how to use the .NET CLI, see .NET CLI overview.

Install the runtime

There are different runtimes you can install. Refer to the Choose the correct runtime section to understand what's included with each runtime.

01. Install WinGet.
01. Open a terminal, such as **PowerShell** or **Command Prompt**.
01. Run the `winget install` command and pass the name of the SDK package:

To learn how to use the .NET CLI, see .NET CLI overview.

Search for versions

Use the `winget search` command to search for different versions of the package you want to install. For example, the following command searches for all .NET SDKs available via WinGet:

The search results are printed in a table with each package identifier.

Install preview versions

If a preview version is available, substitute the version number in the **Id** with the word `Preview`. The following

example installs the preview release of the .NET Desktop Runtime:

Contoh Kode / Perintah CLI:

```
To learn how to use the .NET CLI, see [.NET CLI overview](../tools/index.md).

### Install the runtime

There are different runtimes you can install. Refer to the [Choose the correct runtime](#choose-the-correct-runtime) section to understand what's included with each runtime.

01. [Install WinGet](/windows/package-manager/winget/#install-winget).
01. Open a terminal, such as **PowerShell** or **Command Prompt**.
01. Run the `winget install` command and pass the name of the SDK package:

// ----------------------------------------

To learn how to use the .NET CLI, see [.NET CLI overview](../tools/index.md).

### Search for versions

Use the `winget search` command to search for different versions of the package you want to install. For example, the following command searches for all .NET SDKs available via WinGet:

// ----------------------------------------

The search results are printed in a table with each package identifier.

// ----------------------------------------

### Install preview versions

If a preview version is available, substitute the version number in the **Id** with the word `Preview`. The following example installs the preview release of the .NET Desktop Runtime:
```

[csharp_023] C# - Installation & Setup

"Install .NET on Windows": Install with PowerShell

Sumber: ht

Install with PowerShell

Installing .NET through the `dotnet-install` PowerShell script is recommended for continuous integration and nonadmin installs. If you're installing .NET for normal use on your system, use either the .NET Installer or Windows Package Manager installation methods.

The script defaults to installing the latest long term support (LTS) version, which is .NET 10. You can choose a specific release by specifying the `-Channel` switch. Include the `-Runtime` switch to install a runtime. Otherwise, the script installs the SDK. The script is available at <https://dot.net/v1/dotnet-install.ps1> and the source code is hosted on GitHub.
[!div class="button"]
Download the script

For more information about the script, see dotnet-install script reference.

Install the runtime

The .NET Runtime is installed by providing the `-Runtime` switch.

01. Download the install script from <https://dot.net/v1/dotnet-install.ps1>

01. Open PowerShell and navigate to the folder containing the script.
01. Run the following commands to install both the Desktop runtime and ASP.NET Core runtime for maximum compatibility:

To learn how to use the .NET CLI, see .NET CLI overview.

Install the SDK

If you install the SDK, you don't need to install the runtimes.

01. Download the install script from <https://dot.net/v1/dotnet-install.ps1>
01. Open PowerShell and navigate to the folder containing the script.
01. Run the following command to install the .NET SDK.
The SDK is installed by omitting the `-Runtime` switch.

To learn how to use the .NET CLI, see .NET CLI overview.

Contoh Kode / Perintah CLI:

```
To learn how to use the .NET CLI, see [.NET CLI overview](../tools/index.md).

### Install the SDK

If you install the SDK, you don't need to install the runtimes.

01. Download the install script from <https://dot.net/v1/dotnet-install.ps1>
01. Open PowerShell and navigate to the folder containing the script.
01. Run the following command to install the .NET SDK.
```

[csharp_024] C# - Installation & Setup

"Install .NET on Windows": Manual install

Sumber: ht

Manual install

As an alternative to the installers and package managers, you can download and manually install the SDK and runtime. Manual installation is commonly used as part of continuous integration testing or on systems where you don't have administrative privileges. For most developers and users, it's better to use one of the installer methods.

Download a **binary** release for either the SDK or the runtime from one of the following sites. The .NET SDK includes the corresponding runtime:

- [OK]? .NET 10 downloads
- [OK]? .NET 9 downloads
- [OK]? .NET 8 downloads
- All .NET downloads

After you download the binary release, follow these steps to extract it and configure the environment so the .NET CLI commands are available in the terminal:

01. Choose the **Binaries** download (a _.zip_ file) for your CPU architecture.
01. Extract the downloaded file.
01. Set the `DOTNET_ROOT` environment variable to the extracted folder's location.

01. Ensure `DOTNET_ROOT` is in `PATH`.

For more information about .NET environment variables, see .NET SDK and CLI environment variables.

Different versions of .NET can be extracted to the same folder, where they coexist side-by-side.

Example

The following commands use PowerShell to extract the .NET binary release to a `dotnet-install` folder in the current working directory and configure the environment variables for the current session. The `$DOTNET_FILE` variable is the file name of the .NET binary release you downloaded.
If you run these commands, remember to change the `$DOTNET_FILE` value to the name of the .NET binary you downloaded.

You can install more than one version of .NET in the same folder.

To make these settings persist across terminal sessions, set them as user or system environment variables. For more information, see Set environment variables system-wide.

To learn how to use the .NET CLI, see .NET CLI overview.

Set environment variables system-wide

The variables set in the previous example only apply to the current PowerShell session. To make them available in every terminal session, store them as user or system environment variables.

To set the variables for your user account, run the following commands in PowerShell. Replace the path with the folder where you extracted .NET:

To set the variables system-wide for all users, replace `'User'` with `'Machine'` in the previous commands and run PowerShell as an administrator.

Close and reopen your terminal for the changes to take effect.

Contoh Kode / Perintah CLI:

```
You can install more than one version of .NET in the same folder.

To make these settings persist across terminal sessions, set them as user or system environment variables. For more information, see [Set environment variables system-wide](#set-environment-variables-system-wide).

To learn how to use the .NET CLI, see [.NET CLI overview](../tools/index.md).

### Set environment variables system-wide

The variables set in the previous example only apply to the current PowerShell session. To make them available in every terminal session, store them as user or system environment variables.

To set the variables for your user account, run the following commands in PowerShell. Replace the path with the folder where you extracted .NET:
```

[csharp_025] C# - Installation & Setup

"Install .NET on Windows": Validation

Sumber: ht

Validation

!INCLUDE [verify-download-intro]

You can use **PowerShell** or **Command Prompt** to validate the checksum of the file you've downloaded. For example, the following command reports the checksum of the _dotnet-sdk-9.0.306-win-x64.exe_ file:

Compare the checksum with the value provided by the download site.

Use PowerShell and a checksum file to validate

The .NET release notes contain a link to a checksum file you can use to validate your downloaded file. The following steps describe how to download the checksum file and validate a .NET install binary:

01. The release notes page for .NET 9 on GitHub at <https://github.com/dotnet/core/tree/main/release-notes/9.0> contains a section named **Releases**. The table in that section links to the downloads and checksum files for each .NET 9 release. The following image shows the .NET 8 release table as a reference:

:::image type="content" source="media/install-sdk/release-notes-root.png" alt-text="The github release notes version table for .NET":::

01. Select the link for the version of .NET that you downloaded. The previous section used .NET SDK 9.0.306, which is in the .NET 9.0.10 release.
If you're not sure which .NET release contains your checksum file, explore the links until you find it.

01. In the release page, you can see the .NET Runtime and .NET SDK version, and a link to the checksum file. The following image shows the .NET 8 release table as a reference:

:::image type="content" source="media/install-sdk/release-notes-version.png" alt-text="The download table with checksums for .NET":::

01. Copy the link to the checksum file.

01. Use the following script, but replace the link to download the appropriate checksum file:

01. With both the checksum file and the .NET release file downloaded to the same directory, search the checksum file for the checksum of the .NET download:

When validation passes, you see **True** printed:

If you see **False** printed, the file you downloaded isn't valid and shouldn't be used.

Contoh Kode / Perintah CLI:

```
Compare the checksum with the value provided by the download site.

### Use PowerShell and a checksum file to validate

The .NET release notes contain a link to a checksum file you can use to validate your downloaded file. The following steps describe how to download the checksum file and validate a .NET install binary:

01. The release notes page for .NET 9 on GitHub at <https://github.com/dotnet/core/tree/main/release-notes/9.0> contains a section named **Releases**. The table in that section links to the downloads and checksum files for each .NET 9 release. The following image shows the .NET 8 release table as a reference:

       :::image type="content" source="media/install-sdk/release-notes-root.png" alt-text="The github release notes version table for .NET":::

01. Select the link for the version of .NET that you downloaded. The previous section used .NET SDK 9.0.306, which is in the .NET 9.0.10 release.

    > [!TIP]
    > If you're not sure which .NET release contains your checksum file, explore the links until you find it.

01. In the release page, you can see the .NET Runtime and .NET SDK version, and a link to the checksum file. The following image shows the .NET 8 release table as a reference:

     :::image type="content" source="media/install-sdk/release-notes-version.png" alt-text="The download table with checksums for .NET":::

01. Copy the link to the checksum file.

01. Use the following script, but replace the link to download the appropriate checksum file:

// ----------------------------------------

01. With both the checksum file and the .NET release file downloaded to the same directory, search the checksum file for the checksum of the .NET download:

    When validation passes, you see **True** printed:
```

[csharp_026] C# - Installation & Setup

"Install .NET on Windows": Troubleshooting

Sumber: ht

Troubleshooting

After installing the .NET SDK, you might run into problems trying to run .NET CLI commands. This section collects those common problems and provides solutions.

- No .NET SDK was found
- Building apps is slower than expected
- `hostfxr.dll` / `api-ms-win-crt-runtime-l1-1-0.dll` / `api-ms-win-cor-timezone-l1-1-0.dll` is missing

No .NET SDK was found

Most likely you installed both the x86 (32-bit) and x64 (64-bit) versions of the .NET SDK. This is causing a conflict because when you run the `dotnet` command, it's resolving to the x86 version when it should resolve to the x64 version.

This problem is fixed by adjusting the `%PATH%` variable to resolve the x64 version first.

01. Verify that you have both versions installed by running the `where.exe dotnet` command. If you do, you should see an entry for both the _Program Files_ and _Program Files (x86)_ folders. If the _Program Files (x86)_ folder is first, as demonstrated by the following example, it's incorrect and you should continue on to the next step.
Even though _Program Files_ is used in this example, you may see other copies of _dotnet.exe_ listed. Adjust them so that the appropriate _dotnet.exe_ is resolved first.

If it's correct and _Program Files_ is first, you don't have the problem this section is discussing and you should create a .NET help request issue on GitHub.

01. Press the Windows button and type "Edit the system environment variables" into search. Select **Edit the system environment variables**.

:::image type="content" source="media/windows/start-menu.png" alt-text="Windows start menu with edit environment variable":::

01. The **System Properties** window opens up to the **Advanced Tab**. Select **Environment Variables**.

:::image type="content" source="media/windows/system-props.png" alt-text="The Windows system properties panel open.":::

01. On the **Environment Variables** window, under the **System variables** group, select the *Path** row and then select the **Edit** button.

:::image type="content" source="media/windows/list-vars.png" alt-text="The environment variables window with user and system variables.":::

01. Use the **Move Up** and **Move Down** buttons to move the **C:\\Program Files\\dotnet** entry above **C:\\Program Files (x86)\\dotnet**.

:::image type="content" source="media/windows/edit-vars.png" alt-text="The environment variables list for the system.":::

Building apps is slower than expected

Ensure that Smart App Control, a Windows feature, is off. Smart App Control isn't recommended to be enabled on machines used for development. Any setting other than "off" might negatively affect SDK performance.

`hostfxr.dll` / `api-ms-win-crt-runtime-l1-1-0.dll` / `api-ms-win-cor-timezone-l1-1-0.dll` is missing

Install the Microsoft Visual C++ 2015-2019 Redistributable ([64-bit][vcc64] or [32-bit][vcc32]).

[csharp_027] C# - Installation & Setup

"Install .NET on Windows": Related content

Sumber: ht

Related content

- .NET CLI overview
- Upgrade to a new .NET version.
- How to check if .NET is already installed.
- Tutorial: Create a new console application.

[esu]: /troubleshoot/windows-client/windows-7-eos-faq/windows-7-extended-security-updates-faq
[vcc64]: https://aka.ms/vs/16/release/vc_redist.x64.exe
[vcc32]: https://aka.ms/vs/16/release/vc_redist.x86.exe
[kb64]: https://www.microsoft.com/download/details.aspx?id=47442
[kb32]: https://www.microsoft.com/download/details.aspx?id=47409

[csharp_028] C# - Installation & Setup

Install .NET on Linux distributions: Overview

Sumber: ht

Install .NET on Linux
[!div class="op_single_selector"]
- Install on Windows
- Install on macOS
- Install on Linux

This article describes how .NET is available on various Linux distributions. .NET can be installed by a package manager, snap, or manually. .NET is also available as a container image.

[csharp_029] C# - Installation & Setup

Install .NET on Linux distributions: Packages

Sumber: ht

Packages

Packages are published in the Microsoft package repository at <https://packages.microsoft.com/> and are available for the following Linux distributions:

- Azure Linux
- Debian
- openSUSE Leap
- SUSE Enterprise Linux

Distributions are selected per the policy defined at dotnet/core \#9556.

The following Linux distributions publish their own .NET packages:

- Alpine
- CentOS Stream
- Fedora
- Red Hat Enterprise Linux (RHEL)
- Ubuntu

[csharp_030] C# - Installation & Setup

Install .NET on Linux distributions: Snap

Sumber: ht

Snap

.NET SDK snap packages are provided by and maintained by Canonical. Snaps are a great alternative to the package manager built into your Linux distribution.

- Install .NET Runtime with Snap
- Install .NET SDK with Snap

[csharp_031] C# - Installation & Setup

Install .NET on Linux distributions: Manual installation

Sumber: ht

Manual installation

You can install .NET manually in the following ways:

- Manual install
- Scripted install

You might need to install .NET dependencies if you install .NET manually.

[csharp_032] C# - Installation & Setup

Install .NET on Linux distributions: Additional sources

Sumber: ht

Additional sources

.NET is also available from other sources. The packages and containers use a name similar to one of the following names:

- aspnet-runtime
- dotnet-runtime
- dotnet-sdk
- dotnet

Package managers

- <https://formulae.brew.sh/cask/dotnet-sdk>
- <https://ports.macports.org/port/dotnet-cli>
- <https://search.nixos.org/packages?query=dotnet>
- <https://archlinux.org>
- <https://aur.archlinux.org>

Containers

- <https://containers.dev/features>
- <https://images.chainguard.dev>

[csharp_033] C# - Configuration - Language Version

Configure language version: Overview

Sumber: ht

Configure C# language version

The information in this article applies to .NET 5 and later. For UWP projects, see the article on Choosing a UWP version.

!INCLUDE[csharp-version-note]

In Visual Studio, the option to change the language version through the UI is disabled because the default version aligns with the project's target framework (`TFM`). This default configuration ensures compatibility between language features and runtime support. To change the language version in Visual Studio, change the project's target framework.

For example, changing the target `TFM` (for example, from .NET 6 to .NET 9) updates the language version accordingly, from C# 10 to C# 13. This approach prevents issues with runtime compatibility and minimizes unexpected build errors due to unsupported language features.

If you need a specific language version that differs from the one automatically selected, use the methods in this article to override the default settings directly in the project file.
Don't set the `LangVersion` element to `latest`. The `latest` setting means the installed compiler uses its latest version. The value of `latest` can change from machine to machine, making builds unreliable. In addition, it enables language features that might require runtime or library features not included in the current SDK.

If you must specify your C# version explicitly, you can do so in several ways:

- Manually edit the project file.
- Set the language version for multiple projects in a subdirectory.
- Configure the LangVersion compiler option.

You can see the language version in Visual Studio in the project properties page. Under the *Build* tab, the *Advanced* pane displays the version selected.
To know what language version you're currently using, put `#error version` (case sensitive) in your code. This pragma makes the compiler report a compiler error, CS8304, with a message containing the compiler version and the current selected language version. For more information about this pragma, see #error (C# Reference).

[csharp_034] C# - Configuration - Language Version

Configure language version: Edit the project file

Sumber: ht

Edit the project file

Set the language version in your project file. The project file is the `*.csproj` file in the root folder for your project. For example, if you want explicit access to preview features, add an element like the following example:

The value `preview` uses the latest available preview C# language version your compiler supports.

Contoh Kode / Perintah CLI:

```
<PropertyGroup>
   <LangVersion>preview</LangVersion>
</PropertyGroup>
```

[csharp_035] C# - Configuration - Language Version

Configure language version: Configure multiple projects

Sumber: ht

Configure multiple projects

To configure multiple C# projects, create a *Directory.Build.props* file, typically in your solution directory, that contains the `<LangVersion>` element. Add the following setting to the *Directory.Build.props* file:

Builds in all subdirectories of the directory containing that file now use the preview C# version. For more information, see Customize your build.
The versions for C# and Visual Basic are different. Don't use the *Directory.Build.Props* file for a folder where subdirectories contain projects for both languages. The versions don't match.

Contoh Kode / Perintah CLI:

```
<Project>
 <PropertyGroup>
   <LangVersion>preview</LangVersion>
 </PropertyGroup>
</Project>
```

[csharp_036] C# - Configuration - Language Version

Configure language version: C# language version reference

Sumber: ht

C# language version reference
Using a C# language version newer than the version associated with your target TFM isn't supported.

The following table shows all current C# language versions. Older compilers might not understand every value. If you install the latest .NET SDK, you have access to everything listed.

!INCLUDE [langversion-table]

[csharp_037] C# - Configuration - Compiler Options

"Compiler Options - language feature rules": Overview

Sumber: ht

C# compiler options for language feature rules

The following options control how the compiler interprets language features. The new MSBuild syntax is shown in **Bold**. The older *csc.exe* syntax is shown in `code style`.

- **CheckForOverflowUnderflow** / `-checked`: Generate overflow checks.
- **AllowUnsafeBlocks** / `-unsafe`: Allow `unsafe` code.
- **DefineConstants** / `-define`: Define conditional compilation symbols.
- **LangVersion** / `-langversion`: Specify language version such as `default` (latest major version), or `latest` (latest version, including minor versions).
- **Nullable** / `-nullable`: Enable nullable context, or nullable warnings.

For more information about configuring these options for your project, see Compiler options.

[csharp_038] C# - Configuration - Compiler Options

"Compiler Options - language feature rules": CheckForOverflowUnderflow

Sumber: ht

CheckForOverflowUnderflow

The **CheckForOverflowUnderflow** option controls the default overflow-checking context that defines the program behavior if integer arithmetic overflows.

When **CheckForOverflowUnderflow** is `true`, the default context is a checked context and overflow checking is enabled. When **CheckForOverflowUnderflow** is `false`, the default context is an unchecked context. The default value for this option is `false`, which means overflow checking is disabled.

You can also explicitly control the overflow-checking context for parts of your code by using the `checked` and `unchecked` statements.

For information about how the overflow-checking context affects operations and what operations it affects, see the article about `checked` and `unchecked` statements.

Contoh Kode / Perintah CLI:

```
<CheckForOverflowUnderflow>true</CheckForOverflowUnderflow>
```

[csharp_039] C# - Configuration - Compiler Options

"Compiler Options - language feature rules": AllowUnsafeBlocks

Sumber: ht

AllowUnsafeBlocks

The **AllowUnsafeBlocks** compiler option allows code that uses the unsafe keyword to compile. The default value for this option is `false`, meaning unsafe code isn't allowed.

For more information about unsafe code, see Unsafe Code and Pointers.

Enable the updated memory safety rules

The updated memory safety rules are a preview feature in C# 15 and .NET 11. They use two independent compiler settings:

- The `preview` language version enables the new syntax and pointer relaxations.
- The `updated-memory-safety-rules` compiler feature enables the updated rules, including *requires-unsafe* caller obligations, and causes the compiler to record the choice in the assembly with the <xref:System.Runtime.CompilerServices.MemorySafetyRulesAttribute> attribute.

A future stable SDK property, `MemorySafetyRules`, is planned as a third activation tier for when the feature exits preview (for example, `<MemorySafetyRules>2</MemorySafetyRules>`), but that property isn't implemented yet.

For a project, use both settings:

For a file-based program, add the equivalent directives:

The **AllowUnsafeBlocks** property is independent. It controls whether the source can use the `unsafe` keyword. A project can enable the updated rules without allowing unsafe code, in which case it receives errors when it calls requires-unsafe APIs.

Whether one assembly enforces the updated rules against another depends on which side opts in:

- **Updated-model caller, updated-model callee**: The callee's `unsafe` markers travel through metadata. The caller wraps each call to a requires-unsafe member in an `unsafe` block.
- **Updated-model caller, original-model callee**: A compatibility mode treats any callee member with a pointer type in its signature as requires-unsafe, so the call site needs an enclosing `unsafe` block. This mode keeps a pointer-based API from silently losing its `unsafe` requirement.
- **Original-model caller, updated-model callee**: The original pointer rules still apply. A requires-unsafe member that has no pointer type in its signature becomes callable from safe code, because the original-model caller can't read the new markers.

Contoh Kode / Perintah CLI:

```
<AllowUnsafeBlocks>true</AllowUnsafeBlocks>

// ----------------------------------------

<PropertyGroup>
  <LangVersion>preview</LangVersion>
  <Features>$(Features);updated-memory-safety-rules</Features>
</PropertyGroup>

// ----------------------------------------

#:property Features=$(Features);updated-memory-safety-rules
#:property LangVersion=preview
```

[csharp_040] C# - Configuration - Compiler Options

"Compiler Options - language feature rules": DefineConstants

Sumber: ht

DefineConstants

The **DefineConstants** option defines symbols in all source code files of your program.

This option specifies the names of one or more symbols that you want to define. The **DefineConstants** option has the same effect as the #define preprocessor directive except that the compiler option is in effect for all files in the project. A symbol remains defined in a source file until an #undef directive in the source file removes the definition. When you use the `-define` option, an `#undef` directive in one file has no effect on other source code files in the project. You can use symbols created by this option with #if, #else, #elif, and #endif to compile source files conditionally. The C# compiler itself defines no symbols or macros that you can use in your source code; all symbol definitions must be user-defined.
The C# `#define` directive doesn't allow a symbol to have a value, as in languages such as C++. For example, `#define` can't create a macro or define a constant. If you need to define a constant, use an `enum` variable. If you want to create a C++-style macro, consider alternatives such as generics. Since macros are notoriously error-prone, C# disallows their

use but provides safer alternatives.

Contoh Kode / Perintah CLI:

```
<DefineConstants>name;name2</DefineConstants>
```

[csharp_041] C# - Configuration - Compiler Options

"Compiler Options - language feature rules": LangVersion

Sumber: ht

LangVersion

The default language version for the C# compiler depends on the target framework for your application and the version of the SDK or Visual Studio installed. Those rules are defined in C# language versioning.
Don't set the `LangVersion` element to `latest`. The `latest` setting means the installed compiler uses its latest version. That version can change from machine to machine, making builds unreliable. In addition, it enables language features that might require runtime or library features that aren't included in the current SDK.

The **LangVersion** option causes the compiler to accept only syntax that's included in the specified C# language specification, for example:

Some preview features require a separate opt-in in addition to `<LangVersion>preview</LangVersion>`. For example, the C# 15 updated memory safety rules use the `updated-memory-safety-rules` compiler feature. For more information, see Enable the updated memory safety rules.

The following values are valid:

!INCLUDE [lang-versions-table]

Considerations

- To ensure that your project uses the default compiler version recommended for your target framework, don't use the **LangVersion** option. Update the target framework to access newer language features.

- Specifying **LangVersion** with the `default` value is different from omitting the **LangVersion** option. Specifying `default` uses the latest version of the language that the compiler supports, without taking into account the target framework. For example, building a project that targets .NET 6 from Visual Studio version 17.6 uses C# 10 if **LangVersion** isn't specified, but uses C# 11 if **LangVersion** is set to `default`.

- The **LangVersion** compiler option doesn't affect metadata referenced by your C# application.

- Because each version of the C# compiler contains extensions to the language specification, **LangVersion** doesn't give you the equivalent functionality of an earlier version of the compiler.

- While C# version updates generally coincide with major .NET releases, the new syntax and features aren't necessarily tied to that specific framework version. Each specific feature has its own minimum .NET API or common language runtime requirements that might allow it to run on down-level frameworks by including NuGet packages or other libraries.

- Regardless of which **LangVersion** setting you use, use the current version of the common language runtime to create your *.exe* or *.dll*. One exception is friend assemblies and ModuleAssemblyName, which work under **-langversion:ISO-1**.

For other ways to specify the C# language version, see C# language versioning.

For information about how to set this compiler option programmatically, see <xref:VSLangProj80.CSharpProjectConfigurationProperties3.LanguageVersion*>.

C# language specification

Version	Link	Description
C# 8.0 and later	[download PDF][csharp-8]	C# Language Specification Version 7: .NET Foundation
C# 7.3	[download PDF][csharp-7]	Standard ECMA-334 7th Edition
C# 6.0	[download PDF][csharp-6]	Standard ECMA-334 6th Edition
C# 5.0	[Download PDF][csharp-5]	Standard ECMA-334 5th Edition
C# 3.0	[Download DOC][csharp-3]	C# Language Specification Version 3.0: Microsoft Corporation
C# 2.0	[Download PDF][csharp-2]	Standard ECMA-334 4th Edition
C# 1.2	[Download DOC][csharp-1.2]	Standard ECMA-334 2nd Edition
C# 1.0	[Download DOC][csharp-1]	Standard ECMA-334 1st Edition

[csharp-8]: /dotnet/csharp/language-reference/language-specification/introduction
[csharp-7]: https://ecma-international.org/wp-content/uploads/ECMA-334_7th_edition_december_2023.pdf
[csharp-6]: https://www.ecma-international.org/wp-content/uploads/ECMA-334_6th_edition_june_2022.pdf
[csharp-5]: https://www.ecma-international.org/wp-content/uploads/ECMA-334_5th_edition_december_2017.pdf
[csharp-3]: https://download.microsoft.com/download/3/8/8/388e7205-bc10-4226-b2a8-75351c669b09/CSharp%20Language%20Specification.doc
[csharp-2]: https://www.ecma-international.org/wp-content/uploads/ECMA-334_4th_edition_june_2006.pdf
[csharp-1.2]: https://www.ecma-international.org/wp-content/uploads/ECMA-334_2nd_edition_december_2002.pdf
[csharp-1]: https://www.ecma-international.org/wp-content/uploads/ECMA-334_1st_edition_december_2001.pdf

Minimum SDK version needed to support all language features

The following table lists the minimum versions of the SDK with the C# compiler that supports the corresponding language version:

C# version	Minimum SDK version
C# 12	Microsoft Visual Studio/Build Tools 2022 version 17.8, or .NET 8 SDK
C# 11	Microsoft Visual Studio/Build Tools 2022 version 17.4, or .NET 7 SDK
C# 10	Microsoft Visual Studio/Build Tools 2022, or .NET 6 SDK
C# 9.0	Microsoft Visual Studio/Build Tools 2019 version 16.8, or .NET 5 SDK
C# 8.0	Microsoft Visual Studio/Build Tools 2019, version 16.3, or .NET Core 3.0 SDK
C# 7.3	Microsoft Visual Studio/Build Tools 2017, version 15.7
C# 7.2	Microsoft Visual Studio/Build Tools 2017, version 15.5
C# 7.1	Microsoft Visual Studio/Build Tools 2017, version 15.3
C# 7.0	Microsoft Visual Studio/Build Tools 2017
C# 6	Microsoft Visual Studio/Build Tools 2015
C# 5	Microsoft Visual Studio/Build Tools 2012 or bundled .NET Framework 4.5 compiler
C# 4	Microsoft Visual Studio/Build Tools 2010 or bundled .NET Framework 4.0 compiler
C# 3	Microsoft Visual Studio/Build Tools 2008 or bundled .NET Framework 3.5 compiler
C# 2	Microsoft Visual Studio/Build Tools 2005 or bundled .NET Framework 2.0 compiler
C# 1.0/1.2	Microsoft Visual Studio/Build Tools .NET 2002 or bundled .NET Framework 1.0 compiler

Contoh Kode / Perintah CLI:

```
<LangVersion>9.0</LangVersion>
```

[csharp_042] C# - Configuration - Compiler Options

"Compiler Options - language feature rules": Nullable

Sumber: ht

Nullable

Use the **Nullable** option to specify the nullable context. Set it in the project's configuration by using the `<Nullable>` tag:

The argument must be one of `enable`, `disable`, `warnings`, or `annotations`. The `enable` argument turns on the nullable context. The `disable` argument turns off the nullable context. The `warnings` argument turns on the nullable warning context. The `annotations` argument turns on the nullable annotation context. For more information about these values, see Nullable contexts. To learn more about enabling nullable reference types in an existing codebase, see nullable migration strategies.
If you don't set a value, the default value is `disable`. However, .NET 6 and newer templates set the **Nullable** value to `enable` by default.

Flow analysis infers the nullability of variables within executable code. The inferred nullability of a variable is independent of the variable's declared nullability. The compiler analyzes method calls even when the call is conditionally omitted from the compiled output. For example, the compiler still analyzes a call to <xref:System.Diagnostics.Debug.Assert*?displayProperty=nameWithType> for nullability even though the call is conditional and isn't compiled into release builds.

Invocation of methods annotated with the following attributes also affects flow analysis:

- Simple preconditions: <xref:System.Diagnostics.CodeAnalysis.AllowNullAttribute> and <xref:System.Diagnostics.CodeAnalysis.DisallowNullAttribute>
- Simple postconditions: <xref:System.Diagnostics.CodeAnalysis.MaybeNullAttribute> and <xref:System.Diagnostics.CodeAnalysis.NotNullAttribute>
- Conditional postconditions: <xref:System.Diagnostics.CodeAnalysis.MaybeNullWhenAttribute> and <xref:System.Diagnostics.CodeAnalysis.NotNullWhenAttribute>
- <xref:System.Diagnostics.CodeAnalysis.DoesNotReturnIfAttribute> (for example, `DoesNotReturnIf(false)` for <xref:System.Diagnostics.Debug.Assert*?displayProperty=nameWithType>) and <xref:System.Diagnostics.CodeAnalysis.DoesNotReturnAttribute>
- <xref:System.Diagnostics.CodeAnalysis.NotNullIfNotNullAttribute>
- Member postconditions: <xref:System.Diagnostics.CodeAnalysis.MemberNotNullAttribute.%23ctor(System.String)> and <xref:System.Diagnostics.CodeAnalysis.MemberNotNullAttribute.%23ctor(System.String[])>

The global nullable context doesn't apply to generated code files. Regardless of this setting, the nullable context is *disabled* for any source file marked as generated. A file is marked as generated in one of the following ways:
1. In the .editorconfig, specify `generated_code = true` in a section that applies to that file.
1. Include `<auto-generated>` or `<auto-generated/>` in a comment at the top of the file. You can place it on any line in the comment, but the comment block must be the first element in the file.
1. Start the file name with *TemporaryGeneratedFile_*
1. End the file name with *.designer.cs*, *.generated.cs*, *.g.cs*, or *.g.i.cs*.

Generators can opt in by using the `#nullable` preprocessor directive.

Contoh Kode / Perintah CLI:

```
<Nullable>enable</Nullable>
```

[csharp_043] C# - Configuration - CLI Tools

dotnet new <TEMPLATE>: Overview

Sumber: ht

dotnet new <TEMPLATE>

This article applies to: [OK]? .NET 6 SDK and later versions

[csharp_044] C# - Configuration - CLI Tools

dotnet new <TEMPLATE>: Name

Sumber: ht

Name

`dotnet new` - Creates a new project, configuration file, or solution based on the specified template.

[csharp_045] C# - Configuration - CLI Tools

dotnet new <TEMPLATE>: Synopsis

Sumber: ht

Synopsis

Contoh Kode / Perintah CLI:

```
dotnet new <TEMPLATE> [--dry-run] [--force] [-lang|--language {"C#"|"F#"|VB}]
    [-n|--name <OUTPUT_NAME>] [-f|--framework <FRAMEWORK>] [--no-update-check]
    [-o|--output <OUTPUT_DIRECTORY>] [--project <PROJECT_PATH>]
    [-d|--diagnostics] [--verbosity <LEVEL>] [Template options]

dotnet new -h|--help
```

[csharp_046] C# - Configuration - CLI Tools

dotnet new <TEMPLATE>: Description

Sumber: ht

Description

The `dotnet new` command creates a .NET project or other artifacts based on a template.

The command calls the template engine to create the artifacts on disk based on the specified template and options.
!INCLUDE [new syntax]

Tab completion

Starting with .NET SDK 7.0.100, tab completion is available for `dotnet new`. It supports completion for installed template names, as well as completion for the options a selected template provides.
To activate tab completion for the .NET SDK, see Enable tab completion.

Implicit restore

!INCLUDE[dotnet restore note]

[csharp_047] C# - Configuration - CLI Tools

dotnet new <TEMPLATE>: Arguments

Sumber: ht

Arguments

- **`TEMPLATE`**

  The template to instantiate when the command is invoked. Each template might have specific options you can pass. For more information, see Template options.

  You can run `dotnet new list` to see a list of all installed templates.

  The `search` command should be used to search for templates in NuGet.org.

  For a list of templates that ship with the .NET SDK, see Preinstalled templates.

[csharp_048] C# - Configuration - CLI Tools

dotnet new <TEMPLATE>: Options

Sumber: ht

Options

- **`--dry-run`**

  Displays a summary of what would happen if the given command were run if it would result in a template creation.

- **`--force`**

  Forces content to be generated even if it would change existing files. This is required when the template chosen would override existing files in the output directory.

- !INCLUDE [help]

- **`-lang|--language {C#|F#|VB}`**

  The language of the template to create. The language accepted varies by the template (see defaults in the arguments section). Not valid for some templates.
Some shells interpret `#` as a special character. In those cases, enclose the language parameter value in quotes. For example, `dotnet new console -lang "F#"`.

- **`-n|--name <OUTPUT_NAME>`**

  The name for the created output. If no name is specified, the name of the current directory is used.

- **`-f|--framework <FRAMEWORK>`**

  Specifies the target framework. It expects a target framework moniker (TFM). Examples: "net6.0", "net7.0-macos". This value will be reflected in the project file.

- **`-no-update-check`**

  Disables checking for template package updates when instantiating a template. Available since .NET SDK 6.0.100.

When instantiating the template from a template package that was installed by using `dotnet new --install`, `dotnet new` checks if there is an update for the template.
Starting with .NET 6, no update checks are done for .NET default templates. To update .NET default templates, install the patch version of the .NET SDK.

- **`-o|--output <OUTPUT_DIRECTORY>`**

Location to place the generated output. The default is the current directory.

- **`--project <PROJECT_PATH>`**

The project that the template is added to. This project is used for context evaluation. If not specified, the project in the current or parent directories will be used. Available since .NET SDK 7.0.100.

- **`-d|--diagnostics`**

Enables diagnostic output. Available since .NET SDK 7.0.100.

- !INCLUDE [verbosity]

[csharp_049] C# - Configuration - CLI Tools

dotnet new <TEMPLATE>: Template options

Sumber: ht

Template options

Each template may have additional options defined. For more information, see .NET default templates for `dotnet new`.

[csharp_050] C# - Configuration - CLI Tools

dotnet new <TEMPLATE>: Examples

Sumber: ht

Examples

- Create a C# console application project:

- Create an F# console application project in the current directory:

- Create a .NET Standard 2.0 class library project in the specified directory:

- Create a new ASP.NET Core C# MVC project in the current directory with no authentication:

- Create a new xUnit project:

- Create a *global.json* in the current directory setting the SDK version to 8.0.101:

- Show help for the C# console application template:

- Show help for the F# console application template:

Contoh Kode / Perintah CLI:

```
dotnet new console

// ----------------------------------------

dotnet new console --language "F#"

// ----------------------------------------

dotnet new classlib --framework "netstandard2.0" -o MyLibrary

// ----------------------------------------

dotnet new mvc -au None

// ----------------------------------------

dotnet new xunit

// ----------------------------------------

dotnet new globaljson --sdk-version 8.0.101 --roll-forward latestFeature

// ----------------------------------------

dotnet new console -h

// ----------------------------------------

dotnet new console --language "F#" -h
```

[csharp_051] C# - Configuration - CLI Tools

dotnet new <TEMPLATE>: See also

Sumber: ht

See also

- dotnet new list command
- dotnet new search command
- dotnet new install command
- .NET default templates for dotnet new
- .NET templates for authors
- Create a custom template for dotnet new

[csharp_052] C# - Language Overview

Overview

Sumber: ht

A tour of the C# language

This article gives you a high-level overview of the C# language. Whether you're writing your first program or you're an experienced developer exploring C#, you'll find the key concepts and features here.
New to programming? Skim this article for the big picture, then start the beginner tutorials to write code hands-on.
Coming from Java, JavaScript, or Python? Read the tips for Java, JavaScript, or Python developers alongside this article.
Experienced developer new to C#? This article covers what makes C# distinctive. See What you can build with C# to find the workload that fits your goals.

The C# language is the most popular language for the .NET platform, a free, cross-platform, open source development environment. C# programs can run on many different devices, from Internet of Things (IoT) devices to the cloud and everywhere in between. You can write apps for phone, desktop, and laptop computers and servers. See What you can build with C# for an overview of application types.

C# is a cross-platform general purpose language that makes developers productive while writing highly performant code. C# has broad support in the ecosystem and many workloads. Based on object-oriented principles, it incorporates many features from other paradigms, not least functional programming. Low-level features support high-efficiency scenarios without writing unsafe code. Most of the runtime and libraries are written in C#, and advances in C# often benefit all .NET developers.

C# is in the C family of languages. C# syntax is familiar if you used C, C++, JavaScript, TypeScript, or Java. Like C and C++, semicolons (`;`) define the end of statements. C# identifiers are case-sensitive. C# has the same use of braces, `{` and `}`, control statements like `if`, `else`, and `switch`, and looping constructs like `for` and `while`. C# also has a `foreach` statement for any collection type.

[csharp_053] C# - Language Overview

Overview: Hello world

Sumber: ht

Hello world

The "Hello, World" program traditionally introduces a programming language. Here's the program in C#:

The line starting with `//` is a *single line comment*. C# single line comments start with `//` and continue to the end of the current line. C# also supports *multi-line comments*. Multi-line comments start with `/*` and end with `*/`. The `WriteLine` method of the `Console` class, which is in the `System` namespace, produces the output of the program. The standard class libraries provide this class, and every C# program automatically references these libraries by default. Another program form requires you to declare the containing class and method for the program's entry point. The compiler synthesizes these elements when you use top-level statements.

This alternative format is still valid and contains many of the basic concepts in all C# programs. Many existing C# samples use the following equivalent format:

The preceding "Hello, World" program starts with a `using` directive that references the `System` namespace.

Namespaces provide a hierarchical means of organizing C# programs and libraries. Namespaces contain types and other namespaces. For example, the `System` namespace contains many types, such as the `Console` class referenced in the program, and many other namespaces, such as `IO` and `Collections`. A `using` directive that references a given namespace enables unqualified use of the types that are members of that namespace. Because of the `using` directive, the program can use `Console.WriteLine` as shorthand for `System.Console.WriteLine`. In the earlier example, that namespace was implicitly included.
The preceding program declares the `Program` class with a single member: the method named `Main`. Many existing C# samples and tutorials use this format. By convention, when there are no top-level statements, a static method named `Main` serves as the entry point of a C# program. Both formats compile to the same code. You don't need to use this format -- file-based apps and top-level statements are simpler ways to get started.

Contoh Kode / Perintah CLI:

```
// This line prints "Hello, World"
Console.WriteLine("Hello, World");

// ----------------------------------------

using System;
namespace TourOfCsharp;

class Program
{
    static void Main()
    {
        // This line prints "Hello, World"
        Console.WriteLine("Hello, World");
    }
}
```

[csharp_054] C# - Language Overview

Overview: File-based apps

Sumber: ht

File-based apps

C# is a *compiled* language. In most C# programs, you use the `dotnet build` command to compile a group of source files into a binary package. Then, you use the `dotnet run` command to run the program. (You can simplify this process because `dotnet run` compiles the program before running it if necessary.) These tools support a rich language of configuration options and command-line switches. The `dotnet` command line interface (CLI), which is included in the .NET SDK, provides many tools to generate and modify C# files.

Beginning with C# 14 and .NET 10, you can create *file-based apps*, which simplifies building and running C# programs. You use the `dotnet run` command to run a program contained in a single `*.cs` file. For example, if the following code is stored in a file named `hello-world.cs`, you can run it by typing `dotnet run hello-world.cs`:

The first line of the program contains the `#!` sequence (shebang) for unix operating systems. This enables executing the file directly using its name when the *execute* (`+x`) permission is set on the file. For example, you can run the C# file directly from the command line:

The source for these programs must be a single file, but otherwise all C# syntax is valid. You can use file-based apps for small command-line utilities, prototypes, or other experiments.

Contoh Kode / Perintah CLI:

```
./hello-world.cs

// ----------------------------------------

#!/usr/bin/env dotnet

Console.WriteLine("Hello, World!");
```

[csharp_055] C# - Language Overview

Overview: Familiar C# features

Sumber: ht

Familiar C# features

C# is approachable for beginners yet offers advanced features for experienced developers writing specialized applications. You can be productive quickly. You can learn more specialized techniques as you need them for your applications.

C# apps benefit from the runtime's automatic memory management. C# apps also use the extensive runtime libraries provided by the .NET SDK. Some components are platform independent, like file system libraries, data collections, and math libraries. Others are specific to a single workload, like the ASP.NET Core web libraries or the .NET MAUI UI library. A rich open source ecosystem on NuGet augments the libraries that are part of the runtime. These libraries provide even more components you can use.

C# is a *strongly typed* language. Every variable you declare has a type known at compile time. The compiler, or editing tools, tell you if you're using that type incorrectly. You can fix those errors before you ever run your program. Fundamental data types are built into the language and runtime: value types like `int`, `double`, `char`, reference types like `string`, arrays, and other collections. As you write your programs, you create your own types. Those types can be `struct` types for values, or `class` types that define object-oriented behavior. You can add the `record` modifier to either `struct` or `class` types so the compiler synthesizes code for equality comparisons. You can also create `interface` definitions, which define a contract, or a set of members, that a type implementing that interface must provide. You can also define generic types and methods. Generics use *type parameters* to provide a placeholder for an actual type when used.

As you write code, you define functions, also called methods, as members of `struct` and `class` types. These methods define the behavior of your types. You can overload methods with different numbers or types of parameters. Methods can optionally return a value. In addition to methods, C# types can have properties, which are data elements backed by functions called *accessors*. C# types can define events, which allow a type to notify subscribers of important actions. C# supports object oriented techniques such as inheritance and polymorphism for `class` types.

C# apps use exceptions to report and handle errors. This practice is familiar if you used C++ or Java. Your code throws an exception when it can't do what was intended. Other code, no matter how many levels up the call stack, can optionally recover by using a `try` - `catch` block.
To learn more about types, methods, and exceptions, visit the C# fundamentals section. It covers the type system, object-oriented programming, and exception handling in depth.

[csharp_056] C# - Language Overview

Overview: Distinctive C# features

Sumber: ht

Distinctive C# features

Some elements of C# might be less familiar.

C# provides pattern matching. Those expressions enable you to inspect data and make decisions based on its characteristics. Pattern matching provides a great syntax for control flow based on data. The following code shows how methods for the boolean *and*, *or*, and *xor* operations could be expressed using pattern matching syntax:

Pattern matching expressions can be simplified by using `_` as a catchall for any value. The following example shows how you can simplify the *and* method:

The preceding examples also declare *tuples*, lightweight data structures. A *tuple* is an ordered, fixed-length sequence of values with optional names and individual types. You enclose the sequence in `(` and `)` characters. The declaration `(left, right)` defines a tuple with two boolean values: `left` and `right`. Each switch arm declares a tuple value such as `(true, true)`. Tuples provide convenient syntax to declare a single value with multiple values of any type.

Collection expressions provide a common syntax to provide collection values. You write values or expressions between `[` and `]` characters and the compiler converts that expression to the required collection type:

The previous example shows different collection types that you can initialize by using collection expressions. One example uses the `[]` empty collection expression to declare an empty collection. Another example uses the `..` *spread element* to expand a collection and add all its values to the collection expression.

You can use *index* and *range* expressions to retrieve one or more elements from an indexable collection:

The `^` index indicates *from the end* rather than from the start. The `^0` element is one past the end of the collection, so `^1` is the last element. The `..` in a range expression denotes the range of elements to include. The range starts with the first index and includes all elements up to, but not including, the element at the last index.

For more information about index and range expressions, see the Explore indexes and ranges article.

Language integrated query (LINQ) provides a common pattern-based syntax to query or transform any collection of data. LINQ unifies the syntax for querying in-memory collections, structured data like XML or JSON, database storage, and even cloud based data APIs. You learn one set of syntax and you can search and manipulate data regardless of its storage. The following query finds all students whose grade point average is greater than 3.5:

The preceding query works for many storage types represented by `Students`. It could be a collection of objects, a database table, a cloud storage blob, or an XML structure. The same query syntax works for all storage types.

The Task based asynchronous programming model enables you to write code that reads as though it runs synchronously, even though it runs asynchronously. It utilizes the `async` and `await` keywords to describe methods that are asynchronous, and when an expression evaluates asynchronously. The following sample awaits an asynchronous web request. When the asynchronous operation completes, the method returns the length of the response:

C# also supports an `await foreach` statement to iterate a collection backed by an asynchronous operation, like a GraphQL paging API. The following sample reads data in chunks, returning an iterator that provides access to each element when it's available:

Callers can iterate the collection by using an `await foreach` statement:

Finally, as part of the .NET ecosystem, you can use Visual Studio or Visual Studio Code with the C# Dev Kit. These tools provide a rich understanding of C#, including the code you write. They also provide debugging capabilities.
To learn more about pattern matching, LINQ, and async programming, see the pattern matching, LINQ overview, and asynchronous programming sections.

Contoh Kode / Perintah CLI:

```
public static bool Or(bool left, bool right) =>
        (left, right) switch
        {
            (true, true) => true,
            (true, false) => true,
            (false, true) => true,
            (false, false) => false,
        };

    public static bool And(bool left, bool right) =>
        (left, right) switch
        {
            (true, true) => true,
            (true, false) => false,
            (false, true) => false,
            (false, false) => false,
        };
    public static bool Xor(bool left, bool right) =>
        (left, right) switch
        {
            (true, true) => false,
            (true, false) => true,
            (false, true) => true,
            (false, false) => false,
        };

// ----------------------------------------

public static bool ReducedAnd(bool left, bool right) =>
        (left, right) switch
        {
            (true, true) => true,
            (_, _) => false,
```

```
        };

// ----------------------------------------

int[] numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
            List<string> names = ["Alice", "Bob", "Charlie", "David"];

            IEnumerable<int> moreNumbers = [.. numbers, 11, 12, 13];
            IEnumerable<string> empty = [];

// ----------------------------------------

string second = names[1]; // 0-based index
            string last = names[^1]; // ^1 is the last element
            int[] smallNumbers = numbers[0..5]; // elements at indexes 0 to 4

// ----------------------------------------

var honorRoll = from student in Students
                        where student.GPA > 3.5
                        select student;

// ----------------------------------------

public static async Task<int> GetPageLengthAsync(string endpoint)
    {
        var client = new HttpClient();
        var uri = new Uri(endpoint);
        byte[] content = await client.GetByteArrayAsync(uri);
        return content.Length;
    }

// ----------------------------------------

public static async IAsyncEnumerable<int> ReadSequence()
    {
        int index = 0;
        while (index < 100)
        {
            int[] nextChunk = await GetNextChunk(index);
            if (nextChunk.Length == 0)
            {
                yield break;
            }
            foreach (var item in nextChunk)
            {
                yield return item;
            }
            index++;
        }
    }

// ----------------------------------------

await foreach (var number in ReadSequence())
        {
            Console.WriteLine(number);
        }
```

[csharp_057] C# - Language Overview

Overview: Next steps

Sumber: ht

Next steps

This article provided a quick tour of the C# language. Here's where to go next, depending on your experience:

- **Start coding**: Work through the beginner tutorials to learn C# step by step.
- **Go deeper**: Visit the C# fundamentals section for detailed coverage of the type system, object-oriented programming, and error handling.
- **Build something**: Explore what you can build with C# to find ideas that interest you.
- **Coming from another language?** Read the guides for Java, JavaScript, or Python developers.
- **Build file-based apps**: Learn how to build file-based apps for small programs and prototypes.

[csharp_058] C# - Program Structure

"Top-level statements - programs without Main methods": Overview

Sumber: ht

Top-level statements - programs without `Main` methods
New to developing software? Start with the Get started tutorials first. Those tutorials use top-level statements, so you'll already be familiar with the basics.
Looking for the `Main` method alternative? See Main method entry point for the explicit `Main` method approach.

Use *top-level statements* for new apps. By using top-level statements, you can write executable code directly at the root of a file.

Here's a *Program.cs* file that is a complete C# program:

When you create a new console app by using `dotnet new console`, it uses top-level statements by default. They work well for programs of any size - from small utilities and Azure Functions to full applications. If you have an existing application that uses an explicit `Main` method, there's no need to convert it. Both styles compile to equivalent code.

The following sections explain the rules on what you can and can't do with top-level statements.

Contoh Kode / Perintah CLI:

```
Console.WriteLine("Hello World!");
```

[csharp_059] C# - Program Structure

"Top-level statements - programs without Main methods": Entry point rules

Sumber: ht

Entry point rules

An application must have only one entry point. A project can have only one file with top-level statements, but it can have any number of source code files that don't have top-level statements. You can explicitly write a `Main` method, but it can't function as an entry point. In a project with top-level statements, you can't use the `-main` compiler option to select the entry point, even if the project has one or more `Main` methods.

The compiler generates a method to serve as the program entry point for a project with top-level statements. The signature of the method depends on whether the top-level statements contain the `await` keyword or the `return` statement. The following table shows what the method signature looks like, using the method name `Main` in the table

for convenience.

Top-level code contains	Implicit `Main` signature
`await` and `return`	`static async Task<int> Main(string[] args)`
`await`	`static async Task Main(string[] args)`
`return`	`static int Main(string[] args)`
No `await` or `return`	`static void Main(string[] args)`

Starting with C# 14, programs can be *file-based apps*, where a single file contains the program. You run *file-based apps* by using the command `dotnet <file.cs>`, or directly using the filename on Unix (for example, `./file.cs`). The latter requires including the `#!/usr/bin/env dotnet` directive as the first line and setting the execute permission (`chmod +x <file>`).

[csharp_060] C# - Program Structure

"Top-level statements - programs without Main methods": `using` directives

Sumber: ht

`using` directives

For the single file containing top-level statements, `using` directives must come first in that file, as in the following example:

Contoh Kode / Perintah CLI:

```
using System.Text;

StringBuilder builder = new();
builder.AppendLine("The following arguments are passed:");

foreach (var arg in args)
{
    builder.AppendLine($"Argument={arg}");
}

Console.WriteLine(builder.ToString());

return 0;
```

[csharp_061] C# - Program Structure

"Top-level statements - programs without Main methods": Namespaces and type definitions

Sumber: ht

Namespaces and type definitions

Top-level statements are implicitly in the global namespace. A file with top-level statements can also contain namespaces and type definitions, but they must come after the top-level statements. For example:

Contoh Kode / Perintah CLI:

```
MyClass.TestMethod();
MyNamespace.MyClass.MyMethod();

public class MyClass
{
    public static void TestMethod()
    {
        Console.WriteLine("Hello World!");
```

```
    }
}

namespace MyNamespace
{
    class MyClass
    {
        public static void MyMethod()
        {
            Console.WriteLine("Hello World from MyNamespace.MyClass.MyMethod!");
        }
    }
}
```

[csharp_062] C# - Program Structure

"Top-level statements - programs without Main methods": `args`

Sumber: ht

`args`

Top-level statements can reference the `args` variable to access any command-line arguments passed to the app when it starts. The `args` variable is never `null`, but its `Length` is zero if no command-line arguments were provided. For example:

Contoh Kode / Perintah CLI:

```
if (args.Length > 0)
{
    foreach (var arg in args)
    {
        Console.WriteLine($"Argument={arg}");
    }
}
else
{
    Console.WriteLine("No arguments");
}
```

[csharp_063] C# - Program Structure

"Top-level statements - programs without Main methods": `await` and exit code

Sumber: ht

`await` and exit code

Use `await` to call an async method. When your top-level code contains `await`, the compiler generates an entry point that returns a `Task`. The runtime monitors that `Task` for completion, keeping the process alive until all asynchronous work finishes. For example:

To return an exit code when the application ends, use the `return` statement. The compiler generates an entry point that returns `Task<int>` when your code contains both `await` and `return`, or `int` when it contains only `return`. For example:

Contoh Kode / Perintah CLI:

```
Console.Write("Hello ");
await Task.Delay(5000);
Console.WriteLine("World!");
```

```
// ----------------------------------------

string? s = Console.ReadLine();

int returnValue = int.Parse(s ?? "-1");
return returnValue;
```

[csharp_064] C# - Program Structure

"Top-level statements - programs without Main methods": Related content

Sumber: ht

Related content

- Main() and command-line arguments
- General structure of a C# program
- Feature specification - Top-level statements

[csharp_065] C# - Type System

"The C# type system": Overview

Sumber: ht

The C# type system
New to developing software? Start with the Get started tutorials first. They walk you through writing programs and introduce types as you go.
Experienced in another language? If you already understand type systems, skim the value vs. reference distinction and the choose which kind of type guide, then jump to the articles on specific types.

C# is a strongly typed language. Every variable, constant, and expression has a type. The compiler enforces *type safety* by checking that every operation in your code is valid for the types involved. For example, you can add two `int` values, but you can't add an `int` and a `bool`:
Unlike C and C++, in C#, `bool` isn't convertible to `int`.

Type safety catches errors at compile time, before your code runs. The compiler also embeds type information into the executable as metadata, which the common language runtime (CLR) uses for additional safety checks at run time.

Contoh Kode / Perintah CLI:

```
int a = 5;
int b = a + 2; // OK

bool test = true;

// Error. Operator '+' cannot be applied to operands of type 'int' and 'bool'.
// int c = a + test;
```

[csharp_066] C# - Type System

"The C# type system": Declare variables with types

Sumber: ht

Declare variables with types

When you declare a variable, you specify its type explicitly or use `var` to let the compiler infer the type from the assigned value:

Method parameters and return values also have types. The following method takes a `string` and an `int`, and returns a `string`:

After you declare a variable, you can't change its type or assign a value that's incompatible with the declared type. You can convert values to other types. The compiler performs *implicit conversions* that don't lose data automatically. *Explicit conversions* (casts) require you to indicate the conversion in your code. For more information, see Casting and type conversions.

Contoh Kode / Perintah CLI:

```
// Explicit type:
int count = 10;
double temperature = 36.6;

// Compiler-inferred type:
var name = "C#";
var items = new List<string> { "one", "two", "three" };

// ----------------------------------------

static string GetGreeting(string name, int visitCount)
{
    return visitCount switch
    {
        1 => $"Welcome, {name}!",
        _ => $"Welcome back, {name}! Visit #{visitCount}."
    };
}
```

[csharp_067] C# - Type System

"The C# type system": Built-in types and custom types

Sumber: ht

Built-in types and custom types

C# provides built-in types for common data: integers, floating-point numbers, `bool`, `char`, and `string`. Every C# program can use these built-in types without any extra references.

Beyond built-in types, you can create your own types by using several constructs:

- **Classes** -- Reference types for modeling behavior and complex objects. Support inheritance and polymorphism.
- **Structs** -- Value types for small, lightweight data. Each variable holds its own copy.
- **Records** -- Classes or structs with compiler-generated equality, `ToString`, and nondestructive mutation through `with` expressions.
- **Interfaces** -- Contracts that define members any class or struct can implement.
- **Enumerations** -- Named sets of integral constants, such as days of the week or file access modes.
- **Tuples** -- Lightweight structural types that group related values without defining a named type.
- **Generics** -- Type-parameterized constructs like `List<T>` and `Dictionary<TKey, TValue>` that provide type safety while reusing the same logic for different types.

[csharp_068] C# - Type System

"The C# type system": Value types and reference types

Sumber: ht

Value types and reference types

Every type in C# is either a *value type* or a *reference type*. This distinction determines how variables store data and how assignment works.

Value types hold their data directly. When you assign a value type to a new variable, the runtime copies the data. Changes to one variable don't affect the other. Structs, enums, and the built-in numeric types are all value types.

Reference types hold a reference to an object on the managed heap. When you assign a reference type to a new variable, both variables point to the same object. Changes through one variable are visible through the other. Classes, arrays, delegates, and strings are reference types.

The following example shows the difference. The first block shows the definition for the `Coords` record struct, which is a value type. The second block shows the different behavior for value types and reference types.

All types ultimately derive from <xref:System.Object?displayProperty=nameWithType>. Value types derive from <xref:System.ValueType?displayProperty=nameWithType>, which derives from `object`. This unified hierarchy is called the Common Type System (CTS). For more information about inheritance, see Inheritance.

Contoh Kode / Perintah CLI:

```
public readonly record struct Coords(int X, int Y);

// ----------------------------------------

// Value type: each variable holds its own copy
var point1 = new Coords(3, 4);
var point2 = point1;
Console.WriteLine($"point1: ({point1.X}, {point1.Y})");
Console.WriteLine($"point2: ({point2.X}, {point2.Y})");
// point1 and point2 are independent copies

// Reference type: both variables refer to the same object
var list1 = new List<int> { 1, 2, 3 };
var list2 = list1;
list2.Add(4);
Console.WriteLine($"list1 count: {list1.Count}"); // 4  --  same object
```

[csharp_069] C# - Type System

"The C# type system": Choose which kind of type

Sumber: ht

Choose which kind of type

When you define a new type, the kind you choose shapes how your code behaves. Use the following guidelines to make an initial decision:

- **Tuple** -- Temporary grouping of values that doesn't need a named type or behavior.
- **`struct`** or **`record struct`** -- Small data (roughly 64 bytes or less), value semantics, or immutability. Record

structs add value-based equality and `with` expressions.
- **`record class`** -- Primarily data, with value-based equality, `ToString`, and nondestructive mutation. Supports inheritance.
- **`class`** -- Complex behavior, polymorphism, or mutable state. Most custom types are classes.
- **`interface`** -- A contract that unrelated types can implement. Defines capabilities rather than identity.
- **`enum`** -- A fixed set of named constants, such as status codes or options.

More than one option is often reasonable.

[csharp_070] C# - Type System

"The C# type system": Compile-time type and run-time type

Sumber: ht

Compile-time type and run-time type

A variable can have different types at compile time and run time. The *compile-time type* is the declared or inferred type in source code. The *run-time type* is the actual type of the instance the variable refers to. The run-time type must be the same as the compile-time type, or a type that derives from it or implements it. An assignment is only valid when an implicit conversion exists from the run-time type to the compile-time type, such as an identity, reference, boxing, or numeric conversion.

In the preceding example, `boxed` has a compile-time type of `object` but a run-time type of `string`. The assignment works because `string` derives from `object`. Similarly, `characters` has a compile-time type of `IEnumerable<char>`, and the assignment works because `string` implements that interface. The compile-time type controls overload resolution and available conversions. The run-time type controls virtual method dispatch, `is` expressions, and `switch` expressions.

Contoh Kode / Perintah CLI:

```
// Compile-time and run-time types match:
string message = "Hello, world!";

// Compile-time type differs from run-time type:
object boxed = "This is a string at run time";
IEnumerable<char> characters = "abcdefghijklmnopqrstuvwxyz";
```

[csharp_071] C# - Type System

"The C# type system": See also

Sumber: ht

See also

- Value types
- Reference types
- Common Type System

[csharp_072] C# - Type System

"The C# type system": C# language specification

Sumber: ht

C# language specification

!INCLUDE[CSharplangspec]

[csharp_073] C# - Classes & Records

"C# record types": Overview

Sumber: ht

C# record types
New to developing software? Start with the Get started tutorials first. You'll encounter records when you need concise data types with built-in equality.
Experienced in another language? C# records are similar to data classes in Kotlin or case classes in Scala. They're types optimized for storing data, with compiler-generated equality, `ToString`, and copy semantics. Skim the `record class` vs `record struct` and `with` expressions sections for C#-specific patterns.

The `record` keyword is a modifier you apply to either a `class` or a `struct`. It tells the compiler to generate value equality, a formatted `ToString`, and nondestructive mutation through `with` expressions. The underlying type, a class or a struct, still determines whether instances use reference or value semantics. The `record` modifier adds data-friendly behavior on top of those semantics. Use records when a type's primary role is storing data and two instances with the same values should be considered equal.

[csharp_074] C# - Classes & Records

"C# record types": When to use records

Sumber: ht

When to use records

Use a record when all of the following conditions are true:

- The type's primary role is storing data.
- Two instances with the same values should be equal.
- You want immutability (especially for `record class` types).
- You want a readable `ToString` without writing one manually.

When choosing between `record class` and `record struct`:

- Use `record class` when you need inheritance, or when the type is large enough that copying on every assignment would be expensive.
- Use `record struct` for small, self-contained values where copy semantics and stack allocation are beneficial.

Avoid records for entity types in Entity Framework Core, which depends on reference equality to track entities. For a broader comparison of type options, see Choose which kind of type.

[csharp_075] C# - Classes & Records

"C# record types": Declare a record

Sumber: ht

Declare a record

You can apply `record` to either a class or a struct. The simplest form uses *positional parameters* that define both the constructor and the properties in a single line:

The same positional syntax works for `record struct` types:

Writing `record` alone is shorthand for `record class` which is a reference type. Writing `record struct` creates a value type. The compiler generates properties from the positional parameters in both cases, but the defaults differ:

- **`record class`**: Properties are `init`-only (immutable after construction).
- **`record struct`**: Properties are read-write by default. Add `readonly` (`readonly record struct`) to make them `init`-only.

You can also write records with standard property syntax when you need more control. For example, to make a property read/write instead of `init`-only:

Contoh Kode / Perintah CLI:

```
public record Person(string FirstName, string LastName);

// ----------------------------------------

public record struct Coordinate(double Latitude, double Longitude);

public readonly record struct Temperature(double Celsius)
{
    public double Fahrenheit => Celsius * 9.0 / 5.0 + 32.0;
}

// ----------------------------------------

public record Product
{
    public required string Name { get; init; }
    public decimal Price { get; set; }
}
```

[csharp_076] C# - Classes & Records

"C# record types": `record class` vs. `record struct`

Sumber: ht

`record class` vs. `record struct`

Because the `record` modifier preserves the underlying type's semantics, a `record class` and a `record struct` behave differently when you assign or compare references. Assigning a `record class` copies the reference. Both variables point to the same object. Assigning a `record struct` copies the data, so changes to one variable don't affect the other:

Record structs also provide compiler-generated value equality:

Choose `record class` when you need inheritance or when instances are large enough that copying would be expensive. Choose `record struct` for small, self-contained data where value-type copy semantics are appropriate. For more on value type semantics, see Structs.

Contoh Kode / Perintah CLI:

```
// Record class  --  assignment copies the reference
        var p1 = new Person("Grace", "Hopper");
        var p2 = p1; // p1 and p2 point to the same object:
        Console.WriteLine(ReferenceEquals(p1, p2)); // True

        // Record struct  --  assignment copies the data
        var c1 = new Coordinate(47.6062, -122.3321);
        var c2 = c1;
        c2.Longitude = 0.0; // mutating c2 doesn't affect c1
        Console.WriteLine(c1.Longitude); // -122.3321
        Console.WriteLine(c2.Longitude); // 0

// ----------------------------------------

var home = new Coordinate(47.6062, -122.3321);
        var copy = home;

        Console.WriteLine(home);           // Coordinate { Latitude = 47.6062, Longitude = -122.3321 }
        Console.WriteLine(home == copy);   // True  --  value equality
```

[csharp_077] C# - Classes & Records

"C# record types": Value equality

Sumber: ht

Value equality

The `record` modifier gives both classes and structs compiler-generated, property-by-property equality. Here's how equality works across all four type kinds:

- **Plain class**: Uses *reference equality* by default. The `==` operator checks whether two variables point to the same object, not whether the data matches.
- **Plain struct**: Supports value equality through <xref:System.ValueType.Equals*?displayProperty=nameWithType>, but the default implementation uses reflection, which is slower and doesn't generate `==`/`!=` operators.
- **`record class`**: The compiler generates `Equals` and `GetHashCode` methods, and `==`/`!=` operators, that compare every property value. Two distinct objects with the same data are equal.
- **`record struct`**: Same compiler-generated equality as a `record class`, but without using reflection, which makes it faster than plain struct equality.

The following example demonstrates record class equality:

The two `Person` instances are different objects, but they're equal because all their property values match. Array properties compare by reference, not by contents. Mutating the shared array is visible through both records because the array itself isn't copied.

Contoh Kode / Perintah CLI:

```
// Person is a record type with three properties: FirstName, LastName, and PhoneNumbers.
        var phones = new string[] { "555-1234" };
        var person1 = new Person("Grace", "Hopper", phones);
        var person2 = new Person("Grace", "Hopper", phones);

        Console.WriteLine(person1 == person2);              // True
        Console.WriteLine(ReferenceEquals(person1, person2)); // False

        person1.PhoneNumbers[0] = "555-9999";
        Console.WriteLine(person2.PhoneNumbers[0]); // 555-9999  --  same array
```

[csharp_078] C# - Classes & Records

"C# record types": Nondestructive mutation with `with` expressions

Sumber: ht

Nondestructive mutation with `with` expressions

Records are often immutable, so you can't change a property after creation. A `with` expression creates a copy with one or more properties changed, leaving the original record unchanged. This approach works for both `record class` and `record struct` types:

The same syntax works for `record struct` types:

A `with` expression copies the existing instance, then applies the specified property changes.

Contoh Kode / Perintah CLI:

```
var original = new Person("Grace", "Hopper");
        var modified = original with { FirstName = "Margaret" };

        Console.WriteLine(original); // Person { FirstName = Grace, LastName = Hopper }
        Console.WriteLine(modified); // Person { FirstName = Margaret, LastName = Hopper }
        Console.WriteLine(original == modified); // False

        var copy = original with { };
        Console.WriteLine(original == copy); // True

// ----------------------------------------

var shifted = home with { Longitude = -122.0 };
        Console.WriteLine(shifted);        // Coordinate { Latitude = 47.6062, Longitude = -122 }
        Console.WriteLine(home == shifted); // False
```

[csharp_079] C# - Classes & Records

"C# record types": Positional records and deconstruction

Sumber: ht

Positional records and deconstruction

Positional records generate a `Deconstruct` method that you use to extract property values into individual variables:

Deconstruction works with both `record class` and `record struct` types. You can use it in assignments, `foreach` loops,

and pattern matching.

Contoh Kode / Perintah CLI:

```
var (first, last) = person;
        Console.WriteLine($"{first} {last}");
        // Grace Hopper
```

[csharp_080] C# - Classes & Records

"C# record types": Record inheritance

Sumber: ht

Record inheritance

A `record class` can inherit from another `record class`. A record can't inherit from a regular class, and a class can't inherit from a record:

Value equality checks include the run-time type, so a `Person` and a `Student` with the same `FirstName` and `LastName` aren't considered equal. Record structs don't support inheritance because structs can't inherit from other types.

Contoh Kode / Perintah CLI:

```
public record Student(string FirstName, string LastName, int GradeLevel)
    : Person(FirstName, LastName);
```

[csharp_081] C# - Classes & Records

"C# record types": See also

Sumber: ht

See also

- Type system overview
- Classes
- Structs
- Records (C# reference)

[csharp_082] C# - Pattern Matching

"Pattern matching overview": Overview

Sumber: ht

Pattern matching overview

This article is part of the **Fundamentals** section for developers who already know at least one programming language and are learning C#. If you're new to programming, start with the Get started tutorials first. Experienced C# developers can use the patterns reference for the complete syntax and rules.

Coming from another language? C# patterns serve a role similar to Java's pattern matching and Python's `match` cases. C# uses patterns in `is` expressions, `switch` statements, and `switch` expressions.

Pattern matching applies a pattern to an expression. A *pattern* is a condition to test the input expression against. The *pattern input* is that expression. C# evaluates the input expression; the result is the *evaluated value*. The pattern tests whether that value has a particular type, equals a particular value, or has a particular shape. When a pattern matches, your code can use information learned by the test, such as a more specific type or a value extracted from an object.

You can use a pattern in three contexts:

- On the right side of the `is` operator.
- In a `case` label of a `switch` statement.
- In an arm of a `switch` expression.

Patterns test one evaluated input against types, constants, or a shape that can contain nested patterns. A Boolean condition can also compare two independently evaluated expressions whose values aren't constants. When either form can express the same test, choose the form that's easier to read.

For example, the following method uses a `switch` expression to choose a delivery message:

Property patterns can include an outer type test and nested patterns, but neither is required in every property pattern. In `StandardDelivery { Days: <= 2 }`, the outer pattern receives the `delivery` expression and tests its type. The `Days` property expression then becomes the input to the nested relational pattern `<= 2`.

The expression before `switch` is the input expression. Each line inside the braces is a *switch arm*. The pattern appears before `=>`, and the result appears after it. C# evaluates the input expression, then selects the first arm, in text order, whose pattern matches and whose optional `when` guard is `true`. The optional `when` guard is an additional Boolean condition written after the pattern. The preceding example showed the following patterns:

- `null` is a *constant pattern*. It tests whether the `delivery` expression evaluates to `null`.
- `ExpressDelivery express` is a *declaration pattern* with two parts. `ExpressDelivery` is the type-test part. It tests whether the evaluated value is a non-null object whose run-time type is compatible with `ExpressDelivery`. `express` is the *variable designation*: it declares a variable named `express` and assigns the matched `ExpressDelivery` object to it.
- `StandardDelivery { Days: <= 2 }` begins with a type test. `StandardDelivery` tests whether the evaluated value is a non-null object of that type. The braces contain a *property pattern*. `Days` names the property to inspect, so the `Days` property expression becomes the input to the recursive pattern. The `<= 2` portion is a *relational pattern*, which tests whether the evaluated value is less than or equal to `2`.
- `_` (underscore) is the *discard pattern*. It matches every evaluated value, including `null`. Because earlier arms already handle `null`, express deliveries, and standard deliveries arriving within two days, this final arm handles every remaining value.

An arm without a `when` guard is *unguarded*. All arms in the first example are unguarded. If an earlier unguarded arm matches every evaluated value that a later arm could match, the later arm is *subsumed*. A *subsumed* pattern is one where every possible input value that matches was already matched by an earlier switch arm. It can never match, so the compiler reports an error. The discard arm (`_`) must come last because it matches every input expression. A guarded arm doesn't subsume a later arm based on its pattern alone because the guard might be `false`.

A switch expression is *exhaustive* when its arms handle every possible input expression. The first example is exhaustive because its final discard arm handles anything the earlier arms don't match. The compiler warns when it detects a potential input value that no arm handles. The compiler can't prove exhaustiveness for every combination of patterns, but this diagnostic helps you write correct pattern matching code. For detailed matching, subsumption, and exhaustiveness rules, see the patterns reference.

Contoh Kode / Perintah CLI:

```
static string GetDeliveryMessage(Delivery? delivery) =>
        delivery switch
        {
            null => "No delivery was scheduled.",
            ExpressDelivery express => $"Express package {express.TrackingCode}",
```

```
            StandardDelivery { Days: <= 2 } => "Standard delivery arriving soon",
            _ => "Standard delivery"
        };
```

[csharp_083] C# - Pattern Matching

"Pattern matching overview": Test one condition with `is`

Sumber: ht

Test one condition with `is`

Use the `is` operator when you need one Boolean test. The following code evaluates the `delivery` expression and applies the declaration pattern `ExpressDelivery express`. The type portion matches when the evaluated value is non-null and its run-time type is compatible with `ExpressDelivery`. When the pattern matches, its variable designation declares `express`:

The compiler tracks whether a local variable has been assigned before your code reads it. This tracking is called *definite assignment*. Inside the `if` block, the condition can be `true` only when the pattern assigned the matching object to `express`. The compiler therefore knows that `express` is definitely assigned there. Your code can safely use its `TrackingCode` property. You don't need a separate cast. If you only need the type test and don't need to declare a variable, use a type pattern, such as `delivery is ExpressDelivery`.

Use `is null` or `is not null` for null checks. These patterns don't call a user-defined `==` or `!=` operator.

For more null-safety guidance, see nullable reference types.

Contoh Kode / Perintah CLI:

```
static void PrintTrackingCode(Delivery delivery)
    {
        if (delivery is ExpressDelivery express)
        {
            Console.WriteLine($"Track express package {express.TrackingCode}");
        }
    }

// ----------------------------------------

static void PrintPackageStatus(Delivery? delivery)
    {
        if (delivery is null)
        {
            Console.WriteLine("No package is available.");
        }
        else
        {
            Console.WriteLine("A package is ready to track.");
        }
    }
```

[csharp_084] C# - Pattern Matching

"Pattern matching overview": Choose between a statement and an expression

Sumber: ht

Choose between a statement and an expression

Patterns work with both forms of `switch`:

- Use a `switch` statement when each match should run one or more statements.
- Use a `switch` expression when each match should produce a value.

The following switch statement reports a delivery update. The express-delivery branch writes two messages, so a statement fits naturally:

Each `case` applies a pattern to the `delivery` expression. The matching section can run any number of statements before `break` exits the switch. The `default` section handles anything that the earlier cases don't match.

The following switch expression replaces an `if` / `else if` chain that assigns one result:

A switch expression is concise because every arm has the same purpose: produce the value returned. Use a switch statement when branches perform actions, and use a switch expression when branches calculate one result.

Contoh Kode / Perintah CLI:

```
static void PrintDeliveryUpdate(Delivery? delivery)
    {
        switch (delivery)
        {
            case null:
                Console.WriteLine("No delivery was scheduled.");
                break;
            case ExpressDelivery express:
                Console.WriteLine($"Express delivery {express.TrackingCode} is ready.");
                Console.WriteLine("Notify the priority desk.");
                break;
            case StandardDelivery standard:
                Console.WriteLine($"Standard delivery arrives in {standard.Days} days.");
                break;
            default:
                Console.WriteLine("Another delivery type is scheduled.");
                break;
        }
    }

// ----------------------------------------

static string GetStatusMessage(StandardDelivery delivery) =>
        delivery.Days switch
        {
            0 => "Delivered today",
            1 => "Arriving tomorrow",
            <= 3 => "Arriving soon",
            _ => "In transit"
        };
```

[csharp_085] C# - Pattern Matching

"Pattern matching overview": Pattern categories

Sumber: ht

Pattern categories

C# includes patterns for common kinds of data tests:

Pattern category	What it tests
Declaration, constant, and `var` patterns	A run-time type, a specific constant value, or any value that you want to capture
Type patterns	A run-time type without declaring a variable
Property and positional patterns	Properties, fields, or deconstructed values
Relational, logical, and parenthesized patterns	Comparisons and combinations such as `and`, `or`, and `not`
List and slice patterns	The values and shape of a supported sequence
Discard patterns and discards	Any remaining value, or a value your code intentionally ignores

The linked Fundamentals articles explain when to choose each category. For complete syntax and examples, see the patterns reference.

[csharp_086] C# - Pattern Matching

"Pattern matching overview": See also

Sumber: ht

See also

- Declaration, constant, and `var` patterns
- Type patterns
- Property and positional patterns
- Relational, logical, and parenthesized patterns
- List and slice patterns
- Discards
- Patterns reference
- `switch` expression reference

[csharp_087] C# - Pattern Matching

"List and slice patterns": Overview

Sumber: ht

List and slice patterns

This article is part of the **Fundamentals** section for developers who already know at least one programming language and are learning C#. Start with the pattern matching overview if patterns are new to you. For complete supported-type and language rules, see list patterns in the language reference.

A sequence's *shape* is the set of observable properties that a pattern requires. These properties can include the number of elements, requirements written for every element or for a sequence of elements as a whole, and marker values at specific indexes. For example, a shape might require exactly two elements with `"Name"` at index 0, or it might require a first marker, a last marker, and a nonempty sequence between them.

A *list pattern* describes a shape by combining an element-count requirement with nested patterns at ordered positions. It tests only the requirements written in the pattern: An element subpattern tests its corresponding element, while a *slice pattern* can allow or test a sequence of otherwise unmatched elements. You can use these parts to match an

exact count and marker values, capture values at fixed positions, allow an unmatched middle, or apply a nested pattern to that middle.

Choose a list pattern when the input's compile-time type provides a `Length` or `Count` property and indexed element access, which retrieves an element by its position, as in `input[index]`. Arrays, `List<T>`, strings, and spans are common examples. The compiler determines eligibility from the variable's declared, compile-time type because it must resolve these required members before the program runs. The <xref:System.Collections.Generic.IEnumerable%601> interface by itself provides enumeration instead of those members, even when the runtime object is an indexable collection.

List patterns use a length or count and indexed element access instead of enumeration. For common built-in types, matching can check only the positions named by the pattern rather than iterate through every element. The cost of those operations depends on the input type. Iterating a long collection can take time, so use a loop or LINQ when the decision requires examining an arbitrary number of elements rather than specific positions.

[csharp_088] C# - Pattern Matching

"List and slice patterns": Match an exact shape

Sumber: ht

Match an exact shape

The following method recognizes a two-column header:

The `columns` expression is the pattern input. `["Name", "Score"]` contains two constant patterns. An exact list pattern omits a slice pattern, requires exactly two elements, and applies each nested pattern to the element at the same index. This pattern describes a shape with an exact count and two marker values: `"Name"` at index 0 and `"Score"` at index 1. Nested element patterns are optional: The empty list pattern `[]` describes, and matches, a sequence with no elements.

Choose a list pattern when the shape combines an element count with requirements at ordered positions. If only the number of elements matters, a `Length` or `Count` property pattern, such as `items is { Count: 0 }`, states that intent more directly.

Contoh Kode / Perintah CLI:

```
static bool IsHeader(string[] columns) =>
        columns is ["Name", "Score"];
```

[csharp_089] C# - Pattern Matching

"List and slice patterns": Use a discard to match any element value

Sumber: ht

Use a discard to match any element value

The following method reads the winner and third-place finisher from a three-name finishing order:

The `var winner` and `var thirdPlace` patterns each declare a variable. When the list pattern matches, C# assigns the matched element value at that position to the corresponding variable, so the result can use `winner` and `thirdPlace`. The discard pattern `_` accepts the second element and discards its value. This shape requires exactly three elements and gives the first and third positions specific meaning, but it doesn't require particular values at any position.

Choose this form when fixed positions have stable meaning. Use a loop or LINQ when you need to inspect an arbitrary number of elements, transform a sequence, search throughout it, or perform aggregation.

Contoh Kode / Perintah CLI:

```
static string GetAnnouncements(List<string> finishingOrder) =>
        finishingOrder switch
        {
            [var winner, _, var thirdPlace] =>
                $"Winner: {winner}; third place: {thirdPlace}",
            _ => "A complete three-runner result isn't available"
        };
```

[csharp_090] C# - Pattern Matching

"List and slice patterns": Allow remaining elements with a slice pattern

Sumber: ht

Allow remaining elements with a slice pattern

In this simplified example, a command line can start with `--verbose` and contain other arguments, and is assumed to end with the input file name. The following method recognizes shapes with a file name in the last position and captures that value:

The *slice pattern* `..` allows zero or more remaining elements between the first and last element patterns. A list pattern can contain at most one slice pattern. The first switch arm describes a shape with at least two elements: the marker `"--verbose"` at index 0, a file name in the last position, and any number of unmatched arguments between them. The standalone slice doesn't test those middle arguments. The second arm accepts any nonempty shape and captures its last element, while the empty pattern handles a sequence with no elements.

A slice can appear at the beginning, middle, or end of a list pattern. Use it when a variable-length sequence is part of the shape but only the surrounding positions need element tests. Use ordinary iteration when every element needs processing.

Contoh Kode / Perintah CLI:

```
static string GetInputFile(string[] arguments) =>
        arguments switch
        {
            ["--verbose", .., var fileName] => $"Verbose processing: {fileName}",
            [.., var fileName] => $"Processing: {fileName}",
            [] => "No input file was provided"
        };
```

[csharp_091] C# - Pattern Matching

"List and slice patterns": Apply a pattern to a slice

Sumber: ht

Apply a pattern to a slice

You can apply another pattern to the part matched by `..`. The following method tests whether an array starts with `"BEGIN"`, ends with `"END"`, and has at least one element between them:

The outer pattern first requires the marker `"BEGIN"` at index 0 and `"END"` at the last index. The property pattern `{

Length: > 0 }` then adds a requirement for the sequence matched by the slice: It must contain at least one element. Together, these requirements describe the complete shape. The pattern doesn't test the values of the elements inside the slice.

Use a slice subpattern only when the middle portion itself needs a test or capture. If only boundary elements matter, plain `..` is simpler.

Contoh Kode / Perintah CLI:

```
static bool HasContent(string[] entries) =>
        entries is ["BEGIN", .. { Length: > 0 }, "END"];
```

[csharp_092] C# - Pattern Matching

"List and slice patterns": See also

Sumber: ht

See also

- Pattern matching overview
- Property and positional patterns
- List pattern reference
- Arrays
- Use a `foreach` statement to iterate through a collection

[csharp_093] C# - Object-Oriented

"Classes, structs, and records": Overview

Sumber: ht

Overview of object oriented techniques in C\#

In C#, the definition of a type—a class, struct, or record—is like a blueprint that specifies what the type can do. An object is basically a block of memory allocated and configured according to the blueprint. This article provides an overview of these blueprints and their features. The next article in this series introduces objects.

[csharp_094] C# - Object-Oriented

"Classes, structs, and records": Encapsulation

Sumber: ht

Encapsulation

Encapsulation is sometimes referred to as the first pillar or principle of object-oriented programming. A class or struct can specify how accessible each of its members is to code outside of the class or struct. Members not intended for consumers outside of the class or assembly are hidden to limit the potential for coding errors or malicious exploits. For more information, see the Object-oriented programming tutorial.

[csharp_095] C# - Object-Oriented

"Classes, structs, and records": Members

Sumber: ht

Members

The *members* of a type include all methods, fields, constants, properties, and events. In C#, there are no global variables or methods as there are in some other languages. Even a program's entry point, the `Main` method, must be declared within a class or struct (implicitly when you use top-level statements).

The following list includes all the various kinds of members that can be declared in a class, struct, or record.

- Fields
- Constants
- Properties
- Methods
- Constructors
- Events
- Finalizers
- Indexers
- Operators
- Nested Types

For more information, see Members.

[csharp_096] C# - Object-Oriented

"Classes, structs, and records": Accessibility

Sumber: ht

Accessibility

Some methods and properties are meant to be called or accessed from code outside a class or struct, known as *client code*. Other methods and properties might be only for use in the class or struct itself. It's important to limit the accessibility of your code so that only the intended client code can reach it. You specify how accessible your types and their members are to client code by using the following access modifiers:

- public
- protected
- internal
- protected internal
- private
- private protected.

The default accessibility is `private`.

[csharp_097] C# - Object-Oriented

"Classes, structs, and records": Inheritance

Sumber: ht

Inheritance

Classes (but not structs) support the concept of inheritance. A class that derives from another class, called the *base class*, automatically contains all the public, protected, and internal members of the base class except its constructors and finalizers.

Classes can be declared as abstract, which means that one or more of their methods have no implementation. Although abstract classes can't be instantiated directly, they can serve as base classes for other classes that provide the missing implementation. Classes can also be declared as sealed to prevent other classes from inheriting from them.

For more information, see Inheritance and Polymorphism.

[csharp_098] C# - Object-Oriented

"Classes, structs, and records": Interfaces

Sumber: ht

Interfaces

Classes, structs, and records can implement multiple interfaces. To implement from an interface means that the type implements all the methods defined in the interface. For more information, see Interfaces.

[csharp_099] C# - Object-Oriented

"Classes, structs, and records": Generic Types

Sumber: ht

Generic Types

Classes, structs, and records can be defined with one or more type parameters. Client code supplies the type when it creates an instance of the type. For example, the <xref:System.Collections.Generic.List`1> class in the <xref:System.Collections.Generic> namespace is defined with one type parameter. Client code creates an instance of a `List<string>` or `List<int>` to specify the type that the list holds. For more information, see Generics.

[csharp_100] C# - Object-Oriented

"Classes, structs, and records": Static Types

Sumber: ht

Static Types

Classes (but not structs or records) can be declared as `static`. A static class can contain only static members and can't be instantiated with the `new` keyword. One copy of the class is loaded into memory when the program loads, and its members are accessed through the class name. Classes, structs, and records can contain static members. For more information, see Static classes and static class members.

[csharp_101] C# - Object-Oriented

"Classes, structs, and records": Nested Types

Sumber: ht

Nested Types

A class, struct, or record can be nested within another class, struct, or record. For more information, see Nested Types.

[csharp_102] C# - Object-Oriented

"Classes, structs, and records": Partial Types

Sumber: ht

Partial Types

You can define part of a class, struct, or method in one code file and another part in a separate code file. For more information, see Partial Classes and Methods.

[csharp_103] C# - Object-Oriented

"Classes, structs, and records": Object Initializers

Sumber: ht

Object Initializers

You can instantiate and initialize class or struct objects, and collections of objects, by assigning values to its properties. For more information, see How to initialize objects by using an object initializer.

[csharp_104] C# - Object-Oriented

"Classes, structs, and records": Anonymous Types

Sumber: ht

Anonymous Types

In situations where it isn't convenient or necessary to create a named class, you can use anonymous types or tuples. Named data members define anonymous types. Tuples are the preferred choice for new code because they offer better performance and deconstruction support. For more information, see Tuples and deconstruction.

[csharp_105] C# - Object-Oriented

"Classes, structs, and records": Extension Members

Sumber: ht

Extension Members

You can "extend" a class without creating a derived class by creating a separate type. That type contains methods that can be called as if they belonged to the original type. For more information, see Extension methods.

[csharp_106] C# - Object-Oriented

"Classes, structs, and records": Implicitly Typed Local Variables

Sumber: ht

Implicitly Typed Local Variables

Within a class or struct method, you can use implicit typing to instruct the compiler to determine a variable's type at compile time. For more information, see var (C# reference).

[csharp_107] C# - Object-Oriented

"Classes, structs, and records": Records

Sumber: ht

Records

You can add the `record` modifier to a class or a struct. Records are types with built-in behavior for value-based equality. A record (either `record class` or `record struct`) provides the following features:

* Concise syntax for creating a reference type with immutable properties.
* Value equality.
  Two variables of a record type are equal if they have the same type, and if, for every field, the values in both records are equal. Classes use reference equality: two variables of a class type are equal if they refer to the same object.
* Concise syntax for nondestructive mutation.
  A `with` expression lets you create a new record instance that is a copy of an existing instance but with specified property values changed.
* Built-in formatting for display.

The `ToString` method prints the record type name and the names and values of public properties.
* Support for inheritance hierarchies in record classes.
Record classes support inheritance. Record structs don't support inheritance.

For more information, see Records.

[csharp_108] C# - Object-Oriented

"Classes, structs, and records": C# Language Specification

Sumber: ht

C# Language Specification

!INCLUDE[CSharplangspec]

[csharp_109] C# - Exception Handling

"Exceptions and Exception Handling": Overview

Sumber: ht

Exceptions and Exception Handling

The C# language's exception handling features help you deal with any unexpected or exceptional situations that occur when a program is running. Exception handling uses the `try`, `catch`, and `finally` keywords to try actions that may not succeed, to handle failures when you decide that it's reasonable to do so, and to clean up resources afterward. Exceptions can be generated by the common language runtime (CLR), by .NET or third-party libraries, or by application code. Exceptions are created by using the `throw` keyword.

In many cases, an exception may be thrown not by a method that your code has called directly, but by another method further down in the call stack. When an exception is thrown, the CLR will unwind the stack, looking for a method with a `catch` block for the specific exception type, and it will execute the first such `catch` block that it finds. If it finds no appropriate `catch` block anywhere in the call stack, it will terminate the process and display a message to the user.

In this example, a method tests for division by zero and catches the error. Without the exception handling, this program would terminate with a **DivideByZeroException was unhandled** error.

Contoh Kode / Perintah CLI:

```
public class ExceptionTest
    {
        static double SafeDivision(double x, double y)
        {
            if (y == 0)
                throw new DivideByZeroException();
            return x / y;
        }

        public static void Main()
        {
            // Input for test purposes. Change the values to see
            // exception handling behavior.
            double a = 98, b = 0;
            double result;

            try
            {
                result = SafeDivision(a, b);
                Console.WriteLine($"{a} divided by {b} = {result}");
            }
            catch (DivideByZeroException)
```

```
            {
                Console.WriteLine("Attempted divide by zero.");
            }
        }
    }
```

[csharp_110] C# - Exception Handling

"Exceptions and Exception Handling": Exceptions Overview

Sumber: ht

Exceptions Overview

Exceptions have the following properties:

- Exceptions are types that all ultimately derive from `System.Exception`.
- Use a `try` block around the statements that might throw exceptions.
- Once an exception occurs in the `try` block, the flow of control jumps to the first associated exception handler that is present anywhere in the call stack. In C#, the `catch` keyword is used to define an exception handler.
- If no exception handler for a given exception is present, the program stops executing with an error message.
- Don't catch an exception unless you can handle it and leave the application in a known state. If you catch `System.Exception`, rethrow it using the `throw` keyword at the end of the `catch` block.
- If a `catch` block defines an exception variable, you can use it to obtain more information about the type of exception that occurred.
- Exceptions can be explicitly generated by a program by using the `throw` keyword.
- Exception objects contain detailed information about the error, such as the state of the call stack and a text description of the error.
- Code in a `finally` block is executed regardless of if an exception is thrown. Use a `finally` block to release resources, for example to close any streams or files that were opened in the `try` block.
- Managed exceptions in .NET are implemented on top of the Win32 structured exception handling mechanism. For more information, see Structured Exception Handling (C/C++) and A Crash Course on the Depths of Win32 Structured Exception Handling.

[csharp_111] C# - Exception Handling

"Exceptions and Exception Handling": C# Language Specification

Sumber: ht

C# Language Specification

For more information, see Exceptions in the C# Language Specification. The language specification is the definitive source for C# syntax and usage.

[csharp_112] C# - Exception Handling

"Exceptions and Exception Handling": See also

Sumber: ht

See also

- <xref:System.Exception?displayProperty=nameWithType>
- Exception-handling statements
- Exceptions

[csharp_113] C# - Asynchronous Programming

Async return types: Overview

Sumber: ht

Async return types (C#)

Async methods can have the following return types:

- <xref:System.Threading.Tasks.Task>, for an async method that performs an operation but returns no value.
- <xref:System.Threading.Tasks.Task`1>, for an async method that returns a value.
- `void`, for an event handler.
- Any type that has an accessible `GetAwaiter` method. The object returned by the `GetAwaiter` method must implement the <xref:System.Runtime.CompilerServices.ICriticalNotifyCompletion?displayProperty=nameWithType> interface.
- <xref:System.Collections.Generic.IAsyncEnumerable`1>, for an async method that returns an *async stream*.

For more information about async methods, see Asynchronous programming with async and await (C#).

Several other types also exist that are specific to Windows workloads:

- <xref:System.Windows.Threading.DispatcherOperation>, for async operations limited to Windows.
- <xref:Windows.Foundation.IAsyncAction>, for async actions in Universal Windows Platform (UWP) apps that don't return a value.
- <xref:Windows.Foundation.IAsyncActionWithProgress`1>, for async actions in UWP apps that report progress but don't return a value.
- <xref:Windows.Foundation.IAsyncOperation`1>, for async operations in UWP apps that return a value.
- <xref:Windows.Foundation.IAsyncOperationWithProgress`2>, for async operations in UWP apps that report progress and return a value.

[csharp_114] C# - Asynchronous Programming

Async return types: Task return type

Sumber: ht

Task return type

Async methods that don't contain a `return` statement or that contain a `return` statement that doesn't return an operand usually have a return type of <xref:System.Threading.Tasks.Task>. Such methods return `void` if they run synchronously. If you use a <xref:System.Threading.Tasks.Task> return type for an async method, a calling method can use an `await` operator to suspend the caller's completion until the called async method finishes.

In the following example, the `WaitAndApologizeAsync` method doesn't contain a `return` statement, so the method returns a <xref:System.Threading.Tasks.Task> object. Returning a `Task` enables `WaitAndApologizeAsync` to be awaited. The <xref:System.Threading.Tasks.Task> type doesn't include a `Result` property because it has no return value.

`WaitAndApologizeAsync` is awaited by using an await statement instead of an await expression, similar to the calling statement for a synchronous void-returning method. The application of an await operator in this case doesn't produce a value. When the right operand of an `await` is a <xref:System.Threading.Tasks.Task`1>, the `await` expression produces a result of `T`. When the right operand of an `await` is a <xref:System.Threading.Tasks.Task>, the `await` and its operand are a statement.

You can separate the call to `WaitAndApologizeAsync` from the application of an await operator, as the following code shows. However, remember that a `Task` doesn't have a `Result` property, and that no value is produced when an await operator is applied to a `Task`.

The following code separates calling the `WaitAndApologizeAsync` method from awaiting the task that the method returns.

Contoh Kode / Perintah CLI:

```
public static async Task DisplayCurrentInfoAsync()
    {
        await WaitAndApologizeAsync();

        Console.WriteLine($"Today is {DateTime.Now:D}");
        Console.WriteLine($"The current time is {DateTime.Now.TimeOfDay:t}");
        Console.WriteLine("The current temperature is 76 degrees.");
    }

    static async Task WaitAndApologizeAsync()
    {
        await Task.Delay(2000);

        Console.WriteLine("Sorry for the delay...\n");
    }
    // Example output:
    //    Sorry for the delay...
    //
    // Today is Monday, August 17, 2020
    // The current time is 12:59:24.2183304
    // The current temperature is 76 degrees.

// ------------------------------------------

Task waitAndApologizeTask = WaitAndApologizeAsync();

        string output =
            $"Today is {DateTime.Now:D}\n" +
            $"The current time is {DateTime.Now.TimeOfDay:t}\n" +
            "The current temperature is 76 degrees.\n";

        await waitAndApologizeTask;
        Console.WriteLine(output);
```

[csharp_115] C# - Asynchronous Programming

Async return types: Task\<TResult\> return type

Sumber: ht

Task\<TResult\> return type

The <xref:System.Threading.Tasks.Task`1> return type is used for an async method that contains a return statement in which the operand is `TResult`.

In the following example, the `GetLeisureHoursAsync` method contains a `return` statement that returns an integer. The method declaration must specify a return type of `Task<int>`. The <xref:System.Threading.Tasks.Task.FromResult*> async method is a placeholder for an operation that returns a <xref:System.DateTime.DayOfWeek>.

When `GetLeisureHoursAsync` is called from within an await expression in the `ShowTodaysInfoAsync` method, the

await expression retrieves the integer value (the value of `leisureHours`) stored in the task returned by the `GetLeisureHoursAsync` method. For more information about await expressions, see await.

You can better understand how `await` retrieves the result from a `Task<T>` by separating the call to `GetLeisureHoursAsync` from the application of `await`, as the following code shows. A call to method `GetLeisureHoursAsync` that isn't immediately awaited returns a `Task<int>`, as you would expect from the declaration of the method. The task is assigned to the `getLeisureHoursTask` variable in the example. Because `getLeisureHoursTask` is a <xref:System.Threading.Tasks.Task`1>, it contains a <xref:System.Threading.Tasks.Task`1.Result> property of type `TResult`. In this case, `TResult` represents an integer type. When `await` is applied to `getLeisureHoursTask`, the await expression evaluates to the contents of the <xref:System.Threading.Tasks.Task`1.Result> property of `getLeisureHoursTask`. The value is assigned to the `ret` variable.
The <xref:System.Threading.Tasks.Task`1.Result> property is a blocking property. If you try to access it before its task is finished, the thread that's currently active is blocked until the task completes and the value is available. In most cases, you should access the value by using `await` instead of accessing the property directly.
The previous example retrieved the value of the <xref:System.Threading.Tasks.Task`1.Result> property to block the main thread so that the `Main` method could print the `message` to the console before the application ended.

Contoh Kode / Perintah CLI:

```
public static async Task ShowTodaysInfoAsync()
    {
        string message =
            $"Today is {DateTime.Today:D}\n" +
            "Today's hours of leisure: " +
            $"{await GetLeisureHoursAsync()}";

        Console.WriteLine(message);
    }

    static async Task<int> GetLeisureHoursAsync()
    {
        DayOfWeek today = await Task.FromResult(DateTime.Now.DayOfWeek);

        int leisureHours =
            today is DayOfWeek.Saturday || today is DayOfWeek.Sunday
            ? 16 : 5;

        return leisureHours;
    }
    // Example output:
    //    Today is Wednesday, May 24, 2017
    //    Today's hours of leisure: 5

// ----------------------------------------

var getLeisureHoursTask = GetLeisureHoursAsync();

        string message =
            $"Today is {DateTime.Today:D}\n" +
            "Today's hours of leisure: " +
            $"{await getLeisureHoursTask}";

        Console.WriteLine(message);
```

[csharp_116] C# - Asynchronous Programming

Async return types: Void return type

Sumber: ht

Void return type

You use the `void` return type in asynchronous event handlers, which require a `void` return type. For methods other than event handlers that don't return a value, you should return a <xref:System.Threading.Tasks.Task> instead, because an async method that returns `void` can't be awaited. Any caller of such a method must continue to completion without waiting for the called async method to finish. The caller must be independent of any values or exceptions that the async method generates.

The caller of a void-returning async method can't catch exceptions thrown from the method. Such unhandled exceptions are likely to cause your application to fail. If a method that returns a <xref:System.Threading.Tasks.Task> or <xref:System.Threading.Tasks.Task`1> throws an exception, the exception is stored in the returned task. The exception is rethrown when the task is awaited. Make sure that any async method that can produce an exception has a return type of <xref:System.Threading.Tasks.Task> or <xref:System.Threading.Tasks.Task`1> and that calls to the method are awaited.

The following example shows the behavior of an async event handler. In the example code, an async event handler must let the main thread know when it finishes. Then the main thread can wait for an async event handler to complete before exiting the program.

[csharp_117] C# - Asynchronous Programming

Async return types: Generalized async return types and ValueTask\<TResult\>

Sumber: ht

Generalized async return types and ValueTask\<TResult\>

An async method can return any type that has an accessible `GetAwaiter` method that returns an instance of an *awaiter type*. In addition, the returned type must match the type of the parameter of `SetResult` and returned type of the `Task` property on the type specified by the <xref:System.Runtime.CompilerServices.AsyncMethodBuilderAttribute?displayProperty=nameWithType> attribute. You can learn more in the article on Attributes read by the compiler or the C# spec for the Task type builder pattern.

This feature is the complement to awaitable expressions, which describes the requirements for the operand of `await`. Generalized async return types enable the compiler to generate `async` methods that return different types. Generalized async return types enabled performance improvements in the .NET libraries. Because <xref:System.Threading.Tasks.Task> and <xref:System.Threading.Tasks.Task`1> are reference types, memory allocation in performance-critical paths, particularly when allocations occur in tight loops, can adversely affect performance. Support for generalized return types means that you can return a lightweight value type instead of a reference type to avoid more memory allocations.

.NET provides the <xref:System.Threading.Tasks.ValueTask`1?displayProperty=nameWithType> structure as a lightweight implementation of a generalized task-returning value. The following example uses the <xref:System.Threading.Tasks.ValueTask`1> structure to retrieve the value of two dice rolls.

Writing a generalized async return type is an advanced scenario, and is targeted for use in specialized environments. Consider using the `Task`, `Task<T>`, and `ValueTask<T>` types instead, which cover most scenarios for asynchronous code.

You can apply the `AsyncMethodBuilder` attribute to an async method (instead of the async return type declaration) to override the builder for that type. Typically you'd apply this attribute to use a different builder provided in the .NET runtime.

Contoh Kode / Perintah CLI:

```
class Program
{
    static readonly Random s_rnd = new Random();

    static async Task Main() =>
        Console.WriteLine($"You rolled {await GetDiceRollAsync()}");

    static async ValueTask<int> GetDiceRollAsync()
    {
        Console.WriteLine("Shaking dice...");

        int roll1 = await RollAsync();
        int roll2 = await RollAsync();

        return roll1 + roll2;
    }

    static async ValueTask<int> RollAsync()
    {
        await Task.Delay(500);

        int diceRoll = s_rnd.Next(1, 7);
        return diceRoll;
    }
}
// Example output:
//    Shaking dice...
//    You rolled 8
```

[csharp_118] C# - Asynchronous Programming

Async return types: Async streams with IAsyncEnumerable\<T\>

Sumber: ht

Async streams with IAsyncEnumerable\<T\>

An async method might return an *async stream*, represented by <xref:System.Collections.Generic.IAsyncEnumerable`1>. An async stream provides a way to enumerate items read from a stream when elements are generated in chunks with repeated asynchronous calls. The following example shows an async method that generates an async stream:

The preceding example reads lines from a string asynchronously. Once each line is read, the code enumerates each word in the string. Callers would enumerate each word using the `await foreach` statement. The method awaits when it needs to asynchronously read the next line from the source string.

Contoh Kode / Perintah CLI:

```
static async IAsyncEnumerable<string> ReadWordsFromStreamAsync()
    {
        string data =
            @"This is a line of text.
                  Here is the second line of text.
                  And there is one more for good measure.
                  Wait, that was the penultimate line.";
```

```
        using var readStream = new StringReader(data);

        string? line = await readStream.ReadLineAsync();
        while (line != null)
        {
            foreach (string word in line.Split(' ', StringSplitOptions.RemoveEmptyEntries))
            {
                yield return word;
            }

            line = await readStream.ReadLineAsync();
        }
    }
```

[csharp_119] C# - Asynchronous Programming

Async return types: See also

Sumber: ht

See also

- <xref:System.Threading.Tasks.Task.FromResult*>
- Process asynchronous tasks as they complete
- Asynchronous programming with async and await (C#)
- async
- await

[csharp_120] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": Overview

Sumber: ht

Task asynchronous programming model

You can avoid performance bottlenecks and enhance the overall responsiveness of your application by using asynchronous programming. However, traditional techniques for writing asynchronous applications can be complicated, making them difficult to write, debug, and maintain.

C# supports simplified approach, async programming, that uses asynchronous support in the .NET runtime. The compiler does the difficult work that the developer used to do, and your application retains a logical structure that resembles synchronous code. As a result, you get all the advantages of asynchronous programming with a fraction of the effort.

This article provides an overview of when and how to use async programming and includes links to other articles that contain details and examples.

[csharp_121] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": Async improves responsiveness

Sumber: ht

Async improves responsiveness

Asynchrony is essential for activities that are potentially blocking, such as web access. Access to a web resource sometimes is slow or delayed. If such an activity is blocked in a synchronous process, the entire application must wait. In an asynchronous process, the application can continue with other work that doesn't depend on the web resource until

the potentially blocking task finishes.

The following table shows typical areas where asynchronous programming improves responsiveness. The listed APIs from .NET and the Windows Runtime contain methods that support async programming.

Application area	.NET types with async methods	Windows Runtime types with async methods
Web access	<xref:System.Net.Http.HttpClient>	<xref:Windows.Web.Http.HttpClient?displayProperty=nameWithType> <xref:Windows.Web.Syndication.SyndicationClient>
Working with files	<xref:System.Text.Json.JsonSerializer> <xref:System.IO.StreamReader> <xref:System.IO.StreamWriter> <xref:System.Xml.XmlReader> <xref:System.Xml.XmlWriter>	<xref:Windows.Storage.StorageFile>
Working with images		<xref:Windows.Media.Capture.MediaCapture> <xref:Windows.Graphics.Imaging.BitmapEncoder> <xref:Windows.Graphics.Imaging.BitmapDecoder>
WCF programming	Synchronous and Asynchronous Operations	

Asynchrony proves especially valuable for applications that access the UI thread because all UI-related activity usually shares one thread. If any process is blocked in a synchronous application, all are blocked. Your application stops responding, and you might conclude that it failed when instead it's just waiting.

When you use asynchronous methods, the application continues to respond to the UI. You can resize or minimize a window, for example, or you can close the application if you don't want to wait for it to finish.

The async-based approach adds the equivalent of an automatic transmission to the list of options that you can choose from when designing asynchronous operations. That is, you get all the benefits of traditional asynchronous programming but with much less effort from the developer.

[csharp_122] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": Async methods are easy to write

Sumber: ht

Async methods are easy to write

The async and await keywords in C# are the heart of async programming. By using those two keywords, you can use resources in .NET Framework, .NET Core, or the Windows Runtime to create an asynchronous method almost as easily as you create a synchronous method. Asynchronous methods that you define by using the `async` keyword are referred to as *async methods*.

The following example shows an async method. Almost everything in the code should look familiar to you.

You can find a complete Windows Presentation Foundation (WPF) example available for download from Asynchronous programming with async and await in C#.

You can learn several practices from the preceding sample. Start with the method signature. It includes the `async` modifier. The return type is `Task<int>` (See "Return Types" section for more options). The method name ends in `Async`. In the body of the method, `GetStringAsync` returns a `Task<string>`. That means that when you `await` the task you get a `string` (`contents`). Before awaiting the task, you can do work that doesn't rely on the `string` from `GetStringAsync`.

Pay close attention to the `await` operator. It suspends `GetUrlContentLengthAsync`:

- `GetUrlContentLengthAsync` can't continue until `getStringTask` is complete.
- Meanwhile, control returns to the caller of `GetUrlContentLengthAsync`.
- Control resumes here when `getStringTask` is complete.
- The `await` operator then retrieves the `string` result from `getStringTask`.

The return statement specifies an integer result. Any methods that are awaiting `GetUrlContentLengthAsync` retrieve the length value.

If `GetUrlContentLengthAsync` doesn't have any work that it can do between calling `GetStringAsync` and awaiting its completion, you can simplify your code by calling and awaiting in the following single statement.

The following characteristics summarize what makes the previous example an async method:

- The method signature includes an `async` modifier.
- The name of an async method, by convention, ends with an "Async" suffix.
- The return type is one of the following types:

  - <xref:System.Threading.Tasks.Task`1> if your method has a return statement in which the operand has type `TResult`.
  - <xref:System.Threading.Tasks.Task> if your method has no return statement or has a return statement with no operand.
  - `void` if you're writing an async event handler.
  - Any other type that has a `GetAwaiter` method.

  For more information, see the Return types and parameters section.

- The method usually includes at least one `await` expression, which marks a point where the method can't continue until the awaited asynchronous operation is complete. In the meantime, the method is suspended, and control returns to the method's caller. The next section of this article illustrates what happens at the suspension point.

In async methods, you use the provided keywords and types to indicate what you want to do, and the compiler does the rest, including keeping track of what must happen when control returns to an await point in a suspended method. Some routine processes, such as loops and exception handling, can be difficult to handle in traditional asynchronous code. In an async method, you write these elements much as you would in a synchronous solution, and the problem is solved.

For more information about asynchrony in previous versions of .NET Framework, see TPL and traditional .NET Framework asynchronous programming.

Contoh Kode / Perintah CLI:

```
string contents = await client.GetStringAsync("https://learn.microsoft.com/dotnet");

// ----------------------------------------

public async Task<int> GetUrlContentLengthAsync()
    {
        using var client = new HttpClient();

        Task<string> getStringTask =
            client.GetStringAsync("https://learn.microsoft.com/dotnet");
```

```
        DoIndependentWork();

        string contents = await getStringTask;

        return contents.Length;
    }

    void DoIndependentWork()
    {
        Console.WriteLine("Working...");
    }
```

[csharp_123] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": What happens in an async method

Sumber: ht

What happens in an async method

The most important thing to understand in asynchronous programming is how the control flow moves from method to method. The following diagram leads you through the process:

:::image type="content" source="media/task-asynchronous-programming-model/navigation-trace-async-program.png" alt-text="Trace navigation of async control flow" lightbox="media/task-asynchronous-programming-model/navigation-trace-async-program.png":::

The numbers in the diagram correspond to the following steps, initiated when a calling method calls the async method.

1. A calling method calls and awaits the `GetUrlContentLengthAsync` async method.

1. `GetUrlContentLengthAsync` creates an <xref:System.Net.Http.HttpClient> instance and calls the <xref:System.Net.Http.HttpClient.GetStringAsync*> asynchronous method to download the contents of a website as a string.

1. Something happens in `GetStringAsync` that suspends its progress. Perhaps it must wait for a website to download or some other blocking activity. To avoid blocking resources, `GetStringAsync` yields control to its caller, `GetUrlContentLengthAsync`.

    `GetStringAsync` returns a <xref:System.Threading.Tasks.Task`1>, where `TResult` is a string, and `GetUrlContentLengthAsync` assigns the task to the `getStringTask` variable. The task represents the ongoing process for the call to `GetStringAsync`, with a commitment to produce an actual string value when the work is complete.

1. Because `getStringTask` isn't awaited yet, `GetUrlContentLengthAsync` can continue with other work that doesn't depend on the final result from `GetStringAsync`. That work is represented by a call to the synchronous method `DoIndependentWork`.

1. `DoIndependentWork` is a synchronous method that does its work and returns to its caller.

1. `GetUrlContentLengthAsync` runs out of work that it can do without a result from `getStringTask`. `GetUrlContentLengthAsync` next wants to calculate and return the length of the downloaded string, but the method can't calculate that value until the method has the string.

    Therefore, `GetUrlContentLengthAsync` uses an await operator to suspend its progress and to yield control to the

method that called `GetUrlContentLengthAsync`. `GetUrlContentLengthAsync` returns a `Task<int>` to the caller. The task represents a promise to produce an integer result that's the length of the downloaded string.
If `GetStringAsync` (and therefore `getStringTask`) completes before `GetUrlContentLengthAsync` awaits it, control remains in `GetUrlContentLengthAsync`. The expense of suspending and then returning to `GetUrlContentLengthAsync` would be wasted if the called asynchronous process `getStringTask` is complete and `GetUrlContentLengthAsync` doesn't have to wait for the final result.

Inside the calling method the processing pattern continues. The caller might do other work that doesn't depend on the result from `GetUrlContentLengthAsync` before awaiting that result, or the caller might await immediately. The calling method is waiting for `GetUrlContentLengthAsync`, and `GetUrlContentLengthAsync` is waiting for `GetStringAsync`.

1. `GetStringAsync` completes and produces a string result. The string result isn't returned by the call to `GetStringAsync` in the way that you might expect. (Remember that the method already returned a task in step 3.) Instead, the string result is stored in the task that represents the completion of the method, `getStringTask`. The await operator retrieves the result from `getStringTask`. The assignment statement assigns the retrieved result to `contents`.

1. When `GetUrlContentLengthAsync` has the string result, the method can calculate the length of the string. Then the work of `GetUrlContentLengthAsync` is also complete, and the waiting event handler can resume. In the full example at the end of the article, you can confirm that the event handler retrieves and prints the value of the length result.
If you're new to asynchronous programming, take a minute to consider the difference between synchronous and asynchronous behavior. A synchronous method returns when its work is complete (step 5), but an async method returns a task value when its work is suspended (steps 3 and 6). When the async method eventually completes its work, the task is marked as completed and the result, if any, is stored in the task.

[csharp_124] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": API async methods

Sumber: ht

API async methods

You might be wondering where to find methods such as `GetStringAsync` that support async programming. .NET Framework 4.5 or higher and .NET Core contain many members that work with `async` and `await`. You can recognize them by the "Async" suffix appended to the member name, and by their return type of <xref:System.Threading.Tasks.Task> or <xref:System.Threading.Tasks.Task`1>. For example, the `System.IO.Stream` class contains methods such as <xref:System.IO.Stream.CopyToAsync*>, <xref:System.IO.Stream.ReadAsync*>, and <xref:System.IO.Stream.WriteAsync*> alongside the synchronous methods <xref:System.IO.Stream.CopyTo*>, <xref:System.IO.Stream.Read*>, and <xref:System.IO.Stream.Write*>.

The Windows Runtime also contains many methods that you can use with `async` and `await` in Windows apps. For more information, see Threading and async programming for UWP development, and Asynchronous programming (Windows Store apps)) and Quickstart: Calling asynchronous APIs in C# or Visual Basic) if you use earlier versions of the Windows Runtime.

[csharp_125] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": Threads

Sumber: ht

Threads

Async methods are intended to be non-blocking operations. An `await` expression in an async method doesn't block the current thread while the awaited task is running. Instead, the expression signs up the rest of the method as a continuation and returns control to the caller of the async method.

The `async` and `await` keywords don't cause extra threads to be created. Async methods don't require multithreading because an async method doesn't run on its own thread. The method runs on the current synchronization context and uses time on the thread only when the method is active. You can use <xref:System.Threading.Tasks.Task.Run*?displayProperty=nameWithType> to move CPU-bound work to a background thread, but a background thread doesn't help with a process that's just waiting for results to become available.

The async-based approach to asynchronous programming is preferable to existing approaches in almost every case. In particular, this approach is better than the <xref:System.ComponentModel.BackgroundWorker> class for I/O-bound operations because the code is simpler and you don't have to guard against race conditions. In combination with the <xref:System.Threading.Tasks.Task.Run*?displayProperty=nameWithType> method, async programming is better than <xref:System.ComponentModel.BackgroundWorker> for CPU-bound operations because async programming separates the coordination details of running your code from the work that `Task.Run` transfers to the thread pool.

[csharp_126] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": Async and await

Sumber: ht

Async and await

If you specify that a method is an async method by using the async modifier, you enable the following two capabilities.

- The marked async method can use await to designate suspension points. The `await` operator tells the compiler that the async method can't continue past that point until the awaited asynchronous process is complete. In the meantime, control returns to the caller of the async method.

  The suspension of an async method at an `await` expression doesn't constitute an exit from the method, and `finally` blocks don't run.

- The marked async method can itself be awaited by methods that call it.

An async method typically contains one or more occurrences of an `await` operator, but the absence of `await` expressions doesn't cause a compiler error. If an async method doesn't use an `await` operator to mark a suspension point, the method executes as a synchronous method does, despite the `async` modifier. The compiler issues a warning for such methods.

`async` and `await` are contextual keywords. For more information and examples, see the following articles:

- async
- await

[csharp_127] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": Return types and parameters

Sumber: ht

Return types and parameters

An async method typically returns a <xref:System.Threading.Tasks.Task> or a <xref:System.Threading.Tasks.Task`1>. Inside an async method, an `await` operator is applied to a task that's returned from a call to another async method.

You specify <xref:System.Threading.Tasks.Task`1> as the return type if the method contains a `return` statement that specifies an operand of type `TResult`.

You use <xref:System.Threading.Tasks.Task> as the return type if the method has no return statement or has a return statement that doesn't return an operand.

You can also specify any other return type, if the type includes a `GetAwaiter` method. <xref:System.Threading.Tasks.ValueTask`1> is an example of such a type. It's available in the System.Threading.Tasks.Extension NuGet package.

The following example shows how you declare and call a method that returns a <xref:System.Threading.Tasks.Task`1> or a <xref:System.Threading.Tasks.Task>:

Each returned task represents ongoing work. A task encapsulates information about the state of the asynchronous process and, eventually, either the final result from the process or the exception that the process raises if it doesn't succeed.

An async method can also have a `void` return type. This return type is used primarily to define event handlers, where a `void` return type is required. Async event handlers often serve as the starting point for async programs.

An async method that has a `void` return type can't be awaited, and the caller of a void-returning method can't catch any exceptions that the method throws.

An async method can't declare in, ref or out parameters, but the method can call methods that have such parameters. Similarly, an async method can't return a value by reference, although it can call methods with ref return values.

For more information and examples, see Async return types (C#).

Asynchronous APIs in Windows Runtime programming have one of the following return types, which are similar to tasks:

- <xref:Windows.Foundation.IAsyncOperation`1>, which corresponds to <xref:System.Threading.Tasks.Task`1>
- <xref:Windows.Foundation.IAsyncAction>, which corresponds to <xref:System.Threading.Tasks.Task>
- <xref:Windows.Foundation.IAsyncActionWithProgress`1>
- <xref:Windows.Foundation.IAsyncOperationWithProgress`2>

Contoh Kode / Perintah CLI:

```
async Task<int> GetTaskOfTResultAsync()
{
    int hours = 0;
    await Task.Delay(0);
```

```
    return hours;
}


Task<int> returnedTaskTResult = GetTaskOfTResultAsync();
int intResult = await returnedTaskTResult;
// Single line
// int intResult = await GetTaskOfTResultAsync();

async Task GetTaskAsync()
{
    await Task.Delay(0);
    // No return statement needed
}

Task returnedTask = GetTaskAsync();
await returnedTask;
// Single line
await GetTaskAsync();
```

[csharp_128] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": Naming convention

Sumber: ht

Naming convention

By convention, methods that return commonly awaitable types (for example, `Task`, `Task<T>`, `ValueTask`, `ValueTask<T>`) should have names that end with "Async". Methods that start an asynchronous operation but don't return an awaitable type shouldn't have names that end with "Async", but might start with "Begin", "Start", or some other verb to suggest this method doesn't return or throw the result of the operation.

You can ignore the convention where an event, base class, or interface contract suggests a different name. For example, you shouldn't rename common event handlers, such as `OnButtonClick`.

[csharp_129] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": Related articles (Visual Studio)

Sumber: ht

Related articles (Visual Studio)

Title	Description
How to make multiple web requests in parallel by using async and await (C#)	Demonstrates how to start several tasks at the same time.
Async return types (C#)	Illustrates the types that async methods can return, and explains when each type is appropriate.
Cancel tasks with a cancellation token as a signaling mechanism.	Shows how to add the following functionality to your async solution: - Cancel a list of tasks (C#) - Cancel tasks after a period of time (C#) - Process asynchronous task as they complete (C#)
Using async for file access (C#)	Lists and demonstrates the benefits of using async and await to access files.
Task-based asynchronous pattern (TAP)	Describes an asynchronous pattern. The pattern is based on the <xref:System.Threading.Tasks.Task> and <xref:System.Threading.Tasks.Task`1> types.
Async Videos on Channel 9	Provides links to various videos about async programming.

[csharp_130] C# - Asynchronous Programming

The Task Asynchronous Programming (TAP) model with async and await": See also

Sumber: ht

See also

- Asynchronous programming with async and await
- async
- await

[csharp_131] C# - LINQ

"Introduction to LINQ Queries": Overview

Sumber: ht

Introduction to LINQ Queries in C\#

A *query* is an expression that retrieves data from a data source. Different data sources have different native query languages, for example SQL for relational databases and XQuery for XML. Developers must learn a new query language for each type of data source or data format that they must support. LINQ simplifies this situation by offering a consistent C# language model for kinds of data sources and formats. In a LINQ query, you always work with C# objects. You use the same basic coding patterns to query and transform data in XML documents, SQL databases, .NET collections, and any other format when a LINQ provider is available.

[csharp_132] C# - LINQ

"Introduction to LINQ Queries": Three Parts of a Query Operation

Sumber: ht

Three Parts of a Query Operation

All LINQ query operations consist of three distinct actions:

1. Obtain the data source.
1. Create the query.
1. Execute the query.

The following example shows how the three parts of a query operation are expressed in source code. The example uses an integer array as a data source for convenience; however, the same concepts apply to other data sources also. This example is referred to throughout the rest of this article.

The following illustration shows the complete query operation. In LINQ, the execution of the query is distinct from the query itself. In other words, you don't retrieve any data by creating a query variable.

!Diagram of the complete LINQ query operation.

Contoh Kode / Perintah CLI:

```
// The Three Parts of a LINQ Query:
    // 1. Data source.
    int[] numbers = [ 0, 1, 2, 3, 4, 5, 6 ];

    // 2. Query creation.
    // numQuery is an IEnumerable<int>
    var numQuery = from num in numbers
                   where (num % 2) == 0
```

```
                    select num;

    // 3. Query execution.
    foreach (int num in numQuery)
    {
        Console.Write("{0,1} ", num);
    }
```

[csharp_133] C# - LINQ

"Introduction to LINQ Queries": The Data Source

Sumber: ht

The Data Source

The data source in the preceding example is an array, which supports the generic <xref:System.Collections.Generic.IEnumerable`1> interface. This fact means it can be queried with LINQ. A query is executed in a `foreach` statement, and `foreach` requires <xref:System.Collections.IEnumerable> or <xref:System.Collections.Generic.IEnumerable`1>. Types that support <xref:System.Collections.Generic.IEnumerable`1> or a derived interface such as the generic <xref:System.Linq.IQueryable`1> are called *queryable types*.

A queryable type requires no modification or special treatment to serve as a LINQ data source. If the source data isn't already in memory as a queryable type, the LINQ provider must represent it as such. For example, LINQ to XML loads an XML document into a queryable <xref:System.Xml.Linq.XElement> type:

With EntityFramework, you create an object-relational mapping between C# classes and your database schema. You write your queries against the objects, and at run-time EntityFramework handles the communication with the database. In the following example, `Customers` represents a specific table in the database, and the type of the query result, <xref:System.Linq.IQueryable`1>, derives from <xref:System.Collections.Generic.IEnumerable`1>.

For more information about how to create specific types of data sources, see the documentation for the various LINQ providers. However, the basic rule is simple: a LINQ data source is any object that supports the generic <xref:System.Collections.Generic.IEnumerable`1> interface, or an interface that inherits from it, typically <xref:System.Linq.IQueryable`1>.
Types such as <xref:System.Collections.ArrayList> that support the non-generic <xref:System.Collections.IEnumerable> interface can also be used as a LINQ data source. For more information, see How to query an ArrayList with LINQ (C#).

Contoh Kode / Perintah CLI:

```
Northwnd db = new Northwnd(@"c:\northwnd.mdf");

// Query for customers in London.
IQueryable<Customer> custQuery =
    from cust in db.Customers
    where cust.City == "London"
    select cust;

// -----------------------------------------

// Create a data source from an XML document.
    // using System.Xml.Linq;
    XElement contacts = XElement.Load(@"c:\myContactList.xml");
```

[csharp_134] C# - LINQ

"Introduction to LINQ Queries": The Query

Sumber: ht

The Query

The query specifies what information to retrieve from the data source or sources. Optionally, a query also specifies how that information should be sorted, grouped, and shaped before being returned. A query is stored in a query variable and initialized with a query expression. You use C# query syntax to write queries.

The query in the previous example returns all the even numbers from the integer array. The query expression contains three clauses: `from`, `where`, and `select`. (If you're familiar with SQL, you noticed that the ordering of the clauses is reversed from the order in SQL.) The `from` clause specifies the data source, the `where` clause applies the filter, and the `select` clause specifies the type of the returned elements. All the query clauses are discussed in detail in this section. For now, the important point is that in LINQ, the query variable itself takes no action and returns no data. It just stores the information that is required to produce the results when the query is executed at some later point. For more information about how queries are constructed, see Standard Query Operators Overview (C#).
Queries can also be expressed by using method syntax. For more information, see Query Syntax and Method Syntax in LINQ.

[csharp_135] C# - LINQ

"Introduction to LINQ Queries": Classification of standard query operators by manner of execution

Sumber: ht

Classification of standard query operators by manner of execution

The LINQ to Objects implementations of the standard query operator methods execute in one of two main ways: *immediate* or *deferred*. The query operators that use deferred execution can be additionally divided into two categories: *streaming* and *nonstreaming*.

Immediate

Immediate execution means that the data source is read and the operation is performed once. All the standard query operators that return a scalar result execute immediately. Examples of such queries are `Count`, `Max`, `Average`, and `First`. These methods execute without an explicit `foreach` statement because the query itself must use `foreach` in order to return a result. These queries return a single value, not an `IEnumerable` collection. You can force *any* query to execute immediately using the <xref:System.Linq.Enumerable.ToList*?displayProperty=nameWithType> or <xref:System.Linq.Enumerable.ToArray*?displayProperty=nameWithType> methods. Immediate execution provides reuse of query results, not query declaration. The results are retrieved once, then stored for future use. The following query returns a count of the even numbers in the source array:

To force immediate execution of any query and cache its results, you can call the <xref:System.Linq.Enumerable.ToList*> or <xref:System.Linq.Enumerable.ToArray*> methods.

You can also force execution by putting the `foreach` loop immediately after the query expression. However, by calling `ToList` or `ToArray` you also cache all the data in a single collection object.

Deferred

Deferred execution means that the operation isn't performed at the point in the code where the query is declared. The operation is performed only when the query variable is enumerated, for example by using a `foreach` statement. The results of executing the query depend on the contents of the data source when the query is executed rather than when the query is defined. If the query variable is enumerated multiple times, the results might differ every time. Almost all the standard query operators whose return type is <xref:System.Collections.Generic.IEnumerable`1> or <xref:System.Linq.IOrderedEnumerable`1> execute in a deferred manner. Deferred execution provides the facility of query reuse since the query fetches the updated data from the data source each time query results are iterated. The following code shows an example of deferred execution:

The `foreach` statement is also where the query results are retrieved. For example, in the previous query, the iteration variable `num` holds each value (one at a time) in the returned sequence.

Because the query variable itself never holds the query results, you can execute it repeatedly to retrieve updated data. For example, a separate application might update a database continually. In *your* application, you could create one query that retrieves the latest data, and you could execute it at intervals to retrieve updated results.

Query operators that use deferred execution can be additionally classified as streaming or nonstreaming.

Streaming

Streaming operators don't have to read all the source data before they yield elements. At the time of execution, a streaming operator performs its operation on each source element as it is read and yields the element if appropriate. A streaming operator continues to read source elements until a result element can be produced. This means that more than one source element might be read to produce one result element.

Nonstreaming

Nonstreaming operators must read all the source data before they can yield a result element. Operations such as sorting or grouping fall into this category. At the time of execution, nonstreaming query operators read all the source data, put it into a data structure, perform the operation, and yield the resulting elements.

Contoh Kode / Perintah CLI:

```
var evenNumQuery = from num in numbers
                       where (num % 2) == 0
                       select num;

    int evenNumCount = evenNumQuery.Count();

// ------------------------------------------

List<int> numQuery2 = (from num in numbers
                           where (num % 2) == 0
                           select num).ToList();

    // or like this:
    // numQuery3 is still an int[]

    var numQuery3 = (from num in numbers
                     where (num % 2) == 0
                     select num).ToArray();

// ------------------------------------------
```

```
foreach (int num in numQuery)
    {
        Console.Write("{0,1} ", num);
    }
```

[csharp_136] C# - LINQ

"Introduction to LINQ Queries": Classification table

Sumber: ht

Classification table

The following table classifies each standard query operator method according to its method of execution.
If an operator is marked in two columns, two input sequences are involved in the operation, and each sequence is evaluated differently. In these cases, it is always the first sequence in the parameter list that is evaluated in a deferred, streaming manner.

Standard query operator	Return type	Immediate execution	Deferred streaming execution	Deferred nonstreaming execution
<xref:System.Linq.Enumerable.Aggregate*>	`TSource`	[OK]		
<xref:System.Linq.Enumerable.All*>	<xref:System.Boolean>	[OK]		
<xref:System.Linq.Enumerable.Any*>	<xref:System.Boolean>	[OK]		
<xref:System.Linq.Enumerable.AsEnumerable*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	
<xref:System.Linq.Enumerable.Average*>	Single numeric value	[OK]		
<xref:System.Linq.Enumerable.Cast*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	
<xref:System.Linq.Enumerable.Concat*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	
<xref:System.Linq.Enumerable.Contains*>	<xref:System.Boolean>	[OK]		
<xref:System.Linq.Enumerable.Count*>	<xref:System.Int32>	[OK]		
<xref:System.Linq.Enumerable.DefaultIfEmpty*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	
<xref:System.Linq.Enumerable.Distinct*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	
<xref:System.Linq.Enumerable.ElementAt*>	`TSource`	[OK]		
<xref:System.Linq.Enumerable.ElementAtOrDefault*>	`TSource?`	[OK]		
<xref:System.Linq.Enumerable.Empty*>	<xref:System.Collections.Generic.IEnumerable`1>	[OK]		
<xref:System.Linq.Enumerable.Except*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	[OK]
<xref:System.Linq.Enumerable.First*>	`TSource`	[OK]		
<xref:System.Linq.Enumerable.FirstOrDefault*>	`TSource?`	[OK]		
<xref:System.Linq.Enumerable.GroupBy*>	<xref:System.Collections.Generic.IEnumerable`1>			[OK]
<xref:System.Linq.Enumerable.GroupJoin*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	[OK]
<xref:System.Linq.Enumerable.Intersect*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	[OK]
<xref:System.Linq.Enumerable.Join*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	[OK]
<xref:System.Linq.Enumerable.Last*>	`TSource`	[OK]		
<xref:System.Linq.Enumerable.LastOrDefault*>	`TSource?`	[OK]		
<xref:System.Linq.Enumerable.LongCount*>	<xref:System.Int64>	[OK]		
<xref:System.Linq.Enumerable.Max*>	Single numeric value, `TSource`, or `TResult?`	[OK]		
<xref:System.Linq.Enumerable.Min*>	Single numeric value, `TSource`, or `TResult?`	[OK]		
<xref:System.Linq.Enumerable.OfType*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	
<xref:System.Linq.Enumerable.OrderBy*>	<xref:System.Linq.IOrderedEnumerable`1>			[OK]
<xref:System.Linq.Enumerable.OrderByDescending*>	<xref:System.Linq.IOrderedEnumerable`1>			[OK]
<xref:System.Linq.Enumerable.Range*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	
<xref:System.Linq.Enumerable.Repeat*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	
<xref:System.Linq.Enumerable.Reverse*>	<xref:System.Collections.Generic.IEnumerable`1>			[OK]
<xref:System.Linq.Enumerable.Select*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	
<xref:System.Linq.Enumerable.SelectMany*>	<xref:System.Collections.Generic.IEnumerable`1>		[OK]	

<xref:System.Linq.Enumerable.SequenceEqual*>	<xref:System.Boolean>	[OK]		
<xref:System.Linq.Enumerable.Single*>	`TSource`	[OK]		
<xref:System.Linq.Enumerable.SingleOrDefault*>	`TSource?`	[OK]		
<xref:System.Linq.Enumerable.Skip*>	<xref:System.Collections.Generic.IEnumerable`1>	[OK]		
<xref:System.Linq.Enumerable.SkipWhile*>	<xref:System.Collections.Generic.IEnumerable`1>	[OK]		
<xref:System.Linq.Enumerable.Sum*>	Single numeric value	[OK]		
<xref:System.Linq.Enumerable.Take*>	<xref:System.Collections.Generic.IEnumerable`1>	[OK]		
<xref:System.Linq.Enumerable.TakeWhile*>	<xref:System.Collections.Generic.IEnumerable`1>	[OK]		
<xref:System.Linq.Enumerable.ThenBy*>	<xref:System.Linq.IOrderedEnumerable`1>			[OK]
<xref:System.Linq.Enumerable.ThenByDescending*>	<xref:System.Linq.IOrderedEnumerable`1>			[OK]
<xref:System.Linq.Enumerable.ToArray*>	`TSource[]` array	[OK]		
<xref:System.Linq.Enumerable.ToDictionary*>	<xref:System.Collections.Generic.Dictionary`2>	[OK]		
<xref:System.Linq.Enumerable.ToList*>	<xref:System.Collections.Generic.IList`1>	[OK]		
<xref:System.Linq.Enumerable.ToLookup*>	<xref:System.Linq.ILookup`2>	[OK]		
<xref:System.Linq.Enumerable.Union*>	<xref:System.Collections.Generic.IEnumerable`1>	[OK]		
<xref:System.Linq.Enumerable.Where*>	<xref:System.Collections.Generic.IEnumerable`1>	[OK]		

[csharp_137] C# - LINQ

"Introduction to LINQ Queries": LINQ to objects

Sumber: ht

LINQ to objects

"LINQ to Objects" refers to the use of LINQ queries with any <xref:System.Collections.IEnumerable> or <xref:System.Collections.Generic.IEnumerable`1> collection directly. You can use LINQ to query any enumerable collections, such as <xref:System.Collections.Generic.List`1>, <xref:System.Array>, or <xref:System.Collections.Generic.Dictionary`2>. The collection can be user-defined or a type returned by a .NET API. In the LINQ approach, you write declarative code that describes what you want to retrieve. LINQ to Objects provides a great introduction to programming with LINQ.

LINQ queries offer three main advantages over traditional `foreach` loops:

- They're more concise and readable, especially when filtering multiple conditions.
- They provide powerful filtering, ordering, and grouping capabilities with a minimum of application code.
- They can be ported to other data sources with little or no modification.

The more complex the operation you want to perform on the data, the more benefit you realize using LINQ instead of traditional iteration techniques.

[csharp_138] C# - LINQ

"Introduction to LINQ Queries": Store the results of a query in memory

Sumber: ht

Store the results of a query in memory

A query is basically a set of instructions for how to retrieve and organize data. Queries are executed lazily, as each subsequent item in the result is requested. When you use `foreach` to iterate the results, items are returned as accessed. To evaluate a query and store its results without executing a `foreach` loop, just call one of the following methods on the query variable:

- <xref:System.Linq.Enumerable.ToList*>
- <xref:System.Linq.Enumerable.ToArray*>
- <xref:System.Linq.Enumerable.ToDictionary*>

- <xref:System.Linq.Enumerable.ToLookup*>

You should assign the returned collection object to a new variable when you store the query results, as shown in the following example:

Contoh Kode / Perintah CLI:

```
List<int> numbers = [ 1, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 ];

        IEnumerable<int> queryFactorsOfFour = from num in numbers
                                              where num % 4 == 0
                                              select num;

        // Store the results in a new variable
        // without executing a foreach loop.
        var factorsofFourList = queryFactorsOfFour.ToList();

        // Read and write from the newly created list to demonstrate that it holds data.
        Console.WriteLine(factorsofFourList[2]);
        factorsofFourList[2] = 0;
        Console.WriteLine(factorsofFourList[2]);
```

[csharp_139] C# - LINQ

"Introduction to LINQ Queries": See also

Sumber: ht

See also

- Walkthrough: Writing Queries in C#
- foreach, in
- Query Keywords (LINQ)

[csharp_140] C# - Language Reference - Statements

"Statement keywords": Overview

Sumber: ht

Statement keywords (C# Reference)

Statements are program instructions. Except as described in the topics referenced in the following list, the program executes statements in sequence. The following list shows the C# statement keywords. For more information about statements that don't use a keyword, see Statements.

- Selection statements
  - `if`
  - `switch`
- Iteration statements
  - `do`
  - `for`
  - `foreach`
  - `while`
- Jump statements
  - `break`
  - `continue`
  - `goto`
  - `return`
- Exception-handling statements

  - `throw`
  - `try-catch`
  - `try-finally`
  - `try-catch-finally`
- checked and unchecked statements
  - `checked`
  - `unchecked`
- fixed statement
  - `fixed`
- lock statement
  - `lock`
- yield statement
  - `yield`

[csharp_141] C# - Language Reference - Statements

"Statement keywords": See also

Sumber: ht

See also

- Statements
- C# Keywords

[csharp_142] C# - Language Reference - Parameters

"Method parameters and modifiers": Overview

Sumber: ht

Method parameters and modifiers

By default, C# passes arguments to functions *by value*. This approach passes a copy of the variable to the method. For value (`struct`) types, the method gets a copy of the *value*. For reference (`class`) types, the method gets a copy of the *reference*. You can use parameter modifiers to pass arguments *by reference*.

Because a struct is a value type, passing a struct by value to a method sends a copy of the argument to the method. The method works with this copy. The method can't access the original struct in the calling method and can't change it. The method can change only the copy.

A class instance is a reference type, not a value type. When you pass a reference type by value to a method, the method gets a copy of the reference to the instance. Both variables refer to the same object. The parameter is a copy of the reference. The called method can't reassign the instance in the calling method. However, the called method can use the copy of the reference to access the instance members. If the called method changes an instance member, the calling method also sees those changes since it references the same instance.

!INCLUDE[csharp-version-note]

[csharp_143] C# - Language Reference - Parameters

"Method parameters and modifiers": Pass by value and pass by reference

Sumber: ht

Pass by value and pass by reference

All the examples in this section use the following two `record` types to illustrate the differences between `class` types and `struct` types:

The output of the following example illustrates the difference between passing a struct type by value and passing a class type by value. Both `Mutate` methods change property values of their argument. When the parameter is a `struct` type, those changes affect a copy of the argument's data. When the parameter is a `class` type, those changes affect the instance referred to by the argument:

The `ref` modifier is one way to pass arguments *by reference* to methods. The following code replicates the preceding example, but passes parameters by reference. The modifications made to the `struct` type are visible in the calling method when the struct is passed by reference. There's no semantic change when a reference type is passed by reference:

The preceding examples modified properties of a parameter. A method can also reassign a parameter to a new value. Reassignment behaves differently for struct and class types when passed by value or by reference. The following example shows how struct types and class types behave when parameters that are passed by value are reassigned:

The preceding sample shows that when you reassign a parameter to a new value, that change isn't visible from the calling method, regardless of whether the type is a value type or a reference type. The following example shows the behavior when you reassign a parameter that the method received by reference:

The preceding example shows how reassigning the value of a parameter that is passed by reference is visible in the calling context.

Contoh Kode / Perintah CLI:

```
public record struct Point(int X, int Y);
// This doesn't use a primary constructor because the properties implemented for `record` types are
// readonly in record class types. That would prevent the mutations necessary for this example.
public record class Point3D
{
    public int X { get; set; }
    public int Y { get; set; }
    public int Z { get; set; }
}

// ----------------------------------------

public class PassTypesByValue
{
    public static void Mutate(Point pt)
```

```
    {
        Console.WriteLine($"\tEnter {nameof(Mutate)}:\t\t{pt}");
        pt.X = 19;
        pt.Y = 23;

        Console.WriteLine($"\tExit {nameof(Mutate)}:\t\t{pt}");
    }
    public static void Mutate(Point3D pt)
    {
        Console.WriteLine($"\tEnter {nameof(Mutate)}:\t\t{pt}");
        pt.X = 19;
        pt.Y = 23;
        pt.Z = 42;

        Console.WriteLine($"\tExit {nameof(Mutate)}:\t\t{pt}");
    }

    public static void TestPassTypesByValue()
    {
        Console.WriteLine("===== Value Types =====");

        var ptStruct = new Point { X = 1, Y = 2 };
        Console.WriteLine($"After initialization:\t\t{ptStruct}");

        Mutate(ptStruct);

        Console.WriteLine($"After called {nameof(Mutate)}:\t\t{ptStruct}");

        Console.WriteLine("===== Reference Types =====");

        var ptClass = new Point3D { X = 1, Y = 2, Z = 3 };

        Console.WriteLine($"After initialization:\t\t{ptClass}");

        Mutate(ptClass);
        Console.WriteLine($"After called {nameof(Mutate)}:\t\t{ptClass}");

        // Output:
        // ===== Value Types =====
        // After initialization:           Point { X = 1, Y = 2 }
        //         Enter Mutate:           Point { X = 1, Y = 2 }
        //         Exit Mutate:            Point { X = 19, Y = 23 }
        // After called Mutate:            Point { X = 1, Y = 2 }
        // ===== Reference Types =====
        // After initialization:           Point3D { X = 1, Y = 2, Z = 3 }
        //         Enter Mutate:           Point3D { X = 1, Y = 2, Z = 3 }
        //         Exit Mutate:            Point3D { X = 19, Y = 23, Z = 42 }
        // After called Mutate:            Point3D { X = 19, Y = 23, Z = 42 }
    }
}

// ----------------------------------------

public class PassTypesByReference
{
    public static void Mutate(ref Point pt)
    {
        Console.WriteLine($"\tEnter {nameof(Mutate)}:\t\t{pt}");
        pt.X = 19;
        pt.Y = 23;

        Console.WriteLine($"\tExit {nameof(Mutate)}:\t\t{pt}");
```

```
    }
    public static void Mutate(ref Point3D pt)
    {
        Console.WriteLine($"\tEnter {nameof(Mutate)}:\t\t{pt}");
        pt.X = 19;
        pt.Y = 23;
        pt.Z = 42;

        Console.WriteLine($"\tExit {nameof(Mutate)}:\t\t{pt}");
    }

    public static void TestPassTypesByReference()
    {
        Console.WriteLine("===== Value Types =====");

        var pStruct = new Point { X = 1, Y = 2 };
        Console.WriteLine($"After initialization:\t\t{pStruct}");

        Mutate(ref pStruct);

        Console.WriteLine($"After called {nameof(Mutate)}:\t\t{pStruct}");

        Console.WriteLine("===== Reference Types =====");

        var pClass = new Point3D { X = 1, Y = 2, Z = 3 };

        Console.WriteLine($"After initialization:\t\t{pClass}");

        Mutate(ref pClass);
        Console.WriteLine($"After called {nameof(Mutate)}:\t\t{pClass}");

        // Output:
        // ===== Value Types =====
        // After initialization:          Point { X = 1, Y = 2 }
        //         Enter Mutate:          Point { X = 1, Y = 2 }
        //         Exit Mutate:           Point { X = 19, Y = 23 }
        // After called Mutate:           Point { X = 19, Y = 23 }
        // ===== Reference Types =====
        // After initialization:          Point3D { X = 1, Y = 2, Z = 3 }
        //         Enter Mutate:          Point3D { X = 1, Y = 2, Z = 3 }
        //         Exit Mutate:           Point3D { X = 19, Y = 23, Z = 42 }
        // After called Mutate:           Point3D { X = 19, Y = 23, Z = 42 }
    }
}

// ----------------------------------------

public class PassByValueReassignment
{
    public static void Reassign(Point pt)
    {
        Console.WriteLine($"\tEnter {nameof(Reassign)}:\t\t{pt}");
        pt = new Point { X = 13, Y = 29 };

        Console.WriteLine($"\tExit {nameof(Reassign)}:\t\t{pt}");
    }

    public static void Reassign(Point3D pt)
    {
        Console.WriteLine($"\tEnter {nameof(Reassign)}:\t\t{pt}");
        pt = new Point3D { X = 13, Y = 29, Z = -42 };
```

```
        Console.WriteLine($"\tExit {nameof(Reassign)}:\t\t{pt}");
    }

    public static void TestPassByValueReassignment()
    {
        Console.WriteLine("===== Value Types =====");

        var ptStruct = new Point { X = 1, Y = 2 };
        Console.WriteLine($"After initialization:\t\t{ptStruct}");

        Reassign(ptStruct);

        Console.WriteLine($"After called {nameof(Reassign)}:\t\t{ptStruct}");

        Console.WriteLine("===== Reference Types =====");

        var ptClass = new Point3D { X = 1, Y = 2, Z = 3 };

        Console.WriteLine($"After initialization:\t\t{ptClass}");

        Reassign(ptClass);
        Console.WriteLine($"After called {nameof(Reassign)}:\t\t{ptClass}");

        // Output:
        // ===== Value Types =====
        // After initialization:           Point { X = 1, Y = 2 }
        //         Enter Reassign:         Point { X = 1, Y = 2 }
        //         Exit Reassign:          Point { X = 13, Y = 29 }
        // After called Reassign:          Point { X = 1, Y = 2 }
        // ===== Reference Types =====
        // After initialization:           Point3D { X = 1, Y = 2, Z = 3 }
        //         Enter Reassign:         Point3D { X = 1, Y = 2, Z = 3 }
        //         Exit Reassign:          Point3D { X = 13, Y = 29, Z = -42 }
        // After called Reassign:          Point3D { X = 1, Y = 2, Z = 3 }
    }
}

// ----------------------------------------

public class PassByReferenceReassignment
{
    public static void Reassign(ref Point pt)
    {
        Console.WriteLine($"\tEnter {nameof(Reassign)}:\t\t{pt}");
        pt = new Point { X = 13, Y = 29 };

        Console.WriteLine($"\tExit {nameof(Reassign)}:\t\t{pt}");
    }

    public static void Reassign(ref Point3D pt)
    {
        Console.WriteLine($"\tEnter {nameof(Reassign)}:\t\t{pt}");
        pt = new Point3D { X = 13, Y = 29, Z = -42 };

        Console.WriteLine($"\tExit {nameof(Reassign)}:\t\t{pt}");
    }

    public static void TestPassByReferenceReassignment()
    {
        Console.WriteLine("===== Value Types =====");

        var ptStruct = new Point { X = 1, Y = 2 };
```

```
        Console.WriteLine($"After initialization:\t\t{ptStruct}");

        Reassign(ref ptStruct);

        Console.WriteLine($"After called {nameof(Reassign)}:\t\t{ptStruct}");

        Console.WriteLine("===== Reference Types =====");

        var ptClass = new Point3D { X = 1, Y = 2, Z = 3 };

        Console.WriteLine($"After initialization:\t\t{ptClass}");

        Reassign(ref ptClass);
        Console.WriteLine($"After called {nameof(Reassign)}:\t\t{ptClass}");

        // Output:
        // ===== Value Types =====
        // After initialization:           Point { X = 1, Y = 2 }
        //         Enter Reassign:         Point { X = 1, Y = 2 }
        //         Exit Reassign:          Point { X = 13, Y = 29 }
        // After called Reassign:          Point { X = 13, Y = 29 }
        // ===== Reference Types =====
        // After initialization:           Point3D { X = 1, Y = 2, Z = 3 }
        //         Enter Reassign:         Point3D { X = 1, Y = 2, Z = 3 }
        //         Exit Reassign:          Point3D { X = 13, Y = 29, Z = -42 }
        // After called Reassign:          Point3D { X = 13, Y = 29, Z = -42 }
    }
}
```

[csharp_144] C# - Language Reference - Parameters

"Method parameters and modifiers": Safe context of references and values

Sumber: ht

Safe context of references and values

Methods can store the values of parameters in fields. When you pass parameters by value, it's usually safe. The method copies values, and reference types are reachable when the method stores them in a field. Passing parameters by reference safely requires the compiler to define when it's safe to assign a reference to a new variable. For every expression, the compiler defines a *safe context* that bounds access to an expression or variable. The compiler uses two scopes: *safe-context* and *ref-safe-context*.

- The *safe-context* defines the scope where any expression can be safely accessed.
- The *ref-safe-context* defines the scope where a *reference* to any expression can be safely accessed or modified.

Informally, you can think of these scopes as the mechanism to ensure your code never accesses or modifies a reference that's no longer valid. A reference is valid as long as it refers to a valid object or struct. The *safe-context* defines when a variable can be assigned or reassigned. The *ref-safe-context* defines when a variable can be *ref assigned* or *ref reassigned*. Assignment assigns a variable to a new value; *ref assignment* assigns the variable to *refer to* a different storage location.

[csharp_145] C# - Language Reference - Parameters

"Method parameters and modifiers": Reference parameters

Sumber: ht

Reference parameters

To pass arguments by reference instead of by value, use one of the following modifiers in a parameter declaration:

- `ref`: Initialize the argument before calling the method. The method can assign a new value to the parameter, but it isn't required to.
- `out`: The calling method doesn't need to initialize the argument before calling the method. The method must assign a value to the parameter.
- `ref readonly`: Initialize the argument before calling the method. The method can't assign a new value to the parameter.
- `in`: Initialize the argument before calling the method. The method can't assign a new value to the parameter. The compiler might create a temporary variable to hold a copy of the argument to `in` parameters.

A parameter that's passed by reference is a *reference variable*. It doesn't have its own value. Instead, it refers to a different variable called its *referent*. You can ref reassign reference variables, which changes their referent.

Members of a class can't have signatures that differ only by `ref`, `ref readonly`, `in`, or `out`. A compiler error occurs if the only difference between two members of a type is that one member has a `ref` parameter and the other member has an `out`, `ref readonly`, or `in` parameter. However, you can overload methods when one method has a `ref`, `ref readonly`, `in`, or `out` parameter and the other method has a parameter that's passed by value, as shown in the following example. In other situations that require signature matching, such as hiding or overriding, `in`, `ref`, `ref readonly`, and `out` are part of the signature and don't match each other.

When a parameter has one of the preceding modifiers, the corresponding argument can have a compatible modifier:

- An argument for a `ref` parameter must include the `ref` modifier.
- An argument for an `out` parameter must include the `out` modifier.
- An argument for an `in` parameter can optionally include the `in` modifier. If the `ref` modifier is used on the argument instead, the compiler issues a warning.
- An argument for a `ref readonly` parameter should include either the `in` or `ref` modifiers, but not both. If neither modifier is included, the compiler issues a warning.

When you use these modifiers, they describe how the argument is used:

- `ref` means the method can read or write the value of the argument.
- `out` means the method sets the value of the argument.
- `ref readonly` means the method reads, but can't write the value of the argument. The argument *should* be passed by reference.
- `in` means the method reads, but can't write the value of the argument. The argument is passed by reference or through a temporary variable.

You can't use the previous parameter modifiers in the following kinds of methods:

- Async methods, which you define by using the async modifier.
- Iterator methods, which include a yield return or `yield break` statement.

Extension members also have restrictions on the use of these argument keywords:

- The `out` keyword can't be used on the first argument of an extension method.
- The `ref` keyword can't be used on the first argument of an extension method when the argument isn't a `struct`, or a generic type not constrained to be a struct.
- The `ref readonly` and `in` keywords can't be used unless the first argument is a `struct`.
- The `ref readonly` and `in` keywords can't be used on any generic type, even when constrained to be a struct.

Properties aren't variables. They're methods. You can't use properties as arguments for `ref` parameters.

`ref` parameter modifier

To use a `ref` parameter, both the method definition and the calling method must explicitly use the `ref` keyword, as shown in the following example. (Except that the calling method can omit `ref` when making a COM call.)

You must initialize an argument before you pass it to a `ref` parameter.

`out` parameter modifier

To use an `out` parameter, both the method definition and the calling method must explicitly use the `out` keyword. For example:

You don't need to initialize variables passed as `out` arguments before the method call. However, the called method must assign a value before it returns.

Deconstruct methods declare their parameters with the `out` modifier to return multiple values. Other methods can return value tuples for multiple return values.

You can declare a variable in a separate statement before you pass it as an `out` argument. You can also declare the `out` variable in the argument list of the method call, rather than in a separate variable declaration. `out` variable declarations produce more compact, readable code, and also prevent you from inadvertently assigning a value to the variable before the method call. The following example defines the `number` variable in the call to the Int32.TryParse) method.

You can also declare an implicitly typed local variable.

`ref readonly` modifier

The method declaration requires the `ref readonly` modifier. A modifier at the call site is optional. You can use either the `in` or `ref` modifier. The `ref readonly` modifier isn't valid at the call site. The modifier you use at the call site can help describe characteristics of the argument. You can use `ref` only if the argument is a variable and is writable. You can use `in` only when the argument is a variable. The variable might be writable or readonly. You can't add either modifier if the argument isn't a variable but is an expression. The following examples show these conditions. The following method uses the `ref readonly` modifier to indicate that a large struct should be passed by reference for performance reasons:

You can call the method by using the `ref` or `in` modifier. If you omit the modifier, the compiler issues a warning. When

the argument is an expression, not a variable, you can't add the `in` or `ref` modifiers, so you should suppress the warning:

If the variable is a `readonly` variable, you must use the `in` modifier. The compiler issues an error if you use the `ref` modifier instead.

The `ref readonly` modifier indicates that the method expects the argument to be a variable rather than an expression that isn't a variable. Examples of expressions that aren't variables are constants, method return values, and properties. If the argument isn't a variable, the compiler issues a warning.

`in` parameter modifier

The `in` modifier is required in the method declaration but unnecessary at the call site.

The `in` modifier enables the compiler to create a temporary variable for the argument and pass a readonly reference to that argument. The compiler always creates a temporary variable when the argument must be converted, when there's an implicit conversion from the argument type, or when the argument is a value that isn't a variable. For example, when the argument is a literal value, or the value returned from a property accessor. When your API requires that the argument be passed by reference, choose the `ref readonly` modifier instead of the `in` modifier.

You can gain performance optimization by defining methods with `in` parameters. Some `struct` type arguments might be large in size, and when you call methods in tight loops or critical code paths, the cost of copying those structures is substantial. Declare `in` parameters to specify that you can safely pass arguments by reference because the called method doesn't modify the state of that argument. Passing those arguments by reference avoids the (potentially) expensive copy. You explicitly add the `in` modifier at the call site to ensure the argument is passed by reference, not by value. Explicitly using `in` has the following two effects:

- Specifying `in` at the call site forces the compiler to select a method defined with a matching `in` parameter. Otherwise, when two methods differ only in the presence of `in`, the by value overload is a better match.
- By specifying `in`, you declare your intent to pass an argument by reference. The argument used with `in` must represent a location that can be directly referred to. The same general rules for `out` and `ref` arguments apply: You can't use constants, ordinary properties, or other expressions that produce values. Otherwise, omitting `in` at the call site informs the compiler that it's fine to create a temporary variable to pass by read-only reference to the method. The compiler creates a temporary variable to overcome several restrictions with `in` arguments:
  - A temporary variable allows compile-time constants as `in` parameters.
  - A temporary variable allows properties, or other expressions for `in` parameters.
  - A temporary variable allows arguments where there's an implicit conversion from the argument type to the parameter type.

In all the preceding instances, the compiler creates a temporary variable that stores the value of the constant, property, or other expression.

The following code illustrates these rules:

Now, suppose another method using by-value arguments was available. The results change as shown in the following code:

The only method call where the argument is passed by reference is the final one.
The preceding code uses `int` as the argument type for simplicity. Because `int` is no larger than a reference in most modern machines, there's no benefit to passing a single `int` as a readonly reference.

Contoh Kode / Perintah CLI:

```
static void Method(in int argument)
{
    // implementation removed
}

Method(5); // OK, temporary variable created.
Method(5L); // CS1503: no implicit conversion from long to int
short s = 0;
Method(s); // OK, temporary int created with the value 0
Method(in s); // CS1503: cannot convert from in short to in int
int i = 42;
Method(i); // passed by readonly reference
Method(in i); // passed by readonly reference, explicitly using `in`

// ----------------------------------------

static void Method(int argument)
{
    // implementation removed
}

static void Method(in int argument)
{
    // implementation removed
}

Method(5); // Calls overload passed by value
Method(5L); // CS1503: no implicit conversion from long to int
short s = 0;
Method(s); // Calls overload passed by value.
Method(in s); // CS1503: cannot convert from in short to in int
int i = 42;
Method(i); // Calls overload passed by value
Method(in i); // passed by readonly reference, explicitly using `in`

// ----------------------------------------

void Method(ref int refArgument)
            {
                refArgument = refArgument + 44;
            }

            int number = 1;
            Method(ref number);
            Console.WriteLine(number);
            // Output: 45

// ----------------------------------------

int initializeInMethod;
            OutArgExample(out initializeInMethod);
            Console.WriteLine(initializeInMethod);     // value is now 44
```

```
            void OutArgExample(out int number)
            {
                number = 44;
            }

// ----------------------------------------

string numberAsString = "1640";

            if (Int32.TryParse(numberAsString, out int number))
                Console.WriteLine($"Converted '{numberAsString}' to {number}");
            else
                Console.WriteLine($"Unable to convert '{numberAsString}'");
            // The example displays the following output:
            //       Converted '1640' to 1640

// ----------------------------------------

public static void ForceByRef(ref readonly OptionStruct thing)
        {
            // elided
        }

// ----------------------------------------

ForceByRef(in options);
            ForceByRef(ref options);
            ForceByRef(options); // Warning! variable should be passed with `ref` or `in`
            ForceByRef(new OptionStruct()); // Warning, but an expression, so no variable to reference

// ----------------------------------------

var largeStruct = new LargeStruct { Value1 = 42, Value2 = 3.14, Value3 = "Hello" };

            // Using 'in' avoids copying the large struct and prevents modification
            ProcessLargeStruct(in largeStruct);
            Console.WriteLine($"Original value unchanged: {largeStruct.Value1}");

            // Without 'in', the struct would be copied (less efficient for large structs)
            ProcessLargeStructByValue(largeStruct);
            Console.WriteLine($"Original value still unchanged: {largeStruct.Value1}");

            void ProcessLargeStruct(in LargeStruct data)
            {
                // Can read the values
                Console.WriteLine($"Processing: {data.Value1}, {data.Value2}, {data.Value3}");

                // Uncomment the following line to see error CS8331
                // data.Value1 = 99; // Compilation error: cannot assign to 'in' parameter
            }

            void ProcessLargeStructByValue(LargeStruct data)
            {
                // This method receives a copy of the struct
                Console.WriteLine($"Processing copy: {data.Value1}, {data.Value2}, {data.Value3}");

                // Modifying the copy doesn't affect the original
                data.Value1 = 99;
            }
```

[csharp_146] C# - Language Reference - Parameters

"Method parameters and modifiers": `params` modifier

Sumber: ht

`params` modifier

The parameter with the `params` keyword must be the last parameter in the method declaration. You can only use one `params` keyword in a method declaration.

You must declare the `params` parameter as a collection type. Recognized collection types include:

- A single-dimensional *array type* `T[]`, where the *element type* is `T`.
- A *span type*:
  - `System.Span<T>`
  - `System.ReadOnlySpan<T>`
  In these types, the *element type* is `T`.
- A *type* with an accessible *create method* that has a corresponding *element type*. The *create method* uses the same attribute as collection expressions.
- A *struct* or *class type* that implements <xref:System.Collections.Generic.IEnumerable`1?displayProperty=fullName> where:
  - The *type* has a constructor that you can invoke without arguments, and the constructor is at least as accessible as the declaring member.
  - The *type* has an instance (not an extension) method `Add` where:
    - The method can be invoked with a single value argument.
    - If the method is generic, the type arguments can be inferred from the argument.
    - The method is at least as accessible as the declaring member.
  In this case, the *element type* is the *iteration type* of the *type*.
- An *interface type*:
  - <xref:System.Collections.Generic.IEnumerable`1?displayProperty=fullName>
  - <xref:System.Collections.Generic.IReadOnlyCollection`1?displayProperty=fullName>
  - <xref:System.Collections.Generic.IReadOnlyList`1?displayProperty=fullName>
  - <xref:System.Collections.Generic.ICollection`1?displayProperty=fullName>
  - <xref:System.Collections.Generic.IList`1?displayProperty=fullName>
  In these types, the *element type* is `T`.

Before C# 13, you must use a single-dimensional array for the parameter.

When you call a method with a `params` parameter, you can pass in:

- A comma-separated list of arguments of the type of the array elements.
- A collection of arguments of the specified type.
- No arguments. If you send no arguments, the length of the `params` list is zero.

The following example demonstrates various ways to send arguments to a `params` parameter.

Overload resolution can cause ambiguity when the argument for a `params` parameter is a collection type. The collection type of the argument must be convertible to the collection type of the parameter. When different overloads provide better conversions for that parameter, that method might be better. However, if the argument to the `params` parameter is either discrete elements or missing, all overloads with different `params` parameter types are equal for that parameter.

For more information, see the section on Argument lists in the C# Language Specification. The language specification is the definitive source for C# syntax and usage.

Contoh Kode / Perintah CLI:

```
public static void ParamsModifierExample(params int[] list)
    {
        for (int i = 0; i < list.Length; i++)
        {
            System.Console.Write(list[i] + " ");
        }
        System.Console.WriteLine();
    }

    public static void ParamsModifierObjectExample(params object[] list)
    {
        for (int i = 0; i < list.Length; i++)
        {
            System.Console.Write(list[i] + " ");
        }
        System.Console.WriteLine();
    }

    public static void TryParamsCalls()
    {
        // You can send a comma-separated list of arguments of the
        // specified type.
        ParamsModifierExample(1, 2, 3, 4);
        ParamsModifierObjectExample(1, 'a', "test");

        // A params parameter accepts zero or more arguments.
        // The following calling statement displays only a blank line.
        ParamsModifierObjectExample();

        // An array argument can be passed, as long as the array
        // type matches the parameter type of the method being called.
        int[] myIntArray = { 5, 6, 7, 8, 9 };
        ParamsModifierExample(myIntArray);

        object[] myObjArray = { 2, 'b', "test", "again" };
        ParamsModifierObjectExample(myObjArray);

        // The following call causes a compiler error because the object
        // array cannot be converted into an integer array.
        //ParamsModifierExample(myObjArray);

        // The following call does not cause an error, but the entire
        // integer array becomes the first element of the params array.
        ParamsModifierObjectExample(myIntArray);
    }
    /*
    Output:
        1 2 3 4
        1 a test

        5 6 7 8 9
        2 b test again
        System.Int32[]
    */
```

[csharp_147] C# - Language Reference - Operators

"?? and ??= operators - null-coalescing operators": Overview

Sumber: ht

`??` and `??=` operators - the null-coalescing operators

The null-coalescing operator `??` returns the value of its left-hand operand if it's not `null`. Otherwise, it evaluates the right-hand operand and returns its result. The `??` operator doesn't evaluate its right-hand operand if the left-hand operand evaluates to non-null. The null-coalescing assignment operator `??=` assigns the value of its right-hand operand to its left-hand operand only if the left-hand operand evaluates to `null`. The `??=` operator doesn't evaluate its right-hand operand if the left-hand operand evaluates to non-null.

The left-hand operand of the `??=` operator must be a variable, a property, or an indexer element.

!INCLUDE[csharp-version-note]

The type of the left-hand operand of the `??` and `??=` operators can't be a non-nullable value type. In particular, you can use the null-coalescing operators with unconstrained type parameters:

The null-coalescing operators are right-associative. That is, expressions of the form

are evaluated as

Contoh Kode / Perintah CLI:

```
a ?? b ?? c
d ??= e ??= f

// ----------------------------------------

a ?? (b ?? c)
d ??= (e ??= f)
```

[csharp_148] C# - Language Reference - Operators

"?? and ??= operators - null-coalescing operators": Examples

Sumber: ht

Examples

The `??` and `??=` operators are useful in the following scenarios:

- In expressions that use the [null-conditional operators `?.` and `?[]`](member-access-operators.md#null-conditional-operators--and-), use the `??` operator to provide an alternative expression to evaluate if the result of the expression with null-conditional operations is `null`:

- When you work with nullable value types and need to provide a value of an underlying value type, use the `??` operator to specify the value to provide if a nullable type value is `null`:

Use the <xref:System.Nullable`1.GetValueOrDefault?displayProperty=nameWithType> method if the value to use when a nullable type value is `null` should be the default value of the underlying value type.

- To make the argument-checking code more concise, use a `throw` expression as the right-hand operand of the `??` operator:

The preceding example also demonstrates how to use expression-bodied members to define a property.

- Use the `??=` operator to replace code of the following form:

Use the following code:

Contoh Kode / Perintah CLI:

```
if (variable is null)
  {
      variable = expression;
  }

// ----------------------------------------

variable ??= expression;
```

[csharp_149] C# - Language Reference - Operators

"?? and ??= operators - null-coalescing operators": Operator overloadability

Sumber: ht

Operator overloadability

You can't overload the `??` and `??=` operators.

[csharp_150] C# - Language Reference - Operators

"?? and ??= operators - null-coalescing operators": C# language specification

Sumber: ht

C# language specification

For more information about the `??` operator, see The null coalescing operator section of the C# language specification.

For more information about the `??=` operator, see the Compound assignment section of the C# language specification.

[csharp_151] C# - Language Reference - Operators

"?? and ??= operators - null-coalescing operators": See also

Sumber: ht

See also

- Null check can be simplified (IDE0029, IDE0030, and IDE0270)
- C# operators and expressions
- [?. and ?[] operators](member-access-operators.md#null-conditional-operators--and-)
- ?: operator

[csharp_152] C# - Whats New in C#

What's new in C# 12: Overview

Sumber: ht

What's new in C# 12

C# 12 includes the following new features. You can try these features using the latest Visual Studio 2022 version or the .NET 8 SDK.

- Primary constructors - Introduced in Visual Studio 2022 version 17.6 Preview 2.
- Collection expressions - Introduced in Visual Studio 2022 version 17.7 Preview 5.
- Inline arrays - Introduced in Visual Studio 2022 version 17.7 Preview 3.
- Optional parameters in lambda expressions - Introduced in Visual Studio 2022 version 17.5 Preview 2.
- `ref readonly` parameters - Introduced in Visual Studio 2022 version 17.8 Preview 2.
- Alias any type - Introduced in Visual Studio 2022 version 17.6 Preview 3.
- Experimental attribute - Introduced in Visual Studio 2022 version 17.7 Preview 3.

- Interceptors - *Preview feature* Introduced in Visual Studio 2022 version 17.7 Preview 3.

C# 12 is supported on **.NET 8**. For more information, see C# language versioning.

You can download the latest .NET 8 SDK from the .NET downloads page. You can also download Visual Studio 2022, which includes the .NET 8 SDK.

!INCLUDE [released-version-feedback]

[csharp_153] C# - Whats New in C#

What's new in C# 12: Primary constructors

Sumber: ht

Primary constructors

You can now create primary constructors in any `class` and `struct`. Primary constructors are no longer restricted to `record` types. Primary constructor parameters are in scope for the entire body of the class. To ensure that all primary constructor parameters are definitely assigned, all explicitly declared constructors must call the primary constructor using `this()` syntax. Adding a primary constructor to a `class` prevents the compiler from declaring an implicit parameterless constructor. In a `struct`, the implicit parameterless constructor initializes all fields, including primary constructor parameters to the 0-bit pattern.

The compiler generates public properties for primary constructor parameters only in `record` types, either `record class` or `record struct` types. Nonrecord classes and structs might not always want this behavior for primary constructor parameters.

You can learn more about primary constructors in the tutorial for exploring primary constructors and in the article on instance constructors.

[csharp_154] C# - Whats New in C#

What's new in C# 12: Collection expressions

Sumber: ht

Collection expressions

Collection expressions introduce a new terse syntax to create common collection values. Inlining other collections into these values is possible using a spread element `..e`.

Several collection-like types can be created without requiring external BCL support. These types are:

- Array types, such as `int[]`.
- <xref:System.Span`1?displayProperty=nameWithType> and <xref:System.ReadOnlySpan`1?displayProperty=nameWithType>.
- Types that support collection initializers, such as <xref:System.Collections.Generic.List`1?displayProperty=nameWithType>.

The following examples show uses of collection expressions:

The *spread element*, `..e` in a collection expression adds all the elements in that expression. The argument must be a collection type. The following examples show how the spread element works:

The spread element evaluates each element of the enumerations expression. Each element is included in the output collection.

You can use collection expressions anywhere you need a collection of elements. They can specify the initial value for a collection or be passed as arguments to methods that take collection types. You can learn more about collection expressions in the language reference article on collection expressions or the Section 12.8.25 Collection expressions clause of the C# standard.

Contoh Kode / Perintah CLI:

```
// Create an array:
int[] a = [1, 2, 3, 4, 5, 6, 7, 8];

// Create a list:
List<string> b = ["one", "two", "three"];

// Create a span
Span<char> c  = ['a', 'b', 'c', 'd', 'e', 'f', 'h', 'i'];

// Create a jagged 2D array:
int[][] twoD = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];

// Create a jagged 2D array from variables:
int[] row0 = [1, 2, 3];
int[] row1 = [4, 5, 6];
int[] row2 = [7, 8, 9];
int[][] twoDFromVariables = [row0, row1, row2];
```

```
// ----------------------------------------

int[] row0 = [1, 2, 3];
int[] row1 = [4, 5, 6];
int[] row2 = [7, 8, 9];
int[] single = [.. row0, .. row1, .. row2];
foreach (var element in single)
{
    Console.Write($"{element}, ");
}
// output:
// 1, 2, 3, 4, 5, 6, 7, 8, 9,
```

[csharp_155] C# - Whats New in C#

What's new in C# 12: `ref readonly` parameters

Sumber: ht

`ref readonly` parameters

C# added `in` parameters as a way to pass readonly references. `in` parameters allow both variables and values, and can be used without any annotation on arguments.

The addition of `ref readonly` parameters enables more clarity for APIs that might be using `ref` parameters or `in` parameters:

- APIs created before `in` was introduced might use `ref` even though the argument isn't modified. Those APIs can be updated with `ref readonly`. It won't be a breaking change for callers, as would be if the `ref` parameter was changed to `in`. An example is <xref:System.Runtime.InteropServices.Marshal.QueryInterface*?displayProperty=fullName>.
- APIs that take an `in` parameter, but logically require a variable. A value expression doesn't work. An example is <xref:System.ReadOnlySpan`1.%23ctor(`0@)?displayProperty=fullName>.
- APIs that use `ref` because they require a variable, but don't mutate that variable. An example is <xref:System.Runtime.CompilerServices.Unsafe.IsNullRef*?displayProperty=fullName>.

To learn more about `ref readonly` parameters, see the article on parameter modifiers in the language reference, or the ref readonly parameters feature specification.

[csharp_156] C# - Whats New in C#

What's new in C# 12: Default lambda parameters

Sumber: ht

Default lambda parameters

You can now define default values for parameters on lambda expressions. The syntax and rules are the same as adding default values for arguments to any method or local function.

You can learn more about default parameters on lambda expressions in the article on lambda expressions.

[csharp_157] C# - Whats New in C#

What's new in C# 12: Alias any type

Sumber: ht

Alias any type

You can use the `using` alias directive to alias any type, not just named types. That means you can create semantic aliases for tuple types, array types, pointer types, or other unsafe types. For more information, see Section 14.6.2 Using alias directives in the C# standard. For an example refactoring walkthrough, see Refactor your code using alias any type on the .NET blog.

[csharp_158] C# - Whats New in C#

What's new in C# 12: Inline arrays

Sumber: ht

Inline arrays

Inline arrays are used by the runtime team and other library authors to improve performance in your apps. Inline arrays enable a developer to create an array of fixed size in a `struct` type. A struct with an inline buffer should provide performance characteristics similar to an unsafe fixed size buffer. You likely won't declare your own inline arrays, but you use them transparently when they're exposed as <xref:System.Span`1?displayProperty=nameWithType> or <xref:System.ReadOnlySpan`1?displayProperty=nameWithType> objects from runtime APIs.

An *inline array* is declared similar to the following `struct`:

You use them like any other array:

The difference is that the compiler can take advantage of known information about an inline array. You likely consume inline arrays as you would any other array. For more information on how to declare inline arrays, see the language reference on `struct` types.

Contoh Kode / Perintah CLI:

```
[System.Runtime.CompilerServices.InlineArray(10)]
public struct Buffer
{
    private int _element0;
}

// ----------------------------------------

var buffer = new Buffer();
for (int i = 0; i < 10; i++)
{
    buffer[i] = i;
}

foreach (var i in buffer)
{
    Console.WriteLine(i);
}
```

[csharp_159] C# - Whats New in C#

What's new in C# 12: Experimental attribute

Sumber: ht

Experimental attribute

Types, methods, or assemblies can be marked with the <xref:System.Diagnostics.CodeAnalysis.ExperimentalAttribute?displayProperty=nameWithType> to indicate an experimental feature. The compiler issues a warning if you access a method or type annotated with the <xref:System.Diagnostics.CodeAnalysis.ExperimentalAttribute>. All types included in an assembly marked with the `Experimental` attribute are experimental. You can read more in the article on General attributes read by the compiler, or the feature specification.

[csharp_160] C# - Whats New in C#

What's new in C# 12: Interceptors

Sumber: ht

Interceptors
Interceptors are an experimental feature, available in preview mode with C# 12. The feature may be subject to breaking changes or removal in a future release. Therefore, it is not recommended for production or released applications.
In order to use interceptors, the user project must specify the property `<InterceptorsPreviewNamespaces>`. This is a list of namespaces which are allowed to contain interceptors.
For example: `<InterceptorsPreviewNamespaces>$(InterceptorsPreviewNamespaces);Microsoft.AspNetCore.Http.Generated;MyLibrary.Generated</InterceptorsPreviewNamespaces>`

An *interceptor* is a method that can declaratively substitute a call to an *interceptable* method with a call to itself at compile time. This substitution occurs by having the interceptor declare the source locations of the calls that it intercepts. Interceptors provide a limited facility to change the semantics of existing code by adding new code to a compilation, for example in a source generator.

You use an *interceptor* as part of a source generator to modify, rather than add code to an existing source compilation. The source generator substitutes calls to an interceptable method with a call to the *interceptor* method.

If you're interested in experimenting with interceptors, you can learn more by reading the feature specification. If you use the feature, make sure to stay current with any changes in the feature specification for this experimental feature. If the feature is finalized, we'll add more guidance on this site.

[csharp_161] C# - Whats New in C#

What's new in C# 12: See also

Sumber: ht

See also

- What's new in .NET 8

[csharp_162] C# - Whats New in C#

What's new in C# 13: Overview

Sumber: ht

What's new in C# 13

C# 13 includes the following new features. You can try these features using the latest Visual Studio 2022 version or the .NET 9 SDK:

- `params` collections
- New `lock` type and semantics.
- New escape sequence - `\e`.
- Method group natural type improvements
- Implicit indexer access in object initializers
- Enable `ref` locals and `unsafe` contexts in iterators and async methods
- Enable `ref struct` types to implement interfaces.
- Allow ref struct types as arguments for type parameters in generics.
- Partial properties and indexers are now allowed in `partial` types.
- Overload resolution priority allows library authors to designate one overload as better than others.

Beginning with Visual Studio 17.12, C# 13 includes the `field` contextual keyword as a preview feature.

C# 13 is supported on **.NET 9**. For more information, see C# language versioning.

You can download the latest .NET 9 SDK from the .NET downloads page. You can also download Visual Studio 2022, which includes the .NET 9 SDK.

You can find any breaking changes introduced in C# 13 in our article on breaking changes.

!INCLUDE [released-version-feedback]

[csharp_163] C# - Whats New in C#

What's new in C# 13: `params` collections

Sumber: ht

`params` collections

The `params` modifier isn't limited to array types. You can now use `params` with any recognized collection type, including <xref:System.Span`1?displayProperty=nameWithType>, <xref:System.ReadOnlySpan`1?displayProperty=nameWithType>, and types that implement <xref:System.Collections.Generic.IEnumerable`1?displayProperty=nameWithType> and have an `Add` method. In addition to concrete types, the interfaces <xref:System.Collections.Generic.IEnumerable`1?displayProperty=nameWithType>, <xref:System.Collections.Generic.IReadOnlyCollection`1?displayProperty=nameWithType>, <xref:System.Collections.Generic.IReadOnlyList`1?displayProperty=nameWithType>, <xref:System.Collections.Generic.ICollection`1?displayProperty=nameWithType>, and <xref:System.Collections.Generic.IList`1?displayProperty=nameWithType> can also be used.

When an interface type is used, the compiler synthesizes the storage for the arguments supplied. You can learn more in the C# language specification for `params` collections.

For example, method declarations can declare spans as `params` parameters:

Contoh Kode / Perintah CLI:

```
public void Concat<T>(params ReadOnlySpan<T> items)
{
    for (int i = 0; i < items.Length; i++)
    {
        Console.Write(items[i]);
        Console.Write(" ");
    }
    Console.WriteLine();
}
```

[csharp_164] C# - Whats New in C#

What's new in C# 13: New lock object

Sumber: ht

New lock object

The .NET 9 runtime includes a new type for thread synchronization, the <xref:System.Threading.Lock?displayProperty=fullName> type. This type provides better thread synchronization through its API. The <xref:System.Threading.Lock.EnterScope?displayProperty=nameWithType> method enters an exclusive scope. The `ref struct` returned from that supports the `Dispose()` pattern to exit the exclusive scope.

The C# `lock` statement recognizes if the target of the lock is a `Lock` object. If so, it uses the updated API, rather than the traditional API using <xref:System.Threading.Monitor?displayProperty=nameWithType>. The compiler also recognizes if you convert a `Lock` object to another type and the `Monitor` based code would be generated. You can read more in the C# language specification for the new lock object.

This feature allows you to get the benefits of the new library type by changing the type of object you `lock`. No other code needs to change.

[csharp_165] C# - Whats New in C#

What's new in C# 13: New escape sequence

Sumber: ht

New escape sequence

You can use `\e` as a character literal escape sequence for the `ESCAPE` character, Unicode `U+001B`. Previously, you used `\u001b` or `\x1b`. Using `\x1b` wasn't recommended because if the next characters following `1b` were valid hexadecimal digits, those characters became part of the escape sequence.

[csharp_166] C# - Whats New in C#

What's new in C# 13: Method group natural type

Sumber: ht

Method group natural type

This feature makes small optimizations to overload resolution involving method groups. A *method group* is a method and all overloads with the same name. The previous behavior was for the compiler to construct the full set of candidate methods for a method group. If a natural type was needed, the natural type was determined from the full set of candidate methods.

The new behavior is to prune the set of candidate methods at each scope, removing those candidate methods that aren't applicable. Typically, the removed methods are generic methods with the wrong arity, or constraints that aren't satisfied. The process continues to the next outer scope only if no candidate methods are found. This process more closely follows the general algorithm for overload resolution. If all candidate methods found at a given scope don't

match, the method group doesn't have a natural type.

You can read the details of the changes in the anonymous function type clause of the C# language specification.

[csharp_167] C# - Whats New in C#

What's new in C# 13: Implicit index access

Sumber: ht

Implicit index access

The implicit "from the end" index operator, `^`, is now allowed in an object initializer expression for single-dimension collections. For example, you can now initialize a single-dimension array using an object initializer as shown in the following code:

The `TimerRemaining` class includes a `buffer` array initialized to a length of 10. The preceding example assigns values to this array using the "from the end" index operator (`^`), effectively creating an array that counts down from 9 to 0.

In versions before C# 13, the `^` operator can't be used in an object initializer. You need to index the elements from the front.

Contoh Kode / Perintah CLI:

```
public class TimerRemaining
{
    public int[] buffer { get; set; } = new int[10];
}

var countdown = new TimerRemaining()
{
    buffer =
    {
        [^1] = 0,
        [^2] = 1,
        [^3] = 2,
        [^4] = 3,
        [^5] = 4,
        [^6] = 5,
        [^7] = 6,
        [^8] = 7,
        [^9] = 8,
        [^10] = 9
    }
};
```

[csharp_168] C# - Whats New in C#

What's new in C# 13: `ref` and `unsafe` in iterators and `async` methods

Sumber: ht

`ref` and `unsafe` in iterators and `async` methods

This feature and the following two features enable `ref struct` types to use new constructs. You won't use these features unless you write your own `ref struct` types. More likely, you see an indirect benefit as <xref:System.Span`1?displayProperty=nameWithType> and <xref:System.ReadOnlySpan`1?displayProperty=nameWithType> gain more functionality.

Before C# 13, iterator methods (methods that use `yield return`) and `async` methods couldn't declare local `ref` variables, nor could they have an `unsafe` context.

In C# 13, `async` methods can declare `ref` local variables, or local variables of a `ref struct` type. However, those variables can't be accessed across an `await` boundary. Neither can they be accessed across a `yield return` boundary.

This relaxed restriction enables the compiler to allow verifiably safe use of `ref` local variables and `ref struct` types in more places. You can safely use types like <xref:System.ReadOnlySpan`1?displayProperty=nameWithType> in these methods. The compiler tells you if you violate safety rules.

In the same fashion, C# 13 allows `unsafe` contexts in iterator methods. However, all `yield return` and `yield break` statements must be in safe contexts.

[csharp_169] C# - Whats New in C#

What's new in C# 13: `allows ref struct`

Sumber: ht

`allows ref struct`

Before C# 13, `ref struct` types couldn't be declared as the type argument for a generic type or method. Now, generic type declarations can add an anti-constraint, `allows ref struct`. This anti-constraint declares that the type argument supplied for that type parameter can be a `ref struct` type. The compiler enforces ref safety rules on all instances of that type parameter.

For example, you might declare a generic type like the following code:

This enables types such as <xref:System.Span`1?displayProperty=nameWithType> and <xref:System.ReadOnlySpan`1?displayProperty=nameWithType> to be used with generic algorithms, where applicable. You can learn more in the updates for `where` and the programming guide article on generic constraints.

Contoh Kode / Perintah CLI:

```
public class C<T> where T : allows ref struct
{
    // Use T as a ref struct:
    public void M(scoped T p)
    {
        // The parameter p must follow ref safety rules
    }
}
```

[csharp_170] C# - Whats New in C#

What's new in C# 13: `ref struct` interfaces

Sumber: ht

`ref struct` interfaces

Before C# 13, `ref struct` types weren't allowed to implement interfaces. Beginning with C# 13, they can. You can declare that a `ref struct` type implements an interface. However, to ensure ref safety rules, a `ref struct` type can't be converted to an interface type. That conversion is a boxing conversion, and could violate ref safety. Explicit interface method declarations in a `ref struct` can be accessed only through a type parameter where that type parameter `allows ref struct`. Also, `ref struct` types must implement all methods declared in an interface, including those methods with a default implementation.

Learn more in the updates on `ref struct` types and the addition of the `allows ref struct` generic constraint.

[csharp_171] C# - Whats New in C#

What's new in C# 13: More partial members

Sumber: ht

More partial members

You can declare `partial` properties and `partial` indexers in C# 13. Partial properties and indexers generally follow the same rules as `partial` methods: you create one *declaring declaration* and one *implementing declaration*. The signatures of the two declarations must match. One restriction is that you can't use an auto-property declaration for *implementing* a partial property. Properties that don't declare a body are considered as the *declaring declaration*.

You can learn more in the article on partial members.

Contoh Kode / Perintah CLI:

```
public partial class C
{
    // Declaring declaration
    public partial string Name { get; set; }
}

public partial class C
{
    // implementation declaration:
    private string _name;
    public partial string Name
    {
        get => _name;
        set => _name = value;
    }
}
```

[csharp_172] C# - Whats New in C#

What's new in C# 13: Overload resolution priority

Sumber: ht

Overload resolution priority

In C# 13, the compiler recognizes the <xref:System.Runtime.CompilerServices.OverloadResolutionPriorityAttribute> to prefer one overload over another. Library authors can use this attribute to ensure that a new, better overload is preferred over an existing overload. For example, you might add a new overload that's more performant. You don't want to break existing code that uses your library, but you want users to update to the new version when they recompile. You can use Overload resolution priority to inform the compiler which overload should be preferred. Overloads with the highest priority are preferred.

This feature is intended for library authors to avoid ambiguity when adding new overloads. Library authors should use care with this attribute to avoid confusion.

[csharp_173] C# - Whats New in C#

What's new in C# 13: The `field` keyword

Sumber: ht

The `field` keyword

The `field` contextual keyword is in C# 13 as a preview feature. The token `field` accesses the compiler synthesized backing field in a property accessor. It enables you to write an accessor body without declaring an explicit backing field in your type declaration. You can declare a body for one or both accessors for a field backed property.

The `field` feature is released as a preview feature. We want to learn from your experiences using it. There's a potential breaking change or confusion reading code in types that also include a field named `field`. You can use `@field` or `this.field` to disambiguate between the `field` keyword and the identifier.
You should be careful using the `field` keyword feature in a class that has a field named `field`. The new `field` keyword shadows a field named `field` in the scope of a property accessor. You can either change the name of the `field` variable, or use the `@` token to reference the `field` identifier as `@field`. You can learn more by reading the feature specification for the `field` keyword.

If you try this feature and have feedback, add it to the feature issue in the `csharplang` repository.

[csharp_174] C# - Whats New in C#

What's new in C# 13: See also

Sumber: ht

See also

- What's new in .NET 9